

Supplemental Materials for Full Datasets for Data, rather than algorithms, constrains the predictability of binary classification

Fei Jing,¹ Zi-Ke Zhang,^{2,*} Yi-Cheng Zhang,^{3,4,†} and Qingpeng Zhang^{1,‡}

¹*Musketeers Foundation Institute of Data Science,*

The University of Hong Kong, Hong Kong SAR, China

²*Center for Digital Communication Studies, Zhejiang University, Hangzhou 310058, China*

³*Department of Physics, University of Fribourg, Chemin du Musée 3, 1700 Fribourg, Switzerland*

⁴*School of Physics, Zhejiang University, Hangzhou 310058, China*

CONTENTS

I. Preliminaries	3
II. Objective Functions	4
A. Optimal square loss and statistical ensemble on square cost function	4
1. Optimal square loss	4
2. Statistical ensemble on square loss function	5
B. Optimal logistic loss and statistical ensemble on logistic cost function	5
1. Optimal logistic loss	5
2. Statistical ensemble on logistic cost function	6
C. Optimal Hinge loss and statistical ensemble on Hinge cost function	7
1. Optimal Hinge loss	7
2. Statistical ensemble on Hinge cost function	7
D. Optimal Softmax loss and statistical ensemble on Softmax cost function	8
1. Optimal Softmax loss	8
2. Statistical ensemble on Softmax cost function	9
III. Evaluation Measurements and Statistical Ensembles	9
A. Optimal ROC Curve and statistical ensemble on AR cost function	9
1. Optimal ROC Curve	9
2. Statistical ensemble on AR cost function	12
B. Optimal PR Curve and statistical ensemble on AP cost function	13
1. Optimal PR Curve	13
2. statistical ensemble on AP cost function	14
C. Optimal Accuracy and statistical ensemble on AC cost function	15
1. Optimal Accuracy	15
2. statistical ensemble on AC cost function	15
IV. Universal Optimal Classifier	17
V. Sensitivity Analysis	17
VI. Random Division	18
VII. Overlapping and Boundary	19
VIII. Feature Engineering	22
A. Feature Selection	23
B. Feature Extraction	24
C. Feature Generated from Other Samples	24

* zkz@zju.edu.cn

† yi-cheng.zhang@unifr.ch

‡ qpzhang@hku.hk

42	IX. Datasets	25
43	References	27
44	Supplementary Tables and Figures	27

I. PRELIMINARIES

Binary classification involves assigning elements of a set into one of two groups, or classes, based on a classification rule. This process is crucial in various applications such as medical diagnostics, quality control, and information retrieval. In machine learning and data analysis, binary classification is a supervised learning task aimed at predicting one of two possible outcomes for a given input.

In this context, the data is labeled as positive (often denoted by 1) or negative (denoted by -1). The objective is to construct a predictive model capable of accurately classifying new, unseen instances into these categories by learning from patterns in the training data. Initially, a labeled dataset is gathered, where each instance is associated with a known class label. The dataset is then divided into a training set for model development and a test set for evaluating performance on new data. During training, the model identifies patterns that distinguish between the classes. Techniques such as XGBoost, MLP, SVM, LR, DT, RF, KNN, and Naïve Bayes can be utilized for binary classification.

Let's consider a binary classification scenario where the goal is to predict a binary label. An input-output pair is represented as $z = (x, y)$, where $x \in \mathcal{X}$ is the feature vector (input data) and $y \in \{1, -1\}$ is the class label. Given a training set $\mathcal{S} = \{(x_i, y_i) | i = 1, 2, \dots, m\}$, we define $\mathcal{S}_+$ as the subset containing n_+ positive samples and $\mathcal{S}_-$ as the subset with n_- negative samples, such that the total number of instances is $m = |\mathcal{S}| = n_+ + n_-$. Let $\mathcal{P}(x_i)$ and $\mathcal{N}(x_i)$ denote the counts of positive and negative instances for a given feature vector x_i .

Classifiers are mappings that assign instances to specific classes. Some produce a continuous output, allowing various thresholds to define class membership—these are known as **continuous classifiers**. They combine a classification function $f(x) : \mathcal{X} \rightarrow \mathbb{R}$ with a threshold t to translate scores into binary classes. Others yield a discrete class label—these are **discrete classifiers** and are described by the function $f(x) : \mathcal{X} \rightarrow \{1, -1\}$. Logistic regression and neural networks are examples of continuous classifiers, while SVM, decision trees, and random forests are discrete classifiers.

Objective functions gauge the alignment between model predictions and actual labels. During training, the aim is to minimize the difference between these predictions and true labels. We discuss several objective functions for binary classification problems:

- Square loss function: $\min_f \frac{1}{m} \sum_{x_i} \left(f(x_i) - y_i \right)^2$;
- Logistic loss function: $\min_f \frac{1}{m} \sum_{x_i} -y_i \log f(x_i) - (1 - y_i) \log (1 - f(x_i))$;
- Hinge loss function: $\min_f \frac{1}{2m} \sum_{x_i} \max \left\{ 0, 1 - f(x_i)y_i \right\}$;
- Softmax function: $\min_f \frac{1}{m} \sum_{x_i} -\log f(x_i) - \log (1 - f(x_i))$.

Square and Logistic loss functions are typically suited for continuous classifiers, while Hinge and Softmax losses can be applied to both continuous and discrete classifiers.

For binary classification outcomes, we consider the instances to be either positive or negative, leading to four potential results from the classifier:

- TP (True Positive): $\sum_{x_i \in \mathcal{S}} \frac{\mathcal{P}(x_i)}{2} \left(f(x_i) + 1 \right)$;
- FP (False Positive): $\sum_{x_i \in \mathcal{S}} \frac{\mathcal{N}(x_i)}{2} \left(f(x_i) + 1 \right)$;
- FN (False Negative): $\sum_{x_i \in \mathcal{S}} \frac{\mathcal{P}(x_i)}{2} \left(1 - f(x_i) \right)$;
- TN (True Negative): $\sum_{x_i \in \mathcal{S}} \frac{\mathcal{N}(x_i)}{2} \left(1 - f(x_i) \right)$.

II. OBJECTIVE FUNCTIONS

The objective function in the training process of a classification model serves as a guide for parameter adjustment to fit the dataset. Here, we discuss four commonly used objective functions and show a direct correlation between their optimal solutions [1] and the dataset through discrete analysis.

A. Optimal square loss and statistical ensemble on square cost function

1. Optimal square loss

Given a dataset $\mathcal{S}$ with feature domain $\mathcal{X}$, and assuming $f(x)$ as a continuous classification function with parameters to be trained, the square error loss function is given by:

$$\min_f \frac{1}{m} \sum_{x_i} \left(f(x_i) - y_i \right)^2. \quad (1)$$

The optimal solution for this objective function is expressed as:

$$f_{\text{Square}}^* = \arg \min_f \frac{1}{m} \sum_{x_i} \left(f(x_i) - y_i \right)^2. \quad (2)$$

It's evident that:

$$\min_f \sum_{x_i} \left(f(x_i) - y_i \right)^2 \geq \sum_{x_i} \min_{f(x_i)} \left(f(x_i) - y_i \right)^2. \quad (3)$$

Moreover, equality holds:

$$\min_f \sum_{x_i} \left(f(x_i) - y_i \right)^2 = \sum_{x_i} \min_{f(x_i)} \left(f(x_i) - y_i \right)^2. \quad (4)$$

if and only if $f(x_i)$ and $f(x_j)$ are nearly independent for any $x_i \neq x_j \in \mathcal{S}$. In the binary classification context, the square loss function can be transformed to:

$$\begin{aligned} \min_f \sum_{x_i} \left(f(x_i) - y_i \right)^2 &= \sum_{x_i} \min_{f(x_i)} \left(f(x_i) - y_i \right)^2 \\ &= \sum_{x_i} \min_{f(x_i)} \mathcal{P}(x_i) \left(f(x_i) - 1 \right)^2 + \mathcal{N}(x_i) \left(f(x_i) + 1 \right)^2. \end{aligned} \quad (5)$$

The optimal solution becomes:

$$f_{\text{Square}}^*(x_i) = \arg \min_{f(x_i)} \mathcal{P}(x_i) \left(f(x_i) - 1 \right)^2 + \mathcal{N}(x_i) \left(f(x_i) + 1 \right)^2 \quad (6)$$

and is simply:

$$f_{\text{Square}}^*(x_i) = \frac{\mathcal{P}(x_i) - \mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} \quad (7)$$

for each $x_i \in \mathcal{S}$. Finally, we have

$$\text{minimum square loss} = \frac{4}{m} \sum_{x_i} \frac{\mathcal{P}(x_i) \mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}. \quad (8)$$

2. Statistical ensemble on square loss function

In this context, every classifier f can be transformed into a n -dimensional vector

$$\pi^f = (x_1^f, x_2^f, \dots, x_n^f) \quad (9)$$

in which $x_i^f = f(x_i)$ for each $x_i \in \mathcal{S}$. Next, the energy of π^f is defined as

$$\begin{aligned} E^{Square}(\pi^f) &= \frac{1}{m} \sum_{x_i} \left(\pi_i^f - y_i \right)^2 \\ &= \frac{1}{m} \sum_{x_i} \mathcal{P}(x_i) \left(\pi_i^f - 1 \right)^2 + \mathcal{N}(x_i) \left(\pi_i^f + 1 \right)^2. \end{aligned} \quad (10)$$

Let $\Pi(\mathcal{S}) = \mathbb{R}^n$. Then, the partition function is

$$\begin{aligned} Z^{Square} &= \sum_{\pi_f \in \Pi(\mathcal{S})} \exp(-\beta E^{Square}(\pi^f)) \\ &= \int \prod_{x_i^f} d\pi_i^f \exp \left\{ -\beta \left(\frac{1}{m} \sum_{x_i} \mathcal{P}(x_i) \left(\pi_i^f - 1 \right)^2 + \mathcal{N}(x_i) \left(\pi_i^f + 1 \right)^2 \right) \right\} \end{aligned} \quad (11)$$

And, the ground state is calculated as

$$\begin{aligned} E_0^{Square} &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{Square} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int \prod_{x_i^f} d\pi_i^f \exp \left\{ -\beta \left(\frac{1}{m} \sum_{x_i} \mathcal{P}(x_i) \left(\pi_i^f - 1 \right)^2 + \mathcal{N}(x_i) \left(\pi_i^f + 1 \right)^2 \right) \right\} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int \prod_{x_i^f} d\pi_i^f \prod_{x_i} \exp \left\{ -\frac{\beta}{m} \left\{ \mathcal{P}(x_i) \left(\pi_i^f - 1 \right)^2 + \mathcal{N}(x_i) \left(\pi_i^f + 1 \right)^2 \right\} \right\} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \prod_{x_i^f} \int d\pi_i^f \exp \left\{ -\frac{\beta}{m} \left\{ \mathcal{P}(x_i) \left(\pi_i^f - 1 \right)^2 + \mathcal{N}(x_i) \left(\pi_i^f + 1 \right)^2 \right\} \right\} \\ &= \sum_{x_i} \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int d\pi_i^f \exp \left\{ -\frac{\beta}{m} \left\{ \mathcal{P}(x_i) \left(\pi_i^f - 1 \right)^2 + \mathcal{N}(x_i) \left(\pi_i^f + 1 \right)^2 \right\} \right\}. \end{aligned} \quad (12)$$

Utilizing Gaussian transformation, we finally obtain

$$E_0^{Square} = \frac{4}{m} \sum_{x_i} \frac{\mathcal{P}(x_i) \mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}. \quad (13)$$

B. Optimal logistic loss and statistical ensemble on logistic cost function

1. Optimal logistic loss

The Logistic loss function, frequently used for binary classification, is defined for a dataset $\mathcal{S}$ and feature domain $\mathcal{X}$ as:

$$\min_f \frac{1}{m} \sum_{x_i} -y_i \log f(x_i) - (1 - y_i) \log (1 - f(x_i)), \quad (14)$$

with the optimal solution:

$$f_{\text{Logistic}}^* = \arg \min_f \frac{1}{m} \sum_{x_i} -y_i \log f(x_i) - (1 - y_i) \log (1 - f(x_i)). \quad (15)$$

110 Close to the optimal, the logistic loss function can be approximated as:

$$\min_f \frac{1}{m} \sum_{x_i} \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log f(x_i) - 2\mathcal{N}(x_i) \log \left(1 - f(x_i) \right). \quad (16)$$

111 The corresponding optimal solution then is:

$$f_{\text{Logistic}}^*(x_i) = \arg \min_{f(x_i)} \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log f(x_i) - 2\mathcal{N}(x_i) \log \left(1 - f(x_i) \right) \quad (17)$$

112 for each $x_i \in \mathcal{S}$. Finally, it can be simplified as

$$f_{\text{Logistic}}^*(x_i) = \frac{\mathcal{P}(x_i) - \mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} \quad (18)$$

113 for each $x_i \in \mathcal{S}$.

114 2. Statistical ensemble on logistic cost function

115 In this context, the energy of π^f is defined as

$$E^{\text{Logistic}}(\pi^f) = \frac{1}{m} \sum_{x_i} \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log \pi_i^f - 2\mathcal{N}(x_i) \log \left(1 - \pi_i^f \right). \quad (19)$$

116 Let $\Pi(\mathcal{S}) = \mathbb{R}^n$. Then, the partition function is

$$\begin{aligned} Z^{\text{Logistic}} &= \sum_{\pi_f \in \Pi(\mathcal{S})} \exp(-\beta E^{\text{Logistic}}(\pi^f)) \\ &= \int \prod_{x_i^f} d\pi_i^f \exp \left\{ -\beta \left(\frac{1}{m} \sum_{x_i} \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log \pi_i^f - 2\mathcal{N}(x_i) \log \left(1 - \pi_i^f \right) \right) \right\}. \end{aligned} \quad (20)$$

117 And, the ground state is calculated as

$$\begin{aligned} E_0^{\text{Logistic}} &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{\text{Logistic}} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int_0^1 \prod_{x_i^f} d\pi_i^f \exp \left\{ -\beta \left(\frac{1}{m} \sum_{x_i} \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log \pi_i^f - 2\mathcal{N}(x_i) \log \left(1 - \pi_i^f \right) \right) \right\} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int_0^1 \prod_{x_i^f} d\pi_i^f \prod_{x_i} \exp \left\{ -\frac{\beta}{m} \left\{ \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log \pi_i^f - 2\mathcal{N}(x_i) \log \left(1 - \pi_i^f \right) \right\} \right\} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \prod_{x_i} \int_0^1 d\pi_i^f \exp \left\{ -\frac{\beta}{m} \left\{ \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log \pi_i^f - 2\mathcal{N}(x_i) \log \left(1 - \pi_i^f \right) \right\} \right\} \\ &= \sum_{x_i} \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int_0^1 d\pi_i^f \exp \left\{ -\frac{\beta}{m} \left\{ \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log \pi_i^f - 2\mathcal{N}(x_i) \log \left(1 - \pi_i^f \right) \right\} \right\} \\ &= \sum_{x_i} \min_{\pi_i^f \in (0,1)} \frac{1}{m} \left\{ \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \log \pi_i^f - 2\mathcal{N}(x_i) \log \left(1 - \pi_i^f \right) \right\} \\ &= \frac{1}{m} \sum_{x_i} \left(\mathcal{N}(x_i) - \mathcal{P}(x_i) \right) \ln \frac{\mathcal{P}(x_i) - \mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} - 2\mathcal{N}(x_i) \ln \frac{2\mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}, \end{aligned} \quad (21)$$

118 in which

$$f_{\text{Logistic}}^*(x_i) = \frac{\mathcal{P}(x_i) - \mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} \quad (22)$$

119 for each $x_i \in \mathcal{S}$.

C. Optimal Hinge loss and statistical ensemble on Hinge cost function

1. Optimal Hinge loss

The hinge loss function is commonly used in Support Vector Machines (SVM) and other models employing maximum-margin classifiers, regardless of whether the classification task is discrete or continuous. It penalizes misclassified instances based on their distance from the decision boundary. The Hinge loss function for binary classification is described as follows:

$$\min_f \frac{1}{2m} \sum_{i=1}^m \max\{0, 1 - y_i f(x_i)\}, \quad (23)$$

and its optimal solution is

$$f_{\text{Hinge}}^* = \arg \min_f \frac{1}{2m} \sum_{i=1}^m \max\{0, 1 - y_i f(x_i)\}. \quad (24)$$

Unlike other objective functions, the output for any solution of the hinge loss function is discrete. We employ a similar technique to the hinge loss function as before,

$$\min_{f(x_i) \in \{1, -1\}} \mathcal{P}(x_i) \max\{0, 1 - f(x_i)\} + \mathcal{N}(x_i) \max\{0, 1 + f(x_i)\} \quad (25)$$

for any x_i , where its optimal solution can also be expressed as

$$f_{\text{Hinge}}^*(x_i) = \arg \max_{f(x_i) \in \{1, -1\}} \mathcal{P}(x_i) \max\{0, 1 - f(x_i)\} + \mathcal{N}(x_i) \max\{0, 1 + f(x_i)\} \quad (26)$$

for each $x_i \in \mathcal{S}$. Since $f(x_i)$ must be equal to 1 or -1 , we can rewrite the optimal solution as

$$f_{\text{Hinge}}^*(x_i) = \begin{cases} 1, & \text{if } \mathcal{P}(x_i) \geq \mathcal{N}(x_i), \\ -1, & \text{if } \mathcal{P}(x_i) < \mathcal{N}(x_i), \end{cases} \quad (27)$$

for each $x_i \in \mathcal{S}$. And,

$$\text{minimum Hinge loss} = \frac{1}{m} \sum_{x_i} \min \left\{ \mathcal{P}(x_i), \mathcal{N}(x_i) \right\}. \quad (28)$$

2. Statistical ensemble on Hinge cost function

In this context, the energy of π^f is defined as

$$E^{\text{Hinge}}(\pi^f) = \frac{1}{2m} \sum_{x_i} \mathcal{P}(x_i) \max\{0, 1 - \pi_i^f\} + \mathcal{N}(x_i) \max\{0, 1 + \pi_i^f\}, \quad (29)$$

where $\pi_i^f \in \{1, -1\}$ for each x_i . Let $\Pi(\mathcal{S}) = \{1, -1\}^n$. Then, the partition function is

$$\begin{aligned} Z^{\text{Hinge}} &= \sum_{\pi_f \in \Pi(\mathcal{S})} \exp(-\beta E^{\text{Hinge}}(\pi^f)) \\ &= \sum_{\pi_f \in \Pi(\mathcal{S})} \exp \left\{ -\beta \left(\frac{1}{2m} \sum_{x_i} \mathcal{P}(x_i) \max\{0, 1 - \pi_i^f\} + \mathcal{N}(x_i) \max\{0, 1 + \pi_i^f\} \right) \right\}. \end{aligned} \quad (30)$$

135 And, the ground state is calculated as

$$\begin{aligned}
E_0^{Hinge} &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{Hinge} \\
&= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \sum_{\pi_f \in \Pi(\mathcal{S})} \exp \left\{ -\beta \left(\frac{1}{2m} \sum_{x_i} \mathcal{P}(x_i) \max\{0, 1 - \pi_i^f\} + \mathcal{N}(x_i) \max\{0, 1 + \pi_i^f\} \right) \right\} \\
&= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \sum_{\pi_f \in \Pi(\mathcal{S})} \prod_{x_i} e^{-\frac{\beta}{2m} \left(\mathcal{P}(x_i) \max\{0, 1 - \pi_i^f\} + \mathcal{N}(x_i) \max\{0, 1 + \pi_i^f\} \right)} \\
&= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \prod_{x_i} \left(e^{-\beta \mathcal{N}(x_i)/m} + e^{-\beta \mathcal{P}(x_i)/m} \right) \\
&= \sum_{x_i} \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \left(e^{-\beta \mathcal{N}(x_i)/m} + e^{-\beta \mathcal{P}(x_i)/m} \right) \\
&= \frac{1}{m} \sum_{x_i} \min \left\{ \mathcal{P}(x_i), \mathcal{N}(x_i) \right\}.
\end{aligned} \tag{31}$$

136

D. Optimal Softmax loss and statistical ensemble on Softmax cost function

137

1. Optimal Softmax loss

138 The Softmax loss function, also known as the cross-entropy loss or log-likelihood loss, is essential in machine learning
 139 and deep learning, particularly for classification tasks. It measures the dissimilarity between predicted class proba-
 140 bilities and true class labels. The softmax function is typically used in conjunction with this loss function to convert
 141 raw model outputs into probability distributions over multiple classes. Mathematically, for a feature x , the softmax
 142 function computes the probability of the positive class as follows:

$$f(x) = \frac{e^{z_+}}{e^{z_+} + e^{z_-}}. \tag{32}$$

143 Here, z_+ (z_-) denotes the score that measures the likelihood of the data with feature x belonging to the positive
 144 (negative) class.

145 The softmax loss function is defined as the negative log-likelihood of the true class in binary classification tasks:

$$\min_f \frac{1}{m} \sum_{i=1}^m \left[-y_i \log f(x_i) - (1 - y_i) \log(1 - f(x_i)) \right], \tag{33}$$

146 with the optimal solution being

$$f_{\text{Softmax}}^* = \arg \min_f \frac{1}{m} \sum_{i=1}^m \left[-y_i \log f(x_i) - (1 - y_i) \log(1 - f(x_i)) \right]. \tag{34}$$

147 We can simplify the objective function as

$$\min_{f(x_i)} -\mathcal{P}(x_i) \log f(x_i) - \mathcal{N}(x_i) \log(1 - f(x_i)), \tag{35}$$

148 and its optimal solution as

$$f_{\text{Softmax}}^*(x_i) = \arg \min_{f(x_i)} -\mathcal{P}(x_i) \log f(x_i) - \mathcal{N}(x_i) \log(1 - f(x_i)), \tag{36}$$

149 for every $x_i \in \mathcal{S}$. By performing the necessary derivations, we find the optimal solution for the Softmax loss function
 150 as

$$f_{\text{Softmax}}^*(x_i) = \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}, \tag{37}$$

151 for each $x_i \in \mathcal{S}$.

2. Statistical ensemble on Softmax cost function

In this context, the energy of π^f is defined as

$$E^{Softmax}(\pi^f) = \frac{1}{m} \sum_{i=1}^m -\mathcal{P}(x_i) \log \pi_i^f - \mathcal{N}(x_i) \log(1 - \pi_i^f), \quad (38)$$

Let $\Pi(\mathcal{S}) = \mathbb{R}^n$. Then, the partition function is

$$\begin{aligned} Z^{Softmax} &= \sum_{\pi_f \in \Pi(\mathcal{S})} \exp(-\beta E^{Softmax}(\pi^f)) \\ &= \int \prod_{x_i^f} d\pi_i^f \exp \left\{ -\beta \left(\frac{1}{m} \sum_{i=1}^m -\mathcal{P}(x_i) \log \pi_i^f - \mathcal{N}(x_i) \log(1 - \pi_i^f) \right) \right\}. \end{aligned} \quad (39)$$

And, the ground state is calculated as

$$\begin{aligned} E_0^{Softmax} &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{Softmax} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int_0^1 \prod_{x_i^f} d\pi_i^f \exp \left\{ -\beta \left(\frac{1}{m} \sum_{x_i} -\mathcal{P}(x_i) \log \pi_i^f - \mathcal{N}(x_i) \log(1 - \pi_i^f) \right) \right\} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int_0^1 \prod_{x_i^f} d\pi_i^f \prod_{x_i} \exp \left\{ -\frac{\beta}{m} \left\{ -\mathcal{P}(x_i) \log \pi_i^f - \mathcal{N}(x_i) \log(1 - \pi_i^f) \right\} \right\} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \prod_{x_i} \int_0^1 d\pi_i^f \exp \left\{ -\frac{\beta}{m} \left\{ -\mathcal{P}(x_i) \log \pi_i^f - \mathcal{N}(x_i) \log(1 - \pi_i^f) \right\} \right\} \\ &= \sum_{x_i} \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \int_0^1 d\pi_i^f \exp \left\{ -\frac{\beta}{m} \left\{ -\mathcal{P}(x_i) \log \pi_i^f - \mathcal{N}(x_i) \log(1 - \pi_i^f) \right\} \right\} \\ &= \sum_{x_i} \min_{\pi_i^f \in (0,1)} \frac{1}{m} \left\{ -\mathcal{P}(x_i) \log \pi_i^f - \mathcal{N}(x_i) \log(1 - \pi_i^f) \right\} \\ &= \frac{1}{m} \sum_{x_i} -\mathcal{P}(x_i) \ln \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} - \mathcal{N}(x_i) \ln \frac{\mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}, \end{aligned} \quad (40)$$

in which

$$f_{\text{Logistic}}^*(x_i) = \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} \quad (41)$$

for each $x_i \in \mathcal{S}$.

III. EVALUATION MEASUREMENTS AND STATISTICAL ENSEMBLES

The evaluation of the performance of binary classifiers involves various metrics, among which the Receiver Operating Characteristic (ROC) curve and the Precision-Recall (PR) curve are prominent. These metrics offer insights into the effectiveness of a classifier at different threshold settings. This note delves into the optimal ROC and PR curves [2, 3], providing a mathematical exposition of their derivation and interpretation in the context of a given dataset.

A. Optimal ROC Curve and statistical ensemble on AR cost function

1. Optimal ROC Curve

The ROC curve represents the trade-off between the true positive rate (TPR) and the false positive rate (FPR) of a classifier. The curve is constructed by plotting TPR against FPR at various threshold levels. An optimal ROC curve approaches the top-left corner of the plot, indicating both high TPR and low FPR.

Consider a dataset $\mathcal{S} = \{x_1, x_2, \dots, x_m\}$ with a feature domain $\mathcal{X}$. A classifier f assigns a score to each instance in $\mathcal{S}$, resulting in a sorted sequence $\mathcal{S}^f = \{x_1^f, x_2^f, \dots, x_m^f\}$, where $f(x_i^f) \geq f(x_j^f)$ for $i < j$. The real label corresponding to x_i^f is denoted as y_i^f . To construct the ROC curve, one must calculate the TPR and FPR at various thresholds $t \in \{1, \dots, m\}$ where each $t = k$ indicates that instances $x_1^f, \dots, x_k^f$ are classified as positive. The TPR and FPR are given by:

$$\text{TPR} = \frac{\text{TP}}{n_+} = \frac{1}{n_+} \sum_{i=1}^k \frac{1}{2} (1 + y_i^f), \quad (42)$$

$$\text{FPR} = \frac{\text{FP}}{n_-} = \frac{1}{n_-} \sum_{i=1}^k \frac{1}{2} (1 - y_i^f). \quad (43)$$

To find the optimal ROC curve, the objective is to maximize TPR and minimize FPR for each threshold k . This bi-objective optimization can be expressed as:

$$\begin{aligned} \max_f \quad & \sum_{i=1}^k \frac{1}{2} (1 + y_i^f), \\ \min_f \quad & \sum_{i=1}^k \frac{1}{2} (1 - y_i^f). \end{aligned} \quad (44)$$

Considering that the sum of true positives and false positives equals k , the optimization problem in Eq. (44) simplifies to:

$$\max \quad \sum_{i=1}^k \frac{1}{2} (1 + y_i^f). \quad (45)$$

The calculation of FPR for a fixed k includes instances that either exceed or equal the score $f(x_k^f)$. However, the optimization is primarily concerned with the maximization of true positives. Therefore we have

$$\sum_{i=1}^k \frac{1}{2} (1 + y_i^f) = \sum_{i=1}^m \frac{1}{2} \mathbb{I}(f(x_i) > f(x_k^f)) (1 + y_i) + \frac{\alpha}{2} \mathbb{I}(f(x_i) = f(x_k^f)) (1 + y_i), \quad (46)$$

in which $\alpha = \frac{k - \sum_{i=1}^m \mathbb{I}(f(x_i) > f(x_k^f))}{\sum_{i=1}^m \mathbb{I}(f(x_i) = f(x_k^f))}$ is the ratio of instances with score $f(x_k^f)$ in the subset $\{x_1^f, x_2^f, \dots, x_k^f\}$ to all instances with score $f(x_k^f)$. Without loss of generality, we assume that $f(x_i) = f(x_j) \iff x_i = x_j$ for every $x_i, x_j \in \mathcal{S}$. Then we obtain that

$$\begin{aligned} & \sum_{i=1}^k \frac{1}{2} (1 + y_i^f) \\ &= \sum_{x_i \in \mathcal{S}} \mathbb{I}(f(x_i) > f(x_k^f)) \mathcal{P}(x_i) + \alpha \sum_{x_i \in \mathcal{S}} \mathbb{I}(f(x_i) = f(x_k^f)) \mathcal{P}(x_i) \\ &= \sum_{i=1}^m \mathbb{I}(f(x_i) > f(x_k^f)) \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} + \alpha \sum_{i=1}^m \mathbb{I}(f(x_i) = f(x_k^f)) \frac{\mathcal{P}(x_k^f)}{\mathcal{P}(x_k^f) + \mathcal{N}(x_k^f)} \\ &= \sum_{i=1}^m \mathbb{I}(f(x_i) > f(x_k^f)) \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} + \left(k - \sum_{i=1}^m \mathbb{I}(f(x_i) > f(x_k^f)) \right) \frac{\mathcal{P}(x_k^f)}{\mathcal{P}(x_k^f) + \mathcal{N}(x_k^f)} \\ &= \sum_{i=1}^m \frac{\mathcal{P}(x_i^f)}{\mathcal{P}(x_i^f) + \mathcal{N}(x_i^f)}. \end{aligned} \quad (47)$$

Denote $w_i = \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}$ as the non-negative weight of instance x_i . Hence, the optimization problem (45) can be regarded as a problem of how to choose a k -elements subset of $\mathcal{S}$ which satisfies that the total weight is maximum. And it is naturally rewritten in the form of combinatorial optimization as follows,

$$\begin{aligned} \max \quad & \sum_{i=1}^m w_i z_i \\ \text{s.t.} \quad & \sum_{i=1}^m z_i = k \\ & z_i \in \{0, 1\}, \quad i = 1, 2, \dots, m \end{aligned} \quad (48)$$

Actually, this combinatorial optimization problem belongs to the classical 0-1 knapsack problems, which is the most common problem being solved. Noting that all weights are non-negative, simple greedy algorithm can reach the optimal solution if we select the top k instances with the highest weight. As a rule of how to select optimal k -element subset with arbitrary k , the optimal classifier for best ROC curve is the weight function, that is,

$$f_{\text{ROC}}^*(x_i) = \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)} \quad (49)$$

for every $x_i \in \mathcal{S}$. Here, we denote that $\{x_1^*, x_2^*, \dots, x_n^*\}$ is the sorted sequence of $\mathcal{S}$ in descending order of weight. Naturally, the best ROC curve can be drawn sequentially from a series of data points in (FPR, TPR)-plane as follows,

$$\left\{ \left(\frac{1}{n_-} \sum_{i=1}^k \frac{\mathcal{N}(x_i^*)}{\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)}, \frac{1}{n_+} \sum_{i=1}^k \frac{\mathcal{P}(x_i^*)}{\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)} \right) \right\}_{k=0,1,\dots,n}. \quad (50)$$

The optimal ROC curve can be regarded as a combination of n linear piecewise functions, whose derivatives are composed of the following sequence

$$\left\{ \frac{n_- \mathcal{P}(x_i^*)}{n_+ \mathcal{N}(x_i^*)} \right\}_{k=1,2,\dots,n}. \quad (51)$$

It is easy to check that, the above sequence is monotonically decreasing, since $\{x_1^*, x_2^*, \dots, x_n^*\}$ is sorted by descending order of weight and

$$\frac{\mathcal{P}(x_i^*)}{\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)} = \frac{1}{1 + \frac{1}{\frac{\mathcal{P}(x_i^*)}{\mathcal{N}(x_i^*)}}}. \quad (52)$$

Therefore, the best ROC curve is **concave**. Moreover, the area under the best ROC curve is the upper bound of AR (textAR^u). In other words, AR^u is equal to the area under the curve described as f_{ROC}^* in the following:

$$\begin{aligned} \text{AR}^u &= \sum_{i=1}^m \frac{1}{n_-} \frac{\mathcal{N}(x_i^*)}{\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)} \left(\frac{1}{n_+} \sum_{j < i} \frac{\mathcal{P}(x_j^*)}{\mathcal{P}(x_j^*) + \mathcal{N}(x_j^*)} + \frac{1}{2n_+} \frac{\mathcal{P}(x_i^*)}{\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)} \right) \\ &= \frac{1}{n_- n_+} \sum_{i > j} \frac{\mathcal{N}(x_i^*) \mathcal{P}(x_j^*)}{(\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)) (\mathcal{P}(x_j^*) + \mathcal{N}(x_j^*))} + \frac{1}{2} \sum_{i=1}^m \frac{\mathcal{N}(x_i^*) \mathcal{P}(x_i^*)}{(\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*))^2} \\ &= \frac{1}{2n_- n_+} \sum_{i,j} \frac{\max \{ \mathcal{N}(x_i^*) \mathcal{P}(x_j^*), \mathcal{N}(x_j^*) \mathcal{P}(x_i^*) \}}{(\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)) (\mathcal{P}(x_j^*) + \mathcal{N}(x_j^*))} \\ &= \frac{1}{2n_- n_+} \sum_{i,j} \frac{\max \{ \mathcal{N}(x_i) \mathcal{P}(x_j), \mathcal{N}(x_j) \mathcal{P}(x_i) \}}{(\mathcal{P}(x_i) + \mathcal{N}(x_i)) (\mathcal{P}(x_j) + \mathcal{N}(x_j))} \\ &= \frac{1}{2n_- n_+} \sum_{x_i, x_j} \max \{ \mathcal{P}(x_i) \mathcal{N}(x_j), \mathcal{P}(x_j) \mathcal{N}(x_i) \}. \end{aligned} \quad (53)$$

2. Statistical ensemble on AR cost function

198

For a given dataset $\mathcal{S} := \{x_1, x_2, \dots, x_m\}$, one classifier f can be transformed into a ranking of $\mathcal{S}$ according to the score given by f :

$$\pi_f := \{x_1^f, x_2^f, \dots, x_m^f\}, \quad (54)$$

where x_i^f is the i -th sample of the ordered set ranked by f in $\mathcal{S}$. Here, we define that $\Pi(\mathcal{S})$ as the rankings of all possible classifiers for $\mathcal{S}$. Then, we define the energy of π^f as follows,

$$E^{AR}(\pi^f) = 1 - AR(\pi^f) \quad (55)$$

$$= 1 - \sum_{i < j} p_+(x_i^f) p_-(x_j^f) - \frac{1}{2} \sum_i p_+(x_i^f) p_-(x_i^f) \quad (56)$$

$$= \sum_{i > j} p_+(x_i^f) p_-(x_j^f) + \frac{1}{2} \sum_i p_+(x_i^f) p_-(x_i^f) \quad (57)$$

Treating $\Pi(\mathcal{S})$ as an ensemble, we can write the following partition function,

$$\begin{aligned} Z^{AR} &= \sum_{\pi^f \in \Pi(\mathcal{S})} \exp(-\beta E^{AR}(\pi)) \\ &= \sum_{\pi^f \in \Pi(\mathcal{S})} \exp \left\{ -\beta \left(\sum_{i > j} p_+(x_i^f) p_-(x_j^f) + \frac{1}{2} \sum_i p_+(x_i^f) p_-(x_i^f) \right) \right\}. \end{aligned} \quad (58)$$

And, the ground state energy E_0^{AR} ($1 - AR^u$) can be defined as

$$E_0^{AR} = \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{AR}. \quad (59)$$

Next, we can solve E_0^{AR} as follows.

$$\begin{aligned} E_0^{AR} &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{AR} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \sum_{\pi^f \in \Pi(\mathcal{S})} \exp \left\{ -\beta \left(\sum_{i > j} p_+(x_i^f) p_-(x_j^f) + \frac{1}{2} \sum_i p_+(x_i^f) p_-(x_i^f) \right) \right\} \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \sum_{\pi^f \in \Pi(\mathcal{S})} \prod_{i > j} \exp \left(-\beta p_+(x_i^f) p_-(x_j^f) \right) \prod_{i=1}^n \exp \left(-\frac{\beta}{2} p_+(x_i^f) p_-(x_i^f) \right) \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \prod_{i,j} \left(e^{-\frac{\beta}{2} p_+(x_i^f) p_-(x_j^f)} + e^{-\frac{\beta}{2} p_+(x_j^f) p_-(x_i^f)} \right) \\ &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \sum_{i,j} \ln \left(e^{-\frac{\beta}{2} p_+(x_i^f) p_-(x_j^f)} + e^{-\frac{\beta}{2} p_+(x_j^f) p_-(x_i^f)} \right) \\ &= \sum_{i,j} \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \left(e^{-\frac{\beta}{2} p_+(x_i^f) p_-(x_j^f)} + e^{-\frac{\beta}{2} p_+(x_j^f) p_-(x_i^f)} \right) \\ &= \frac{1}{2} \sum_{i,j} \min \left\{ p_+(x_j) p_-(x_i), p_+(x_i) p_-(x_j) \right\} \\ &= \frac{1}{2n_+ n_-} \sum_{x_i, x_j} \min \left\{ \mathcal{P}(x_i) \mathcal{N}(x_j), \mathcal{P}(x_j) \mathcal{N}(x_i) \right\}. \end{aligned} \quad (60)$$

Then, we know that

$$AR^u = 1 - E_0^{AR} = \frac{1}{2n_+ n_-} \sum_{x_i, x_j} \max \left\{ \mathcal{P}(x_i) \mathcal{N}(x_j), \mathcal{P}(x_j) \mathcal{N}(x_i) \right\}. \quad (61)$$

207 Further, it is worthy noting that

$$\begin{aligned}
AR^u &= \frac{1}{2} \sum_{i,j} \max \left\{ p_+(x_i)p_-(x_j), p_+(x_j)p_-(x_i) \right\} \\
&= \sum_{i < j} \max \left\{ p_+(x_i)p_-(x_j), p_+(x_j)p_-(x_i) \right\} + \frac{1}{2} \sum_i p_+(x_i)p_-(x_i) \\
&= \sum_{i < j} \max \left\{ p_+(x_i)(1 - p_+(x_j)), p_+(x_j)(1 - p_+(x_i)) \right\} + \frac{1}{2} \sum_i p_+(x_i)p_-(x_i) \\
&= \sum_{i < j} \left(\max \left\{ p_+(x_i), p_+(x_j) \right\} - p_+(x_i)p_+(x_j) \right) + \frac{1}{2} \sum_i p_+(x_i)p_-(x_i) \\
&= \sum_{i < j: p_+(x_i) > p_+(x_j)} \left(p_+(x_i) - p_+(x_i)p_+(x_j) \right) + \frac{1}{2} \sum_i p_+(x_i)p_-(x_i) \\
&= \sum_{i < j: p_+(x_i) > p_+(x_j)} p_+(x_i)p_-(x_j) + \frac{1}{2} \sum_i p_+(x_i)p_-(x_i).
\end{aligned} \tag{62}$$

208 Naturally, we construct a ranking $\pi^* = \{x_1^*, x_2^*, \dots, x_m^*\}$ for $\mathcal{S}$ satisfying that $p_+(x_i^*) > p_+(x_j^*)$ for any $i < j$, whose
209 AUC is equal to E_0^{AR} as follows,

$$E^{AR}(\pi_0) = \sum_{j < k: p_+(x'_j) > p_+(x'_k)} p_+(x'_j)p_-(x'_k) + \frac{1}{2} \sum_j p_+(x'_j)p_-(x'_j) = E_0 \tag{63}$$

210 It also means that, the optimal classifier f_{AR}^* corresponding to E_0^{AR} is

$$f_{AR}^*(x_i) = p_+(x_i), \quad \forall x_i \in \mathcal{S}. \tag{64}$$

211 B. Optimal PR Curve and statistical ensemble on AP cost function

212 1. Optimal PR Curve

213 The precision-recall (PR) curve is a widely utilized metric for evaluating binary classifiers, emphasizing the trade-off
214 between precision and recall. It provides a detailed view of a classifier's performance at various decision thresholds.
215 To plot a PR curve, one must compute precision and recall at numerous thresholds. This concept is analogous to
216 the receiver operating characteristic (ROC) curve, which allows adjustment of the threshold to balance precision and
217 recall.

218 For each threshold value, precision and recall are determined using the following formulas:

$$\text{precision} = \frac{\text{TP}}{\text{TP} + \text{FP}}, \tag{65}$$

219 where TP is the number of true positives and FP is the number of false positives. Recall, also known as true positive
220 rate (TPR), is defined as:

$$\text{recall} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \tag{66}$$

221 where FN is the number of false negatives. Both the ROC and PR curves share a commonality in that they evaluate
222 the classifier's performance by computing metrics at various thresholds. At a specific threshold k , the precision and
223 recall can be expressed as:

$$\text{precision} = \frac{\text{TP}}{k} = \frac{n_+}{k} \times \text{TPR}, \tag{67}$$

$$\text{recall} = \text{TPR}, \tag{68}$$

224 where n_+ is the total number of positive samples.

225 The objective in seeking the optimal PR curve is to maximize both precision and recall for a given threshold k .
 226 This can be formulated as the following optimization problem:

$$\max_f \text{TPR}, \quad (69)$$

227 which we have addressed in the previous section. The optimal classifier that achieves the best PR curve is identical
 228 to the one that optimizes the ROC curve and is given by:

$$f_{\text{PR}}^*(x) = f_{\text{ROC}}^*(x) = \frac{\mathcal{P}(x)}{\mathcal{P}(x) + \mathcal{N}(x)}, \quad (70)$$

229 where x belongs to the sample space $\mathcal{S}$. The optimal PR curve is constructed by plotting a series of data points in
 230 the (recall, precision)-plane, which are calculated as follows:

$$\left\{ \left(\frac{1}{n_+} \sum_{i=1}^k \frac{\mathcal{P}(x_i^*)}{\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)}, \frac{1}{k} \sum_{i=1}^k \frac{\mathcal{P}(x_i^*)}{\mathcal{P}(x_i^*) + \mathcal{N}(x_i^*)} \right) \right\}_{k=0,1,\dots,n} \quad (71)$$

231 Furthermore, the upper bound of the area under the PR curve (AP^u) can be calculated as follows:

$$\text{AP}^u = \frac{1}{2n_+} \sum_{x_i^*} p_+(x_i^*) \left(\frac{1}{i-1} \sum_{j=1}^{i-1} p_+(x_j^*) + \frac{1}{i} \sum_{j=1}^i p_+(x_j^*) \right), \quad (72)$$

232 where $p_+(x_i)$ is the probability of a sample x_i being positive:

$$p_+(x_i) = \frac{\mathcal{P}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}. \quad (73)$$

233

2. statistical ensemble on AP cost function

234 Akin to AR ensemble, we define that $\Pi(\mathcal{S})$ includes all possible classifiers for $\mathcal{S}$. For any classifier $\pi_f \in \Pi(\mathcal{S})$, we
 235 define its energy as $1 - AP(\pi_f)$. As a consequence, the partition function and ground state of AP ensemble can be
 236 written as

$$Z^{AP} = \sum_{\pi_f \in \Pi(\mathcal{S})} \exp \left(-\beta (1 - AP(\pi_f)) \right) \quad (74)$$

237 and

$$E_0^{AP} = \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{AP}. \quad (75)$$

238 However, it is very difficult to directly calculate the above ground state because there is no explicit expression for
 239 AP , unlike AR . Alternatively, we have proved in Sec. III B 1 that, the optimal AR shares the same classifier (ranking)
 240 with the optimal AP , that is,

$$f_{AP}^*(x_i) = f_{AR}^*(x_i) = p_+(x_i), \quad \forall x_i \in \mathcal{S}. \quad (76)$$

241 Furthermore, we can write the AP^u in an indirect expression:

$$\text{AP}^u = \sum_{i=1}^m \frac{p_+(x_i^*)}{2n_+} \left(\frac{1}{i-1} \sum_{j=1}^{i-1} p_+(x_j^*) + \frac{1}{i} \sum_{j=1}^i p_+(x_j^*) \right), \quad (77)$$

242 where $\pi^* = \{x_1^*, x_2^*, \dots, x_m^*\}$ has been mentioned in Sec. III A 2.

C. Optimal Accuracy and statistical ensemble on AC cost function

1. Optimal Accuracy

Accuracy (AC) is a fundamental metric that quantifies the proportion of correctly predicted instances against the total number of instances within a dataset. This metric is universally applicable across classifiers. Given a dataset $\mathcal{S}$ and a feature space $\mathcal{X}$, accuracy can be computed with the following expression:

$$\text{AC} = \frac{1}{2m} \sum_{x_i \in \mathcal{S}} (1 + f(x_i)y_i), \quad (78)$$

where m represents the total number of instances, $f(x_i)$ is the predicted label for instance x_i , and y_i is the true label. The accuracy increases by $\frac{1}{2m}$ for each correctly classified instance x_i ; it remains unchanged for incorrect predictions.

While accuracy itself is not a conventional loss function, optimizing for the highest possible accuracy and determining the optimal classifier functions are critical endeavors in machine learning. The mathematical upper limit of accuracy, denoted as AC^u , can be formalized as:

$$\text{AC}^u = \max_f \frac{1}{2m} \sum_{x_i \in \mathcal{S}} (1 + f(x_i)y_i), \quad (79)$$

with the corresponding optimal classifier being:

$$f_{\text{AC}}^* = \arg \max_f \frac{1}{2m} \sum_{x_i \in \mathcal{S}} (1 + f(x_i)y_i). \quad (80)$$

Through further analysis, we can expand and simplify the equation by considering the relationship between probabilities associated with positive and negative instances:

$$\begin{aligned} \max_f \sum_{x_i \in \mathcal{S}} \frac{1}{2m} (1 + f(x_i)y_i) &= \max_f \sum_{x \in \mathcal{X}} \frac{\mathcal{P}(x)}{2} (1 + f(x)) + \frac{\mathcal{N}(x)}{2} (1 - f(x)) \\ &\leq \sum_{x_i \in \mathcal{S}} \max_{f(x)} \frac{\mathcal{P}(x_i)}{2} (1 + f(x_i)) + \frac{\mathcal{N}(x_i)}{2} (1 - f(x_i)), \end{aligned} \quad (81)$$

which leads us to redefine AC^u as:

$$\text{AC}^u = \frac{1}{m} \sum_{x_i} \max_{f(x_i)} \frac{\mathcal{P}(x_i)}{2} (1 + f(x_i)) + \frac{\mathcal{N}(x_i)}{2} (1 - f(x_i)), \quad (82)$$

and the optimal classifier for each instance $x_i \in \mathcal{S}$ becomes:

$$f_{\text{AC}}^*(x_i) = \arg \max_{f(x_i)} \frac{\mathcal{P}(x_i)}{2} (1 + f(x_i)) + \frac{\mathcal{N}(x_i)}{2} (1 - f(x_i)). \quad (83)$$

By enumeration, the solution for the optimal classifier is:

$$f_{\text{AC}}^*(x_i) = \begin{cases} 1 & \text{if } \mathcal{P}(x_i) \geq \mathcal{N}(x_i), \\ -1 & \text{if } \mathcal{P}(x_i) < \mathcal{N}(x_i), \end{cases} \quad (84)$$

resulting in the upper limit of accuracy being:

$$\text{AC}^u = \frac{1}{m} \sum_{x_i \in \mathcal{S}} \max \{ \mathcal{P}(x_i), \mathcal{N}(x_i) \}. \quad (85)$$

2. statistical ensemble on AC cost function

Unlike AR and AP , AC focuses on the predicted binary label for every sample, rather than a ranking of all samples in $\mathcal{S}$. In this context, every classifier f can be transformed into a m -dimensional binary vector

$$\pi^f = (x_1^f, x_2^f, \dots, x_m^f), \quad (86)$$

where $x_i^f \in \{-1, 1\}$ for $i = 1, 2, \dots, m$. This also means that $\Pi(\mathcal{S}) = \{1, -1\}^m$ for AC cost function. And, the energy of π^f is defined as

$$\begin{aligned}
 E^{AC}(\pi^f) &= 1 - AC(\pi^f) \\
 &= 1 - \left(\frac{1}{m} \sum_i \frac{\mathcal{P}(x_i)}{2} (1 + x_i^f) + \frac{\mathcal{N}(x_i)}{2} (1 - x_i^f) \right) \\
 &= \frac{1}{m} \sum_i \frac{\mathcal{P}(x_i)}{2} (1 - x_i^f) + \frac{\mathcal{N}(x_i)}{2} (1 + x_i^f) \\
 &= \frac{1}{2} + \frac{1}{2m} \sum_i x_i^f (\mathcal{N}(x_i) - \mathcal{P}(x_i)).
 \end{aligned} \tag{87}$$

Then, the partition function is

$$\begin{aligned}
 Z^{AC} &= \sum_{\pi^f \in \Pi(\mathcal{S})} \exp(-\beta E^{AC}(\pi^f)) \\
 &= \sum_{\pi^f \in \Pi(\mathcal{S})} \exp \left\{ -\beta \left(\frac{1}{2} + \frac{1}{2m} \sum_i x_i^f (\mathcal{N}(x_i) - \mathcal{P}(x_i)) \right) \right\}
 \end{aligned} \tag{88}$$

And, the ground state is calculated as

$$\begin{aligned}
 E_0^{AC} &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln Z^{AC} \\
 &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \sum_{\pi^f \in \Pi(\mathcal{S})} \exp \left\{ -\beta \left(\frac{1}{2} + \frac{1}{2m} \sum_i x_i^f (\mathcal{N}(x_i) - \mathcal{P}(x_i)) \right) \right\} \\
 &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \sum_{\pi^f \in \Pi(\mathcal{S})} e^{-\beta/2} \prod_i \exp \left(-\frac{\beta}{2m} x_i^f (\mathcal{N}(x_i) - \mathcal{P}(x_i)) \right) \\
 &= \lim_{\beta \rightarrow \infty} -\frac{1}{\beta} \ln \left\{ e^{-\beta/2} \prod_i \left(e^{-\frac{\beta}{2m} (\mathcal{N}(x_i) - \mathcal{P}(x_i))} + e^{-\frac{\beta}{2m} (\mathcal{P}(x_i) - \mathcal{N}(x_i))} \right) \right\} \\
 &= \frac{1}{2} - \lim_{\beta \rightarrow \infty} \frac{1}{\beta} \ln \left\{ \prod_i \left(e^{-\frac{\beta}{2m} (\mathcal{N}(x_i) - \mathcal{P}(x_i))} + e^{-\frac{\beta}{2m} (\mathcal{P}(x_i) - \mathcal{N}(x_i))} \right) \right\} \\
 &= \frac{1}{2} - \sum_i \lim_{\beta \rightarrow \infty} \frac{1}{\beta} \ln \left\{ e^{-\frac{\beta}{2m} (\mathcal{N}(x_i) - \mathcal{P}(x_i))} + e^{-\frac{\beta}{2m} (\mathcal{P}(x_i) - \mathcal{N}(x_i))} \right\} \\
 &= \frac{1}{2} + \frac{1}{2m} \sum_i \min \{ \mathcal{N}(x_i) - \mathcal{P}(x_i), \mathcal{P}(x_i) - \mathcal{N}(x_i) \} \\
 &= \frac{1}{m} \sum_i \min \{ \mathcal{P}(x_i), \mathcal{N}(x_i) \}
 \end{aligned} \tag{89}$$

Indeed,

$$AC^u = 1 - E_0^{AC} = \frac{1}{m} \sum_i \max \{ \mathcal{P}(x_i), \mathcal{N}(x_i) \}. \tag{90}$$

Naturally, the optimal classifier corresponding to AC^u is

$$f_{AC}^*(x_i) = \begin{cases} 1 & \text{if } \mathcal{P}(x_i) \geq \mathcal{N}(x_i) \\ -1 & \text{if } \mathcal{P}(x_i) < \mathcal{N}(x_i) \end{cases}, \tag{91}$$

for any $x_i \in \mathcal{S}$.

IV. UNIVERSAL OPTIMAL CLASSIFIER

The theoretical foundation of optimal classifiers demonstrates their alignment with the common objective functions including Square loss, Logistic loss, Hinge loss, and Softmax loss [4] in section II and section III. Specifically, the optimal classifiers for both the ROC and PR curves are congruent, denoted as $f_{AR}^* \equiv f_{AP}^*$. This implies that continuous binary classifiers converge to a unified optimal form for each feature vector $x_i \in \mathcal{S}$, symbolized as $g_{\text{Square}}^* \equiv g_{\text{Logistic}}^* \sim p_+(x_i) \equiv f_{AR}^* \equiv f_{AP}^*$, where g^* represents the objective function. Similarly, discrete binary classifiers are equivalent, denoted as $g_{\text{Hinge}}^* \equiv g_{\text{Softmax}}^* \equiv f_{AC}^*$.

From this, we can deduce a universal representation for any optimal classifier, encompassing both continuous and discrete forms, that achieves the best performance across various objective functions and evaluation metrics:

$$\mathcal{F}^*(x_i) = \begin{cases} 1 & f^*(x_i) \geq 0 \\ -1 & f^*(x_i) < 0 \end{cases}, \quad (92)$$

where $f^*(x_i) = \frac{\mathcal{P}(x_i) - \mathcal{N}(x_i)}{\mathcal{P}(x_i) + \mathcal{N}(x_i)}$ is the derived optimal classifier that minimizes loss and maximizes performance across various objective functions and evaluation metrics.

Eq. 92 underscores the synergy between the learning process, which is steered by objective functions, and the evaluation process within the realm of binary classification. The efficacy of a binary classifier, irrespective of its computational complexity or efficiency, is inherently bound by the selected objective function and the intrinsic properties of the dataset. This establishes a performance ceiling dictated by both the chosen evaluation metric and the data's innate characteristics.

V. SENSITIVITY ANALYSIS

In Sections 2 and 3, we established the mathematical relationships between the lower bound of the objective function with respect to the training set and the upper bound of the evaluation metrics related to the test set. This section extends that discussion within the out-of-sample framework by combining these relationships to investigate the dual concerns of training loss and generalizability in classification tasks.

Let us denote the training and test sets by $\mathcal{S}_{train}$ and $\mathcal{S}_{test}$, respectively. Furthermore, we denote $\mathcal{P}_{train}(x_i)$ and $\mathcal{N}_{train}(x_i)$ as the respective counts of positive and negative instances with feature x_i in the training set, and similarly, $\mathcal{P}_{test}(x_i)$ and $\mathcal{N}_{test}(x_i)$ for the test set.

For a discrete classifier $f(x)$, the boundary hinge loss within the training set $\mathcal{S}_{train}$ is defined as:

$$\sum_{x_i} \min\{\mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i)\}. \quad (93)$$

Here, Δ_{train} represents the discrepancy between the classifier's hinge loss and the boundary hinge loss, expressed as:

$$\Delta_{train} = \sum_{x_i} \Delta_{train}(x_i),$$

where

$$\Delta_{train}(x_i) = \begin{cases} \mathcal{N}_{train}(x_i) - \min\{\mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i)\} & \text{if } f(x_i) = 1, \\ \mathcal{P}_{train}(x_i) - \min\{\mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i)\} & \text{if } f(x_i) = -1. \end{cases}$$

In parallel, the maximum accuracy achievable on the test set $\mathcal{S}_{test}$ is:

$$\sum_{x_i} \max\{\mathcal{P}_{test}(x_i), \mathcal{N}_{test}(x_i)\}.$$

Similarly, Δ_{test} quantifies the error between the classifier's actual accuracy and the maximum possible accuracy:

$$\Delta_{test} = \sum_{x_i} \Delta_{test}(x_i),$$

where

$$\Delta_{test}(x_i) = \begin{cases} \max\{\mathcal{P}_{test}(x_i), \mathcal{N}_{test}(x_i)\} - \mathcal{P}_{test}(x_i) & \text{if } f(x_i) = 1, \\ \max\{\mathcal{P}_{test}(x_i), \mathcal{N}_{test}(x_i)\} - \mathcal{N}_{test}(x_i) & \text{if } f(x_i) = -1. \end{cases}$$

Therefore, the summation of these two discrepancies represents their respective optimization potential and performance evaluation capacity. For any given feature x_i , the combined error is:

$$(\Delta_{train} + \Delta_{test})(x_i) = \begin{cases} \max\{\mathcal{N}_{train}(x_i) - \mathcal{P}_{train}(x_i), 0\} + \max\{\mathcal{N}_{test}(x_i) - \mathcal{P}_{test}(x_i), 0\} & \text{if } f(x_i) = 1, \\ \max\{\mathcal{P}_{train}(x_i) - \mathcal{N}_{train}(x_i), 0\} + \max\{\mathcal{P}_{test}(x_i) - \mathcal{N}_{test}(x_i), 0\} & \text{if } f(x_i) = -1. \end{cases}$$

We now introduce a lower bound for $(\Delta_{train} + \Delta_{test})$, defined as:

$$\Delta = (\Delta_{train} + \Delta_{test})_{\min} = \sum_{x_i} \Delta(x_i), \quad (94)$$

where

$$\Delta(x_i) = \begin{cases} 0 & \text{if } \mathcal{P}_{train}(x_i) \geq \mathcal{N}_{train}(x_i), \mathcal{P}_{test}(x_i) \geq \mathcal{N}_{test}(x_i) \\ 0 & \text{if } \mathcal{P}_{train}(x_i) < \mathcal{N}_{train}(x_i), \mathcal{P}_{test}(x_i) < \mathcal{N}_{test}(x_i) \\ \epsilon(x_i) & \text{if } \mathcal{P}_{train}(x_i) \geq \mathcal{N}_{train}(x_i), \mathcal{P}_{test}(x_i) < \mathcal{N}_{test}(x_i) \\ \epsilon(x_i) & \text{if } \mathcal{P}_{train}(x_i) < \mathcal{N}_{train}(x_i), \mathcal{P}_{test}(x_i) \geq \mathcal{N}_{test}(x_i) \end{cases}$$

and

$$\epsilon(x_i) = \min\{|\mathcal{P}_{train}(x_i) - \mathcal{N}_{train}(x_i)|, |\mathcal{P}_{test}(x_i) - \mathcal{N}_{test}(x_i)|\}.$$

As we observed, Δ is a general lower bound for the sum of training error and evaluation error regardless of the specific classifier in use. Naturally, $\Delta = 0$ if and only if

$$\begin{cases} \mathcal{P}_{train}(x_i) \geq \mathcal{N}_{train}(x_i) \\ \mathcal{P}_{test}(x_i) \geq \mathcal{N}_{test}(x_i) \end{cases} \quad \text{or} \quad \begin{cases} \mathcal{P}_{train}(x_i) < \mathcal{N}_{train}(x_i) \\ \mathcal{P}_{test}(x_i) < \mathcal{N}_{test}(x_i) \end{cases}$$

for each $x_i \in \mathcal{S}$.

VI. RANDOM DIVISION

Random division method is a standard approach to splitting a dataset into training and testing subsets. In this method, each instance is independently assigned to the training subset with probability p and to the testing subset with probability $1 - p$. For a given feature vector x_i , the counts $\mathcal{P}_{train}(x_i)$ and $\mathcal{N}_{train}(x_i)$ follow binomial distributions $\mathcal{B}(\mathcal{P}(x_i), p)$ and $\mathcal{B}(\mathcal{N}(x_i), p)$ respectively. Considering this, we can investigate the discrepancy Δ within the context of probabilistic partitioning. We define the expected value of Δ as $\mathbb{E}[\Delta] = \frac{1}{m} \sum_{x_i} \mathbb{E}[\Delta(x_i; p)]$, which is calculated as:

$$\begin{aligned} \mathbb{E}[\Delta(x_i; p)] &= \mathbb{E}[\min\{\mathcal{N}_{train}(x_i) - \mathcal{P}_{test}(x_i), \mathcal{P}_{train}(x_i) - \mathcal{N}_{test}(x_i)\}] + \\ &\quad \mathbb{E}[\max\{\mathcal{P}_{test}(x_i), \mathcal{N}_{test}(x_i)\}] - \mathbb{E}[\min\{\mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i)\}] \\ &= \mathbb{E}[\max\{\mathcal{P}_{test}(x_i), \mathcal{N}_{test}(x_i)\}] + \mathbb{E}[\max\{\mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i)\}] - \\ &\quad \max\{\mathcal{P}(x_i), \mathcal{N}(x_i)\}, \end{aligned} \quad (95)$$

where the expected maxima are determined by:

$$\mathbb{E}[\max\{\mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i)\}] = \sum_{i=0}^{\mathcal{P}(x_i)} \sum_{j=0}^{\mathcal{N}(x_i)} \max\{i, j\} \binom{\mathcal{P}(x_i)}{i} \binom{\mathcal{N}(x_i)}{j} p^{\mathcal{P}(x_i) + \mathcal{N}(x_i) - i - j} (1 - p)^{i + j},$$

and

$$\mathbb{E}[\max\{\mathcal{P}_{test}(x_i), \mathcal{N}_{test}(x_i)\}] = \sum_{i=0}^{\mathcal{P}(x_i)} \sum_{j=0}^{\mathcal{N}(x_i)} \max\{i, j\} \binom{\mathcal{P}(x_i)}{i} \binom{\mathcal{N}(x_i)}{j} p^{i + j} (1 - p)^{\mathcal{P}(x_i) + \mathcal{N}(x_i) - i - j}.$$

It's important to note that Δ is symmetric; its value is invariant if we exchange the roles of the training and testing subsets $\mathcal{S}_{train}$ and $\mathcal{S}_{test}$. This symmetry is apparent in the equality $\mathbb{E}[\Delta(x_i; p)] = \mathbb{E}[\Delta(x_i; 1 - p)]$, as both expressions yield the same result.

We observe that Δ exhibits symmetrical behavior, as its value remains invariant under the interchange of the training set $\mathcal{S}_{train}$ and the test set $\mathcal{S}_{test}$. This symmetry property is further substantiated by the equality in expected values for complementary probabilities in the random partitioning process, expressed mathematically as $\mathbb{E}[\Delta(x_i; p)]$.

To enhance the interpretability of experimental outcomes, we derive mathematical representations for two key metrics: the expected maximum accuracy of the test set, denoted as AC^u , and the expected minimum hinge loss for the training set. These metrics, in the context of random partitioning, are defined by the following equations:

- For the expected maximum accuracy of the test set:

$$EAC^u = \frac{1}{m(1-p)} \sum_{x_i} \mathbb{E} \max \left\{ \mathcal{P}_{test}(x_i), \mathcal{N}_{test}(x_i) \right\} \quad (96)$$

- For the expected minimum hinge loss of the training set:

$$\mathbb{E}[\text{minimum hinge loss}] = \frac{1}{mp} \sum_{x_i} \mathbb{E} \min \left\{ \mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i) \right\} \quad (97)$$

The expected minimum, present in the equation for hinge loss, is elucidated as follows:

$$\mathbb{E}[\min\{\mathcal{P}_{train}(x_i), \mathcal{N}_{train}(x_i)\}] = \sum_{i=0}^{\mathcal{P}(x_i)} \sum_{j=0}^{\mathcal{N}(x_i)} \min\{i, j\} \binom{\mathcal{P}(x_i)}{i} \binom{\mathcal{N}(x_i)}{j} p^{\mathcal{P}(x_i)+\mathcal{N}(x_i)-i-j} (1-p)^{i+j}. \quad (98)$$

In these expressions, $\mathcal{P}_{train}(x_i)$ and $\mathcal{N}_{train}(x_i)$ denote the number of positive and negative instances of x_i in the training set, respectively, and analogously for $\mathcal{P}_{test}(x_i)$ and $\mathcal{N}_{test}(x_i)$ in the test set. The binomial coefficients reflect the combinatorial possibilities for selecting i positives out of $\mathcal{P}(x_i)$ and j negatives out of $\mathcal{N}(x_i)$, factoring in the probability p of an instance belonging to the training set.

VII. OVERLAPPING AND BOUNDARY

Indeed, training loss and evaluation metrics inherently have a lower bound (0) and an upper bound (1). However, as analyzed in Supplementary Notes 2 and 3, the precise boundaries do not always align with these natural limits. The reason is that positive and negative samples sometimes overlap, making it impossible for any classifier to achieve 100% prediction accuracy. Consequently, this section will further explore the quantifiable correlation between the degree of overlap among positive and negative samples in a dataset and the performance boundaries. However, prior to this exploration, our first step will be to establish a definition for the term ‘‘overlap’’ within the context of a dataset.

Given positive data distribution $\{\mathcal{P}(x_i)/n_+\}_{x_i \in \mathcal{X}}$ and negative data distribution $\{\mathcal{N}(x_i)/n_-\}_{x_i \in \mathcal{X}}$, these two probability distributions can be abbreviated as

$$\mathcal{P} := \{\hat{p}(x_i) | x_i \in \mathcal{S}\}$$

and

$$\mathcal{Q} := \{\hat{n}(x_i) | x_i \in \mathcal{S}\}$$

respectively, in which $\hat{p}(x_i) = \mathcal{P}(x_i)/n_+$ and $\hat{n}(x_i) = \mathcal{N}(x_i)/n_-$ for every $x_i \in \mathcal{S}$. Considering that Jensen-Shannon divergence is a symmetry and bounded measures to quantify the divergence degree between two probability distributions, we define the overlapping measures between $\mathcal{P}$ and $\mathcal{Q}$ as the complement of $J(\mathcal{P}||\mathcal{N})$, that is,

$$\begin{aligned} D_S &= 1 - J(\mathcal{P}||\mathcal{N}) \\ &= 1 - \frac{1}{2} \left(\text{KL}(\mathcal{P}||\mathcal{M}) + \text{KL}(\mathcal{N}||\mathcal{M}) \right) \\ &= 1 - \frac{1}{2} \left(\sum_{x_i \in \mathcal{S}} \hat{p}(x_i) \log_2 \left(\frac{2\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{2\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) \right) \\ &= -\frac{1}{2} \left(\sum_{x_i \in \mathcal{S}} \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) \right) \end{aligned} \quad (99)$$

where $\mathcal{M} = \frac{1}{2}(\mathcal{P} + \mathcal{N})$ is a mixture distribution of $\mathcal{P}$ and $\mathcal{N}$, and $\text{KL}(\mathcal{P}||\mathcal{N})$ is the Kullback–Leibler divergence of any two distributions $\mathcal{P}$ and $\mathcal{N}$. Here, $D_{\mathcal{S}}$ is also bounded by zero and one, in particular, $D_{\mathcal{S}} = 0$ means that they are completely separated; $D_{\mathcal{S}} = 1$ means that they are totally overlapped. Therefore, the specific value of $D_{\mathcal{S}} \in [0, 1]$ quantitatively characterizes the degree of overlap between $\mathcal{P}$ and $\mathcal{N}$.

Next, we plan to discover the quantitative relationship between $D_{\mathcal{S}}$ and AR^u through describing the fluctuation of AR^u given a fixed overlapping $D_{\mathcal{S}}$. At first, we define $\text{AR}_{\max}^u(D_{\mathcal{S}})$ and $\text{AR}_{\min}^u(D_{\mathcal{S}})$ as the maximum and minimum values of upper bound of AUC (AR^u) of dataset $\mathcal{S}$ with overlapping $D_{\mathcal{S}}$, respectively. In other words, $\text{AR}_{\max}^u(D_{\mathcal{S}})$ and $\text{AR}_{\min}^u(D_{\mathcal{S}})$ can be obtained through solving the two following optimization problems:

$$\begin{aligned}
\min \quad & \sum_{x_i, x_j \in \mathcal{S}} \max \left\{ \hat{p}(x_i)\hat{n}(x_j), \hat{n}(x_i)\hat{p}(x_j) \right\} \\
\text{s.t.} \quad & \sum_{x_i \in \mathcal{S}} \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + 2D_{\mathcal{S}} = 0 \\
& \sum_{x_i \in \mathcal{S}} \hat{p}(x_i) = 1 \\
& \sum_{x_i \in \mathcal{S}} \hat{n}(x_i) = 1 \\
& \hat{p}(x_i) \geq 0 \quad i = 1, 2, \dots, m \\
& \hat{n}(x_i) \geq 0 \quad i = 1, 2, \dots, m
\end{aligned} \tag{100}$$

and

$$\begin{aligned}
\max \quad & \sum_{x_i, x_j \in \mathcal{S}} \max \left\{ \hat{p}(x_i)\hat{n}(x_j), \hat{n}(x_i)\hat{p}(x_j) \right\} \\
\text{s.t.} \quad & \sum_{x_i \in \mathcal{S}} \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + 2D_{\mathcal{S}} = 0 \\
& \sum_{x_i \in \mathcal{S}} \hat{p}(x_i) = 1 \\
& \sum_{x_i \in \mathcal{S}} \hat{n}(x_i) = 1 \\
& \hat{p}(x_i) \geq 0 \quad i = 1, 2, \dots, m \\
& \hat{n}(x_i) \geq 0 \quad i = 1, 2, \dots, m
\end{aligned} \tag{101}$$

Now, we observed an interesting phenomenon that both AR^u and $D_{\mathcal{S}}$ remain unchanged if we swap the values of $(\hat{p}(x_i), \hat{n}(x_i))$ and $(\hat{p}(x_j), \hat{n}(x_j))$. Considering the computational process of AR^u , we assume that $\hat{p}(x_1)/\hat{n}(x_1) \leq \hat{p}(x_2)/\hat{n}(x_2) \leq \dots \leq \hat{p}(x_m)/\hat{n}(x_m)$ without loss of generality. Under this assumption, the AR^u can be simplified as

$$\text{AR}^u = \sum_{i=1}^m \sum_{j=1}^{i-1} \hat{p}(x_i)\hat{n}(x_j) + \frac{1}{2} \sum_{i=1}^m \hat{p}(x_i)\hat{n}(x_i). \tag{102}$$

349 Combined with the above assumption and equation, Eq. 100 and Eq. 101 can be rewritten as

$$\begin{aligned}
\min \quad & \sum_{i=1}^m \sum_{j=1}^{i-1} \hat{p}(x_i) \hat{n}(x_j) + \frac{1}{2} \sum_{i=1}^m \hat{p}(x_i) \hat{n}(x_i) \\
\text{s.t.} \quad & \sum_{i=1}^m \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + 2D_S = 0 \\
& \hat{p}(x_1)/\hat{n}(x_1) \leq \hat{p}(x_2)/\hat{n}(x_2) \leq \cdots \leq \hat{p}(x_m)/\hat{n}(x_m) \\
& \sum_{i=1}^m \hat{p}(x_i) = 1 \\
& \sum_{i=1}^m \hat{n}(x_i) = 1 \\
& \hat{p}(x_i) \geq 0 \quad i = 1, 2, \dots, m \\
& \hat{n}(x_i) \geq 0 \quad i = 1, 2, \dots, m
\end{aligned} \tag{103}$$

350 and

$$\begin{aligned}
\max \quad & \sum_{i=1}^m \sum_{j=1}^{i-1} \hat{p}(x_i) \hat{n}(x_j) + \frac{1}{2} \sum_{i=1}^m \hat{p}(x_i) \hat{n}(x_i) \\
\text{s.t.} \quad & \sum_{i=1}^m \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + 2D_S = 0 \\
& \hat{p}(x_1)/\hat{n}(x_1) \leq \hat{p}(x_2)/\hat{n}(x_2) \leq \cdots \leq \hat{p}(x_m)/\hat{n}(x_m) \\
& \sum_{i=1}^m \hat{p}(x_i) = 1 \\
& \sum_{i=1}^m \hat{n}(x_i) = 1 \\
& \hat{p}(x_i) \geq 0 \quad i = 1, 2, \dots, m \\
& \hat{n}(x_i) \geq 0 \quad i = 1, 2, \dots, m
\end{aligned} \tag{104}$$

351 respectively. According to the symmetry of the ranking assumption

$$\frac{\hat{p}(x_1)}{\hat{n}(x_1)} \leq \frac{\hat{p}(x_2)}{\hat{n}(x_2)} \leq \cdots \leq \frac{\hat{p}(x_m)}{\hat{n}(x_m)},$$

352 it follows that the feasible region is partitioned into $m!$ pairwise symmetric sub-regions. Within each sub-region, the
353 mathematical formulation of $\text{AR}_{\max}^u(D_S)$ is invariant and possesses an identical maximum value. Consequently, we
354 can disregard the ranking assumption in Eq. (104) and deduce a simplified version as follows:

$$\begin{aligned}
\max \quad & \sum_{i=1}^m \sum_{j=1}^{i-1} \hat{p}(x_i) \hat{n}(x_j) + \frac{1}{2} \sum_{i=1}^m \hat{p}(x_i) \hat{n}(x_i), \\
\text{s.t.} \quad & \sum_{i=1}^m \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + 2D_S = 0, \\
& \sum_{i=1}^m \hat{p}(x_i) = 1, \\
& \sum_{i=1}^m \hat{n}(x_i) = 1, \\
& \hat{p}(x_i) \geq 0, \quad i = 1, 2, \dots, m, \\
& \hat{n}(x_i) \geq 0, \quad i = 1, 2, \dots, m.
\end{aligned} \tag{105}$$

We employ the SLSQP solver [5] to resolve the optimization problem outlined in (105) and acquire the corresponding numerical solution for $AR_{\max}^u$. In Fig. S8A, the AR^u curve versus D_S is depicted. Each datum point on the curve represents the numerical solution of the optimization problem under specific parameters. Notably, the curve converges swiftly as m increases, and attains the optimal $AR_{\max}^u$ curve when $m \geq 10$.

Regarding the $AR_{\min}^u$ curve, a heuristic approach is utilized to construct its optimal solution from the feasible region's boundaries. Specifically, we examine a particular case where $\hat{p}(x_1) = 1 - b$, $\hat{p}(x_2) = b$, $\hat{n}(x_1) = 0$, and $\hat{n}(x_2) = 1$, with b being a tunable parameter in the interval $[0, 1]$. In this scenario, the AR^u is expressed as

$$AR^u = 1 - \frac{b}{2}, \quad (106)$$

and D_S is given by

$$\begin{aligned} D_S &= -\frac{1}{2} \left(b \log_2 \frac{b}{b+1} + \log_2 \frac{1}{b+1} \right) \\ &= -\frac{1}{2} (b \log_2 b - (b+1) \log_2(b+1)). \end{aligned} \quad (107)$$

Furthermore, the numerical curve of $AR_{\min}^u$ obtained through the SLSQP solver aligns closely with this heuristic solution as given in (107), providing partial validation for our heuristic approach (see Fig. S8B).

VIII. FEATURE ENGINEERING

In previous sections, we have discussed the boundaries of the objective function and evaluation metrics from the perspective of row data (feature vectors). In fact, column data (features) can also influence the degree of overlap and boundaries in a dataset through their impact on row data. In feature engineering, there are two classic methods of handling column data: feature selection and feature extraction. The former emphasizes adding or removing new features unrelated to existing ones, and the latter is based on extraction and mapping of original features. In this section, we will discuss these two methods separately. But before that, we need to discuss the simplest case first.

Suppose we have an original dataset $\mathcal{S} = \{(x_i, y_i) : i = 1, 2, \dots, m\}$ with k features. This implies that the feature vector x_i can be expressed as $x_i = (x_{i,1}, x_{i,2}, \dots, x_{i,k})$. We also define that there are $\mathcal{P}(x_i)$ positive instances and $\mathcal{N}(x_i)$ negative instances with feature vector x_i in the entire dataset. By introducing a new feature into every instance, we can create a new dataset $\mathcal{S}'$. Using feature vector x_i as an example, these $\mathcal{P}(x_i) + \mathcal{N}(x_i)$ could be added into different feature values, which are included in $\{x_i^s = (x_{i,1}, x_{i,2}, \dots, x_{i,k}, x_{i,k+1}^s) : s = 1, 2, \dots, s_i\}$. For the sake of argument, we illustrate that the original feature vector is split into s_i pairwise distinct feature vectors after the addition of one row data, satisfying that

$$\begin{cases} \sum_{j=1}^{s_i} \mathcal{P}(x_i^j) = \mathcal{P}(x_i) \\ \sum_{j=1}^{s_i} \mathcal{N}(x_i^j) = \mathcal{N}(x_i) \end{cases}, \quad (108)$$

in which s_i is defined as the diversity for x_i . Consequently, we will proceed to prove the following lemma.

Lemma 1. *Upon the inclusion of a new feature into the original dataset, AR^u will either increase or remain constant, while D_S will either decrease or stay the same. These values will remain unchanged if, and only if, the diversity $s_i = 1$ for each x_i .*

Proof. The upper bound of AUC in the original dataset $\mathcal{S}$

$$AR_{\text{original}}^u = \frac{1}{2n_-n_+} \sum_{i,j} \max\{\mathcal{P}(x_i)\mathcal{N}(x_j), \mathcal{P}(x_j)\mathcal{N}(x_i)\}, \quad (109)$$

and the new boundary is

$$\begin{aligned}
AR_{new}^u &= \frac{1}{2n_-n_+} \sum_{i,j} \sum_{i'=1}^{s_i} \sum_{j'=1}^{s_j} \max\{\mathcal{P}(x_i^{i'})\mathcal{N}(x_j^{j'}), \mathcal{P}(x_j^{j'})\mathcal{N}(x_i^{i'})\} \\
&\geq \frac{1}{2n_-n_+} \sum_{i,j} \max \left\{ \sum_{i'=1}^{s_i} \mathcal{P}(x_i^{i'}) \sum_{j'=1}^{s_j} \mathcal{N}(x_j^{j'}), \sum_{j'=1}^{s_j} \mathcal{P}(x_j^{j'}) \sum_{i'=1}^{s_i} \mathcal{N}(x_i^{i'}) \right\}. \\
&= \frac{1}{2n_-n_+} \sum_{i,j} \max\{\mathcal{P}(x_i)\mathcal{N}(x_j), \mathcal{P}(x_j)\mathcal{N}(x_i)\} \\
&= AR_{original}^u
\end{aligned} \tag{110}$$

Similarly, the original D_S can be written as

$$D_S^{original} = -\frac{1}{2} \left(\sum_{x_i \in \mathcal{S}} \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) \right) \tag{111}$$

and the new overlapping is

$$\begin{aligned}
D_S^{new} &= -\frac{1}{2} \left(\sum_{x_i \in \mathcal{S}} \sum_{i'=1}^{s_i} \hat{p}(x_i^{i'}) \log_2 \left(\frac{\hat{p}(x_i^{i'})}{\hat{p}(x_i^{i'}) + \hat{n}(x_i^{i'})} \right) + \hat{n}(x_i^{i'}) \log_2 \left(\frac{\hat{n}(x_i^{i'})}{\hat{p}(x_i^{i'}) + \hat{n}(x_i^{i'})} \right) \right) \\
&\leq -\frac{1}{2} \left(\sum_{x_i \in \mathcal{S}} \sum_{i'=1}^{s_i} \hat{p}(x_i^{i'}) \log_2 \left(\frac{\sum_{i'=1}^{s_i} \hat{p}(x_i^{i'})}{\sum_{i'=1}^{s_i} \hat{p}(x_i^{i'}) + \sum_{i'=1}^{s_i} \hat{n}(x_i^{i'})} \right) + \sum_{i'=1}^{s_i} \hat{n}(x_i^{i'}) \log_2 \left(\frac{\sum_{i'=1}^{s_i} \hat{n}(x_i^{i'})}{\sum_{i'=1}^{s_i} \hat{p}(x_i^{i'}) + \sum_{i'=1}^{s_i} \hat{n}(x_i^{i'})} \right) \right) \\
&= -\frac{1}{2} \left(\sum_{x_i \in \mathcal{S}} \hat{p}(x_i) \log_2 \left(\frac{\hat{p}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) + \hat{n}(x_i) \log_2 \left(\frac{\hat{n}(x_i)}{\hat{p}(x_i) + \hat{n}(x_i)} \right) \right) \\
&= D_S^{original}.
\end{aligned} \tag{112}$$

They are equal to each other if and only if there exists i' satisfying that $\hat{p}(x_i^{i'}) = \hat{p}(x_i)$ and $\hat{n}(x_i^{i'}) = \hat{n}(x_i)$ for every x_i , i.e., $s_i = 1$. $\square$

Actually, adding new features will cause the overlapping of the positive and negative samples of the dataset to decrease or remain unchanged, while reducing the original features will cause the overlapping to increase or remain unchanged. The same conclusion is also applicable to the boundaries of various evaluation indicators and loss functions, such as AR^u , AP^u and AC^u .

A. Feature Selection

Feature selection is the process of selecting a subset of relevant features (variables, predictors) for use in model construction. A feature selection algorithm can be seen as the combination of a search technique for proposing new feature subsets, along with an evaluation measure which scores the different feature subsets. The simplest algorithm is to test each possible subset of features finding the one which minimizes the error rate to reach the best performance. Actually, the exponential number of potential subset selection and the huge amount of training cost for any classifier make this process difficult. However, the boundary theory we proposed can better select subset of all features with high performance measured by AR, AP (best ranking) and AC and low computational cost (time complexity).

Given an integer k_0 , the optimal k_0 feature subset is targeted by AR^u . This implies that we can directly compute the AR^u for each feature subset in the entire dataset $\mathcal{S}$ to assess the data quality and training potential, bypassing the need for a training process. This approach significantly conserves storage space and computational resources. We denote $AR_{k_0}^u$ and $D_S^{k_0}$ as the optimal AR^u and D_S , respectively, when we traverse all possible k_0 feature subsets. The corresponding feature subset is named by *optimal feature subset*. Based on Lemma 1 and the principle of recursion, it is evident that

Theorem 1. $AR_{k_0}^u$ exhibits a monotonic non-decreasing trend and $D_S^{k_0}$ is monotonic non-increasing when $k_0 \in \{1, 2, \dots, k\}$. Here, k represents the total number of features in the entire dataset.

Proof. Based on the principle of recursion, we only need to prove that $\text{AR}_{k_0}^u \leq \text{AR}_{k_0+1}^u$ and $D_S^{k_0} \geq D_S^{k_0+1}$ for any $k_0 \in \{1, 2, \dots, k-1\}$. Assumed that $\mathcal{F}_{k_0}^* = \{f_1^*, f_2^*, \dots, f_{k_0}^*\}$ is the optimal k_0 -feature subset. Then we construct a new $k_0 + 1$ -feature subset $\mathcal{F}_{k_0+1}^* = \mathcal{F}_{k_0}^* + \{f_i\}$, in which f_i is any selected feature not belonging to $\mathcal{F}_{k_0}^*$. According to Lemma 1, we know that the AR^u of $\mathcal{F}_{k_0+1}^*$ is higher than or equal to $\text{AR}_{k_0}^u$. At the same time, we also know that the AR^u of $\mathcal{F}_{k_0+1}$ is lower than or equal to $\text{AR}_{k_0+1}^u$ since the definition of $\text{AR}_{k_0+1}^u$. Therefore, we successfully proved that $\text{AR}_{k_0}^u \leq \text{AR}_{k_0+1}^u$. In a similar way, we can also prove $D_S^{k_0} \geq D_S^{k_0+1}$. $\square$

Theorem 1 elucidates the direct correlation between the performance bounds and the overlapping index within a given dataset. Specifically, it reveals that an increase in the number of features (raw data) leads to a reduction in the overlap between the positive and negative sample distributions, which in turn enhances the performance boundaries. This insight informs the design of a feature selection algorithm that leverages the overlapping index, D_S , to ensure the achievement of the highest possible performance upper limit.

Consider a dataset $\mathcal{S}$ composed of k features $\{F_1, F_2, \dots, F_k\}$. The optimal feature subset of size k_0 , denoted as the subset that minimizes D_S (or maximizes AR^u), can be defined where $k_0 = 1, 2, \dots, k$. However, exhaustively evaluating all $\binom{k}{k_0}$ possible subsets may be computationally prohibitive for large k . To address this, approximation techniques like dynamic programming can be employed to devise an efficient approximation algorithm.

For a more practical example, consider the INE dataset with 13 features; it is possible to achieve the dataset's performance boundary using only 8 features, such that $D_S^8 = D_S$. This indicates that the remaining five features are, to some extent, superfluous. We thus define the optimal feature selection dimension k^* as:

$$k^* = \min\{k_0 : D_S^{k_0} = D_S\}. \quad (113)$$

The feature subset corresponding to k^* is referred to as the *global optimal feature subset*.

B. Feature Extraction

Feature extraction is the procedure of deriving features (traits, properties, attributes) from raw data. It is seen as an equivalent transformation that creates new features from the original ones. In previous subsections, we deduced that the boundary of performance is dictated by the data structure. From a mathematical standpoint, feature extraction can be viewed as a mapping of original features (row data). If we incorporate new extracted data into the original dataset, the diversity for each feature vector is 1. In conjunction with Lemma 1, we can state,

Theorem 2. *The inclusion of extracted features into the original dataset does not alter the boundary of training loss, evaluation measures, and overlapping.*

Proof. Any feature extraction process can be regarded as a mapping from existing feature vectors, so the diversity is 1. Combined with Lemma 1, the boundaries and overlapping should be unchanged. $\square$

C. Feature Generated from Other Samples

Both aforementioned two processes involve manipulating each sample's original feature vector through operations such as selection, transformation, or extraction. Drawing an analogy to clustering algorithms in unsupervised learning, we here explore from the perspective of rows: augmenting samples' features with information derived from other samples, rather than itself, based on agreed rules. E.g. propose a new indicator of a sample, neighbors' income, by counting all her/his neighborhoods within certain distance [6].

For each sample $x_i \in \mathcal{S}$, here we define its new feature vector is $x'_i = (x_{i,1}, x_{i,2}, \dots, x_{i,2k})$, in which $(x_{i,1}, x_{i,2}, \dots, x_{i,k})$ is its original k -feature vector and

$$(x_{i,k+1}, x_{i,k+2}, \dots, x_{i,2k}) = f(\Lambda_r(x_i)). \quad (114)$$

And, $\Lambda_r(x_i)$ includes all samples whose distance from x_i is less than r in the original feature space, and f represents an arbitrary operator, such as a mean function.

We then construct a new dataset $\mathcal{S}'_r = \{x'_1, x'_2, \dots, x'_n\}$ with a tunable parameter $r \in [0, \infty]$. Notably, $\mathcal{S}'_0$ is equivalent to $\mathcal{S}$. We can now state the following theorem:

Theorem 3. *The AR^u of $\mathcal{S}'_r$ is always equal to $\mathcal{S}$ regardless of r .*

451 *Proof.* Based on Theorem 1, it is evident that the AR^u of $\mathcal{S}'_r$ is at least as large as that of $\mathcal{S}$. Additionally, for any
 452 two samples with identical original feature vectors, their newly added features will also be identical. According to
 453 Lemma 1, we can say that the AR^u of $\mathcal{S}'_r$ cannot exceed that of $\mathcal{S}$. Hence, the theorem is proven. $\square$

454 In this section, we use the AR^u measure to characterize the predictability of a given dataset, as demonstrated in
 455 Lemma 1, Theorem 1, and Theorem 2. Notably, other measures of predictability, such as AP^u and AC^u , lead to the
 456 same conclusions. The equivalence of these different measures will be thoroughly explained in Section III.

457 IX. DATASETS

458 We utilized 4 datasets in the main text and 37 additional datasets from Kaggle (<https://www.kaggle.com>). The
 459 specifics of the four real-world datasets used in the main text are as follows:

- 460 • Airlines Delay Dataset (AID): comprised of 539,383 records across 8 distinct attributes, the objective is to
 461 forecast flight delays based on scheduled departure information.
- 462 • Heart Disease Dataset (HED): This dataset encompasses a wide range of cardiovascular risk factors, including
 463 age, gender, height, weight, blood pressure, cholesterol and glucose levels, smoking status, alcohol intake, physical
 464 activity, and presence of cardiovascular diseases, from over 70,000 individuals. It serves as a valuable asset for
 465 applying advanced machine learning methods to investigate the link between these factors and cardiovascular
 466 health, which could enhance disease understanding and prevention strategies.
- 467 • Income Classification Dataset (INE): This dataset features variables such as education, employment, and marital
 468 status to predict whether an individual earns more than \$50K annually.
- 469 • Student Sleep Study Dataset (SUD): Originating from a survey-based analysis of US students' sleep patterns,
 470 this dataset utilizes factors like average sleep duration and phone usage time to infer adequate sleep among
 471 students.

472 We also conducted experiments on 37 real-world binary datasets to additionally validate the universality of the
 473 proposed theory. Detailed data descriptions and corresponding experimental results are shown at [https://github.](https://github.com/Feijing92/binary)
 474 [com/Feijing92/binary](https://github.com/Feijing92/binary).

- 475 • Adult Census Dataset (ACD): Given a set of demographic and employment attributes such as age, education
 476 level, occupation, and hours worked per week, the goal is to build a predictive model that classifies individuals
 477 into two categories: 'over threshold' and 'under threshold'. This task is a binary classification problem, where
 478 the model's output is a binary decision indicating whether an individual's income exceeds the predetermined
 479 threshold or not.
- 480 • Android Malware Detection (AMD): This Kaggle dataset focuses on the classification of software applications
 481 into two categories : malware (1) and goodware (0).
- 482 • ASD questionnaires (ASD): This Kaggle dataset is dedicated to predicting whether a patient has autism using
 483 questionnaire data on the topic of autism.
- 484 • Asthma Disease Prediction (ADP): The ADP dataset is a comprehensive collection of anonymized health records
 485 and patient data, which includes vital patient information, environmental factors, and medical history, enabling
 486 the development of advanced machine learning models to forecast asthma treatment outcomes.
- 487 • Branch Prediction (BP): Branch prediction is a technique used in CPU design that attempts to guess the
 488 outcome of a conditional operation and prepare for the most likely result. A digital circuit that performs this
 489 operation is known as a branch predictor. It is an important component of modern CPU architectures, such
 490 as the x86. The problem of branch prediction can be treated as a binary classification problem, where target
 491 feature will be Branch Taken/ Branch Not.
- 492 • Cancer Prediction Dataset (CPDT): This dataset represents a synthetic collection of responses gathered from a
 493 university-conducted survey, aimed at studying the potential risk factors for lung cancer. The survey includes
 494 a variety of demographic, lifestyle, and health-related questions. This dataset is used to predict whether the
 495 respondent has been diagnosed with lung cancer.

- Car Ownership Prediction (COP): The dataset contains information on the occupation, monthly income, credit score, years of employment, finance status, finance history, number of children. This dataset is used to predict whether each individual owns a car or not.
- Cars Purchase Decision (CPDN): This dataset contains details of 1000 customers who intend to buy a car, considering their annual salaries. This dataset is used to predict whether each customer would buy a car.
- Cardiovascular diseases dataset (CDD): This data set predicts whether a patient has cardiovascular disease based on information such as age, height, weight, and whether he or she smokes.
- Contraceptive Method Choice (CMC): This dataset is a subset of the 1987 National Indonesia Contraceptive Prevalence Survey. The samples are married women who were either not pregnant or do not know if they were at the time of interview. This dataset is used to predict the current contraceptive method choice (no use or use).
- College Attending Plan (CAP): The purpose of this dataset is to use the basic information of high school students to predict whether they plan to go to college, in particular, which factors can better distinguish high school students who are willing to go to college from those who are not.
- Symptoms and COVID Presence (SCP): The purpose of this dataset is to provide symptoms as input and it should be able to predict if COVID is possibly present or not.
- Child Sexual Abuse Awareness Prediction (CSAAP): By asking a series of questions, this dataset predicts a person's level of knowledge about child sexual abuse.
- Cyber Security (CS): This dataset wants to analyze whether the particular URL is prone to phishing (malicious) or not.
- Diabetes Health Indicators (DHI): The Behavioral Risk Factor Surveillance System (BRFSS) is a health-related telephone survey that is collected annually by the CDC. Each year, the survey collects responses from over 400,000 Americans on health-related risk behaviors, chronic health conditions, and the use of preventative services. This dataset predicts whether a patient has diabetes based on their health indicators.
- Diabetes Dataset (DD): This dataset was collected to predict Type 2 diabetes. An article is also published implementing this dataset [7].
- Employee Dataset (ED): This dataset contains information about employees in a company, including their educational backgrounds, work history, demographics, and employment-related factors. It is used to predict an employee will leave or not.
- Fraud Detection Bank Dataset (FDBD): This dataset is used to predict whether transactions within the bank are fraudulent transactions.
- Game of Thrones (GOT): This dataset is used to predict the next death on Game Of Thrones character.
- Happiness Dataset (HD): This Dataset is based on a survey conducted where people rated different metrics of their city on a scale of 5 and answered if they are happy or unhappy.
- Heart Disease Nowadays (HDN): Annual CDC survey data of 400k adults related to their health status can be used to predict whether an adult is diagnosed as heart disease.
- Heart Disease Health Indicators Dataset (HDHID): This dataset is also collected from BRFSS survey and used to predict whether an American has heart disease.
- Immigration Madrid (IM): This dataset comes from human resources data from Spain and other countries and is used to predict whether individuals are hired.
- Loan: This dataset contains basic details and loan history from the last 3 months about customers, which is used to predict whether a customer will take out a loan in the future.
- Naive Bayes Classification Data (NBCD): The dataset includes blood glucose and blood pressure data that can be used to classify whether a patient has diabetes.

- Basketball Players' Career Duration (BPCD): The data consists of performance statistics from each player's rookie year. The target variable is a Boolean value that indicates whether a given player will last in the league for five years.
- Phishing Website Detector (PWD): This dataset is used to detect whether a website is a phishing website.
- QSAR Androgen Receptor Dataset (QARD): This dataset was used to develop classification QSAR models for the discrimination of binder/positive (199) and non-binder/negative (1488) molecules by means of different machine learning methods.
- Simplified Titanic Dataset (STD): This dataset is a simplified version of the famous Titanic dataset, which contains information about passengers aboard the Titanic ship. It is designed specifically for beginners who are learning about data analysis and classification problems.
- Ad Click Prediction (ACP): This dataset is used to predict whether customer will click Ad and make a purchase.
- Happy or Not (HON): The dataset provides information on various aspects such as housing costs, education quality, transportation facilities, security, healthcare availability, quality of public services, food quality, events, and happiness levels. We use this dataset to predict an individual is happy or not.
- Telecom Service Customer (TSC): This dataset predicts whether a customer will continue based on their basic information.
- Blood Transfusion Dataset (BTD): A previous study [8] adopted the donor database of Blood Transfusion Service Center in Hsin-Chu City in Taiwan. The center passes their blood transfusion service bus to one university in Hsin-Chu City to gather blood donated about every three months. To build a FRMTC model, we selected 748 donors at random from the donor database. These 748 donor data, each one included R (Recency - months since last donation), F (Frequency - total number of donation), M (Monetary - total blood donated in c.c.), T (Time - months since first donation), and a binary variable representing whether he/she donated blood in March 2007 (1 stand for donating blood; 0 stands for not donating blood).
- TUNADROMD Malware Detection (TMD): This data set is used to identify whether a piece of software is malware [9].
- Vehicle Stolen Dataset (VSD): This dataset is used to predict whether a vehicle will be stolen.
- Web Club Recruitment (WCR): Dataset for recruitment contest for intelligence group in Web Club NITK.
- Wine Quality Dataset (WQD): The dataset contains the wine features or ingredients with the quality and the type of wines. So, the data can be used for the prediction of Wine Quality as well as the detection of the type of wines from ingredient analysis.

-
- [1] L. Rosasco, E. De Vito, A. Caponnetto, M. Piana, and A. Verri, *Neural Computation* **16**, 1063 (2004).
 - [2] T. Fawcett, *Pattern Recognition Letters* **27**, 861 (2006).
 - [3] T. Yang and Y. Ying, *ACM Computing Surveys* **55**, 1 (2022).
 - [4] T. Hastie, R. Tibshirani, J. H. Friedman, and J. H. Friedman, *The Elements of Statistical Learning* (Springer, 2009).
 - [5] D. Kraft, *A Software Package for Sequential Quadratic Programming*, Tech. Rep. (DFVLR-FB 88-28, 1988).
 - [6] F. Hedefalk and M. Dribe, *Proceedings of the National Academy of Sciences, USA* **117**, 14918 (2020).
 - [7] N. P. Tigga and S. Garg, *Procedia Computer Science* **167**, 706 (2020).
 - [8] I.-C. Yeh, K.-J. Yang, and T.-M. Ting, *Expert Systems with applications* **36**, 5866 (2009).
 - [9] P. Borah, D. K. Bhattacharyya, and J. K. Kalita, in *Distributed Computing and Internet Technology* (Springer, 2021) pp. 203–219.

SUPPLEMENTARY TABLES AND FIGURES

Table S1. Description of 41 real datasets we used in this Letter, including 4 datasets we used in main text and additional 37 datasets we used in Supplemental Materials. It includes the number of instances (m), the number of positive instances (n_+), the number of negative instances (n_-), the number of features (k), the overlapping index (D_S) and the optimal feature selection dimension (k^*).

Name	m	n_+	n_-	k	D_S	k^*
AID	539383	299119	240264	7	0.4837	5
HED	70000	34979	35021	11	0.0015	5
INE	32561	7841	24720	13	0.0657	12
SUD	104	36	68	5	0.3181	5
ACD	48842	11687	37155	13	0.0799	12
AMD	4862	1098	3764	148	0.031	46
ASD	3743	1752	1991	17	0.006	16
ADP	316800	237600	79200	18	0.5409	2
BP	400000	64162	335838	480	0.0391	49
CPD	1000	776	224	10	0.0022	7
COP	482	181	301	7	0.0084	4
CPD	68783	34041	34742	11	0.0645	11
CDD	1000	598	402	3	0.0101	3
CMC	1472	628	844	9	0.0491	9
CAP	8000	5404	2596	4	0.0057	4
SCP	5434	4383	1051	20	0.049	13
CSAAP	3002	1291	1711	8	0.2444	8
CS	11054	6157	4897	30	0.028	23
DHI	253680	39977	213703	21	0.0252	21
DD	951	685	266	17	0.0737	8
ED	4652	1600	3052	8	0.236	8
FDBD	20467	5437	15030	112	0.0004	33
GOT	1946	495	1451	2	0.5272	2
HD	143	77	66	6	0.1197	6
HDN	319795	292422	27373	17	0.0135	17
HDHID	253680	23893	229787	21	0.0179	21
IM	1523	1417	106	5	0.4956	5
LOAN	69713	1020	68693	20	0.0007	9
NBCD	995	497	498	2	0.1971	2
BPCD	1340	509	831	19	0.0333	3
PWD	11054	6157	4897	30	0.028	23
QARD	1687	199	1488	1024	0.0015	37
STD	2240	1662	578	3	0.6963	3
ACP	400	143	257	3	0.0051	3
HON	143	66	77	8	0.014	7
TSC	7043	1869	5174	19	0.0062	10
BTD	748	178	570	4	0.2405	3
TMD	4464	899	3565	241	0.0126	47
VSD	20	13	7	3	0.1026	3
WCR	20000	4388	15612	23	0.0007	12
WQD	32485	24453	8032	12	0.0006	3

Table S2. Comprehensive experimental results and theoretical upper bound analysis of the area under the ROC curve (AUC-ROC, abbreviated as AR) for various binary classifiers on additional 37 real datasets. The classifiers evaluated include XGBoost, Multilayer Perceptron (MLP), Support Vector Machine (SVM), Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), K-Nearest Neighbors (KNN), and Naive Bayes (NB).

Data	XGBoost				MLP	SVM	LR	DT	RF	KNN	NB	Boundaries
	Square	Logistic	Hinge	Softmax								
ACD	0.9229	0.921	0.738	0.7779	0.7386	0.5869	0.6155	0.9664	0.9565	0.7863	0.6891	0.9975
AMD	0.9978	0.9976	0.9907	0.9881	0.994	0.984	0.9643	0.9946	0.9946	0.9795	0.5239	0.9994
ASD	0.999	0.999	0.9917	0.9852	0.9881	0.8538	0.815	0.9979	0.9977	0.9345	0.8196	1.0
ADP	0.8333	0.8333	0.8333	0.5	0.538	0.5	0.5	0.5	0.6712	0.666	0.8333	0.8333
BP	0.9989	0.9989	0.9927	0.9926	0.9926	0.9917	0.9914	0.9937	0.9936	0.985	0.8907	0.9992
CPDT	0.9939	0.9753	0.8282	0.8854	0.6006	0.5	0.5907	0.9978	0.9978	0.6343	0.6037	1.0
COP	0.9997	0.9981	0.9945	0.9834	0.7502	0.5751	0.575	0.9967	0.9956	0.7711	0.6236	1.0
CPDN	0.809	0.8078	0.7298	0.7381	0.6742	0.6119	0.6001	0.9728	0.9726	0.7565	0.5968	0.9984
CDD	0.9859	0.9587	0.849	0.8887	0.5533	0.5	0.5	0.9938	0.9946	0.7428	0.5352	0.9999
CMC	0.9471	0.9014	0.7847	0.8254	0.6792	0.582	0.5843	0.9794	0.9762	0.7228	0.5935	0.999
CAP	0.9287	0.9108	0.8259	0.8441	0.7892	0.5	0.7889	0.9958	0.9963	0.7074	0.7889	1.0
SCP	0.9986	0.9986	0.9755	0.9755	0.9813	0.9755	0.9442	0.9755	0.9755	0.9802	0.8491	0.9986
CSAAP	0.9788	0.9776	0.9353	0.9344	0.9214	0.9276	0.8588	0.9361	0.9353	0.9179	0.8196	0.9797
CS	0.995	0.9946	0.9635	0.9585	0.9839	0.9554	0.9227	0.989	0.9894	0.975	0.6425	0.9997
DHI	0.8237	0.8228	0.5137	0.5711	0.5969	0.5	0.5439	0.9881	0.9787	0.6286	0.6667	0.9997
DD	0.9978	0.9975	0.9678	0.9678	0.9454	0.8388	0.8499	0.9609	0.9632	0.9513	0.8366	0.9978
ED	0.9349	0.9241	0.8296	0.8369	0.7897	0.586	0.623	0.9131	0.9024	0.787	0.6421	0.9806
FDBD	0.9816	0.9812	0.9178	0.9208	0.5692	0.5	0.7658	0.9999	0.9997	0.7794	0.7962	1.0
GOT	0.8413	0.8041	0.602	0.5955	0.5	0.5	0.5	0.7287	0.7061	0.6181	0.5	0.895
HD	0.994	0.964	0.9372	0.9426	0.7684	0.7392	0.645	0.9481	0.9481	0.6948	0.6429	0.9942
HDN	0.8469	0.8471	0.5319	0.543	0.5281	0.5	0.5022	0.9791	0.9804	0.5494	0.6521	0.9999
HDHID	0.8554	0.8549	0.5199	0.5494	0.5322	0.5	0.5423	0.9892	0.9787	0.6003	0.685	0.9999
IM	0.8859	0.8552	0.5138	0.5	0.5	0.5	0.5	0.565	0.5693	0.5251	0.5	0.9052
LOAN	0.8981	0.8938	0.5147	0.5093	0.5051	0.5	0.5	0.9994	0.9941	0.5005	0.5132	1.0
NBCD	0.9856	0.9822	0.9417	0.9397	0.6895	0.7165	0.5729	0.9427	0.9427	0.9246	0.591	0.9869
BPCD	0.9962	0.9874	0.9503	0.9571	0.7495	0.5	0.5299	0.9868	0.9823	0.6698	0.6055	0.9995
PWD	0.9956	0.9954	0.962	0.9614	0.9795	0.9574	0.911	0.989	0.9893	0.9731	0.628	0.9997
QARD	1.0	0.9992	0.9667	0.9796	0.9975	0.8005	0.9925	0.9997	0.9997	0.7136	0.8218	1.0
STD	0.8385	0.8329	0.7036	0.687	0.6402	0.5894	0.5932	0.6857	0.6857	0.6474	0.5997	0.8396
ACP	0.9977	0.9772	0.9507	0.9246	0.5147	0.5	0.5128	0.9981	0.9965	0.6918	0.5174	1.0
HON	0.9999	0.9979	0.9924	0.9924	0.8431	0.7511	0.605	0.9935	0.9924	0.7273	0.7013	0.9999
TSC	0.9296	0.9138	0.7314	0.7823	0.5407	0.5	0.6364	0.9977	0.9962	0.6511	0.7484	1.0
BTD	0.9413	0.9255	0.802	0.8172	0.5314	0.5	0.5	0.8954	0.8722	0.6268	0.5	0.9796
TMD	0.9998	0.9997	0.9954	0.996	0.9837	0.9849	0.986	0.9965	0.9965	0.9848	0.5284	0.9999
VSD	0.9945	0.8022	0.9615	0.8187	0.7088	0.8187	0.7088	0.9286	0.9615	0.8132	0.7088	0.9945
WCR	0.8485	0.8529	0.6914	0.6999	0.5872	0.5	0.5973	0.9998	0.9996	0.6138	0.6835	1.0
WQD	0.9962	0.9922	0.9307	0.9248	0.6951	0.5	0.5084	0.9998	0.9998	0.9884	0.5227	1.0

Table S3. Comprehensive experimental results and theoretical upper bound analysis of the area under the PR curve (AUC-PR, abbreviated as AP) for various binary classifiers on additional 37 real datasets. The classifiers evaluated include XGBoost, Multilayer Perceptron (MLP), Support Vector Machine (SVM), Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), K-Nearest Neighbors (KNN), and Naive Bayes (NB).

Data	XGBoost				MLP	SVM	LR	DT	RF	KNN	NB	Boundaries
	Square	Logistic	Hinge	Softmax								
ACD	0.9731	0.9727	0.859	0.8783	0.8597	0.7938	0.8054	0.9808	0.9742	0.883	0.8373	0.999
AMD	0.9993	0.9992	0.9954	0.9941	0.9972	0.9919	0.9807	0.9975	0.9975	0.9892	0.7845	0.9997
ASD	0.9991	0.9991	0.9872	0.9783	0.9816	0.836	0.7818	0.9971	0.996	0.9136	0.7838	1.0
ADP	0.5	0.5	0.5	0.25	0.2785	0.25	0.25	0.25	0.3784	0.3745	0.5	0.5
BP	0.9997	0.9997	0.9974	0.9974	0.9974	0.997	0.9969	0.9978	0.9977	0.9945	0.9603	0.9998
CPDT	0.9826	0.9223	0.649	0.7754	0.3232	0.224	0.3124	0.9965	0.9965	0.3643	0.3192	1.0
COP	0.9998	0.9988	0.9934	0.9805	0.7733	0.6619	0.6621	0.9975	0.9954	0.7903	0.6892	1.0
CPDN	0.7966	0.7947	0.6638	0.6741	0.6155	0.5697	0.5629	0.9652	0.9566	0.6941	0.5599	0.9979
CDD	0.9817	0.9407	0.7964	0.8365	0.4473	0.402	0.402	0.9926	0.9906	0.6218	0.4393	0.9998
CMC	0.9537	0.9101	0.7587	0.7978	0.6797	0.6167	0.6183	0.9785	0.9683	0.7126	0.6238	0.9988
CAP	0.8798	0.8392	0.6079	0.6661	0.5479	0.3244	0.5467	0.9943	0.9937	0.5216	0.5467	0.9999
SCP	0.9936	0.9937	0.9203	0.9203	0.9191	0.9203	0.8707	0.9203	0.9203	0.9109	0.4427	0.9933
CSAAP	0.9843	0.9833	0.9281	0.9267	0.9123	0.9158	0.8555	0.9307	0.9281	0.9098	0.8091	0.9849
CS	0.9941	0.9936	0.9366	0.9362	0.9748	0.9307	0.8809	0.9843	0.9827	0.9641	0.5266	0.9996
DHI	0.9597	0.9598	0.8461	0.8617	0.869	0.8424	0.8542	0.9957	0.9921	0.878	0.8897	0.9999
DD	0.9944	0.9937	0.9329	0.9329	0.8886	0.6882	0.691	0.9384	0.9365	0.9043	0.655	0.9927
ED	0.9587	0.9508	0.8495	0.856	0.822	0.6975	0.7173	0.9217	0.9095	0.8198	0.7291	0.9885
FDBD	0.9927	0.9926	0.9457	0.9478	0.7624	0.7344	0.8569	0.9999	0.9998	0.864	0.8817	1.0
GOT	0.9337	0.9156	0.7864	0.7837	0.7456	0.7456	0.7456	0.8445	0.8332	0.7936	0.7456	0.958
HD	0.9923	0.9617	0.8973	0.9159	0.6772	0.6478	0.5605	0.9361	0.9361	0.6189	0.5552	0.9908
HDN	0.3751	0.3714	0.1194	0.1346	0.1141	0.0856	0.0866	0.9609	0.9599	0.1477	0.1638	0.9985
HDHID	0.9814	0.9815	0.9092	0.9143	0.9113	0.9058	0.9131	0.9978	0.9956	0.9233	0.9387	1.0
IM	0.3857	0.2898	0.0889	0.0696	0.0696	0.0696	0.0696	0.1692	0.1715	0.0883	0.0696	0.4433
LOAN	0.998	0.9979	0.9858	0.9856	0.9855	0.9854	0.9854	1.0	0.9998	0.9854	0.9857	1.0
NBCD	0.9868	0.9845	0.913	0.9095	0.632	0.6514	0.5425	0.9219	0.9163	0.8889	0.5556	0.9877
BPCD	0.9976	0.9926	0.9465	0.9543	0.7728	0.6201	0.6346	0.9886	0.98	0.7148	0.6755	0.9995
PWD	0.9948	0.9946	0.9318	0.9364	0.9665	0.9317	0.8577	0.9843	0.9831	0.961	0.5167	0.9996
QARD	1.0	0.9999	0.9912	0.9946	0.9993	0.9494	0.998	0.9999	0.9999	0.9289	0.9559	1.0
STD	0.6545	0.6476	0.4616	0.4565	0.4017	0.3433	0.3495	0.4566	0.4566	0.3603	0.3419	0.6551
ACP	0.9987	0.9882	0.9491	0.9276	0.6493	0.6425	0.6484	0.9986	0.9961	0.7475	0.6506	1.0
HON	0.9998	0.9981	0.9872	0.9872	0.8043	0.7137	0.5998	0.994	0.9872	0.6904	0.6705	0.9998
TSC	0.9738	0.9682	0.8383	0.8657	0.7508	0.7346	0.7924	0.9987	0.9974	0.7991	0.8535	1.0
BTD	0.9774	0.9719	0.8902	0.8981	0.7736	0.762	0.762	0.9402	0.9266	0.8112	0.762	0.9922
TMD	0.9999	0.9999	0.9978	0.9981	0.9921	0.993	0.9936	0.9984	0.9984	0.9927	0.8097	1.0
VSD	0.9821	0.697	0.875	0.6952	0.531	0.6952	0.531	0.9071	0.875	0.6214	0.531	0.9821
WCR	0.9441	0.9476	0.8524	0.8559	0.8148	0.7806	0.8155	0.9999	0.9998	0.8217	0.8498	1.0
WQD	0.9901	0.9772	0.8505	0.8557	0.4233	0.2473	0.2525	0.9992	0.9992	0.9694	0.2669	1.0

Table S4. Comprehensive experimental results and theoretical upper bound analysis of Accuracy (AC) for various binary classifiers on additional 37 real datasets. The classifiers evaluated include XGBoost, Multilayer Perceptron (MLP), Support Vector Machine (SVM), Logistic Regression (LR), Decision Tree (DT), Random Forest (RF), K-Nearest Neighbors (KNN), and Naive Bayes (NB).

Data	XGBoost				MLP	SVM	LR	DT	RF	KNN	NB	Boundaries
	Square	Logistic	Hinge	Softmax								
ACD	0.8697	0.8684	0.8555	0.8685	0.839	0.7874	0.7852	0.9717	0.9717	0.8603	0.7934	0.9717
AMD	0.9877	0.985	0.9891	0.9866	0.9918	0.9827	0.9733	0.9922	0.9922	0.9813	0.2668	0.9922
ASD	0.985	0.9842	0.992	0.9856	0.9885	0.8496	0.8135	0.9979	0.9979	0.9348	0.8191	0.9979
ADP	0.75	0.75	0.75	0.75	0.75	0.75	0.75	0.75	0.75	0.75	0.75	0.75
BP	0.9936	0.9936	0.9936	0.9936	0.9934	0.993	0.9928	0.994	0.994	0.991	0.9516	0.994
CPDT	0.969	0.936	0.901	0.938	0.799	0.776	0.796	0.999	0.999	0.812	0.794	0.999
COP	0.9917	0.9834	0.9959	0.9876	0.7842	0.6701	0.6494	0.9959	0.9959	0.7967	0.6909	0.9959
CPDN	0.7394	0.7395	0.7308	0.7387	0.6756	0.614	0.6015	0.9727	0.9726	0.7569	0.5991	0.9727
CDD	0.942	0.89	0.873	0.903	0.633	0.598	0.598	0.995	0.995	0.767	0.625	0.995
CMC	0.8872	0.8315	0.8132	0.8444	0.7079	0.6352	0.6277	0.9783	0.9783	0.7452	0.6332	0.9783
CAP	0.8666	0.8462	0.8042	0.8482	0.7493	0.6756	0.7473	0.9972	0.9972	0.7732	0.7473	0.9972
SCP	0.9827	0.9827	0.9827	0.9827	0.9827	0.9827	0.9707	0.9827	0.9827	0.9809	0.7565	0.9827
CSAAP	0.934	0.933	0.934	0.9334	0.9204	0.9284	0.8511	0.934	0.934	0.9161	0.8165	0.934
CS	0.9631	0.9631	0.9636	0.9602	0.9845	0.957	0.9254	0.9897	0.9897	0.9765	0.6019	0.9897
DHI	0.8524	0.8515	0.8444	0.8511	0.843	0.8424	0.8436	0.9925	0.9925	0.8669	0.7717	0.9925
DD	0.9769	0.9769	0.9769	0.9769	0.9611	0.8854	0.8864	0.9769	0.9769	0.9664	0.8707	0.9769
ED	0.8852	0.8788	0.8768	0.8779	0.8364	0.6975	0.7109	0.9256	0.9256	0.8366	0.6909	0.9256
FDBD	0.9411	0.9406	0.9427	0.9436	0.7579	0.7344	0.8377	0.9998	0.9998	0.8518	0.7647	0.9998
GOT	0.8114	0.7939	0.7955	0.7888	0.7456	0.7456	0.7456	0.8376	0.8376	0.76	0.7456	0.8376
HD	0.951	0.9021	0.9371	0.9441	0.7692	0.7413	0.6503	0.951	0.951	0.7063	0.6434	0.951
HDN	0.9177	0.9175	0.916	0.9175	0.9154	0.9144	0.9144	0.9963	0.9963	0.9192	0.8612	0.9963
HDHID	0.9102	0.9097	0.9077	0.9098	0.9086	0.9058	0.9066	0.9955	0.9955	0.9164	0.8248	0.9955
IM	0.9343	0.9304	0.9317	0.9304	0.9304	0.9304	0.9304	0.9376	0.9376	0.9284	0.9304	0.9376
LOAN	0.9883	0.9865	0.9858	0.9856	0.9812	0.9854	0.9854	0.9998	0.9998	0.9854	0.9809	0.9998
NBCD	0.9417	0.9367	0.9417	0.9397	0.6894	0.7166	0.5729	0.9427	0.9427	0.9246	0.591	0.9427
BPCD	0.9739	0.9507	0.9567	0.9619	0.7672	0.6201	0.6261	0.9851	0.9851	0.709	0.6485	0.9851
PWD	0.9637	0.9634	0.9617	0.9622	0.98	0.9586	0.9127	0.9897	0.9897	0.9747	0.5856	0.9897
QARD	0.9994	0.9917	0.9911	0.9947	0.9994	0.9514	0.9982	0.9994	0.9994	0.9247	0.8162	0.9994
STD	0.8098	0.8085	0.8071	0.8094	0.7893	0.7674	0.7705	0.8098	0.8098	0.7246	0.7585	0.8098
ACP	0.98	0.935	0.9625	0.935	0.6475	0.6425	0.6425	0.9975	0.9975	0.7375	0.635	0.9975
HON	0.993	0.986	0.993	0.993	0.8462	0.7552	0.6084	0.993	0.993	0.7343	0.7063	0.993
TSC	0.8697	0.8495	0.8316	0.8533	0.7477	0.7346	0.7617	0.9974	0.9974	0.7806	0.7312	0.9974
BTD	0.9118	0.8944	0.8984	0.9011	0.7687	0.762	0.762	0.9318	0.9318	0.7874	0.762	0.9318
TMD	0.9966	0.9953	0.9966	0.9969	0.9906	0.9866	0.9877	0.9971	0.9971	0.9904	0.2507	0.9971
VSD	0.95	0.8	0.95	0.85	0.75	0.85	0.75	0.95	0.95	0.8	0.75	0.95
WCR	0.8516	0.8497	0.845	0.8496	0.4293	0.7806	0.803	0.9997	0.9997	0.8057	0.7925	0.9997
WQD	0.9759	0.964	0.9547	0.9555	0.7938	0.7527	0.7527	0.9998	0.9998	0.9913	0.7566	0.9998

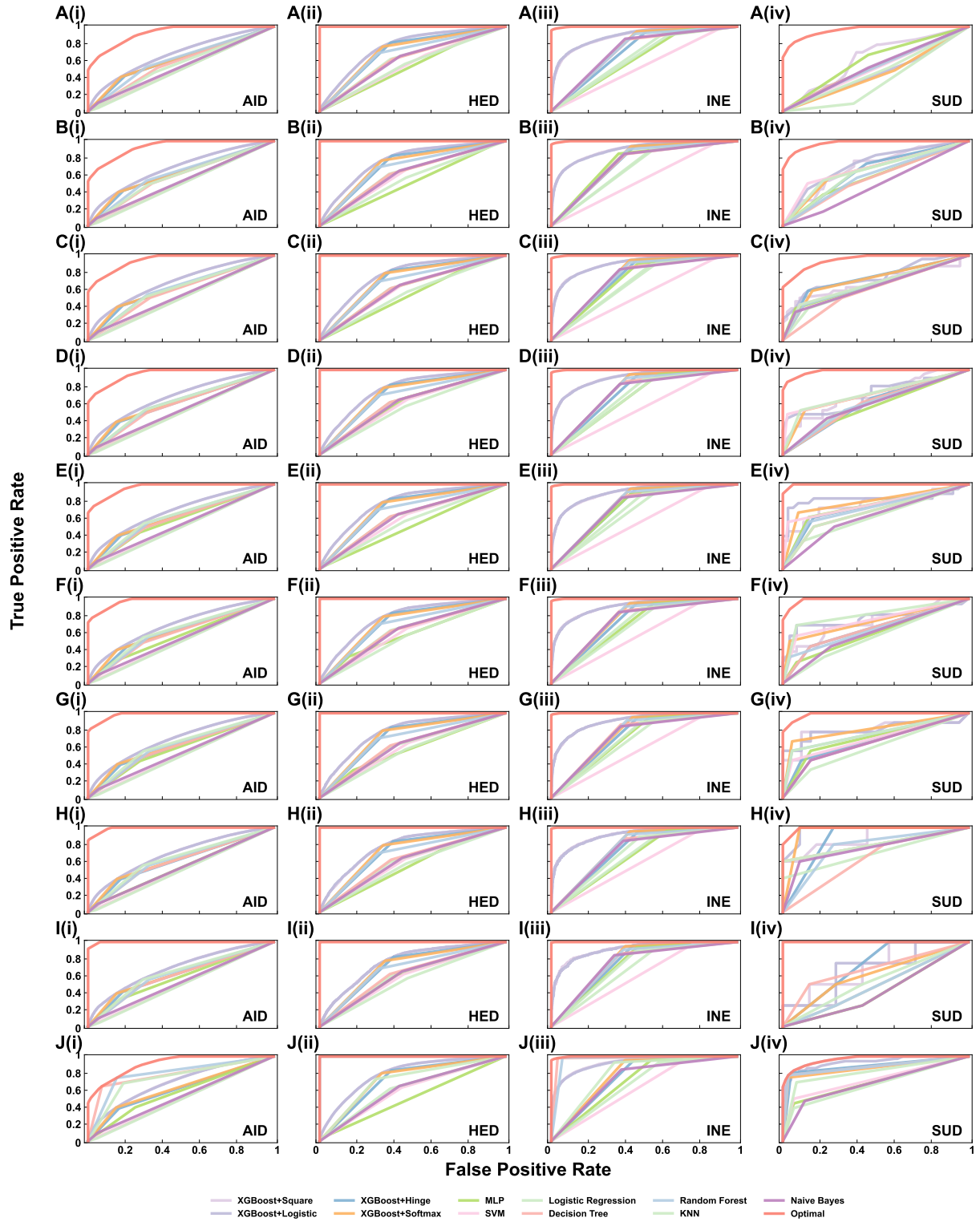


Figure S1. Exact upper bound of AUC and corresponding optimal ROC curves for four real-world datasets when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I), $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

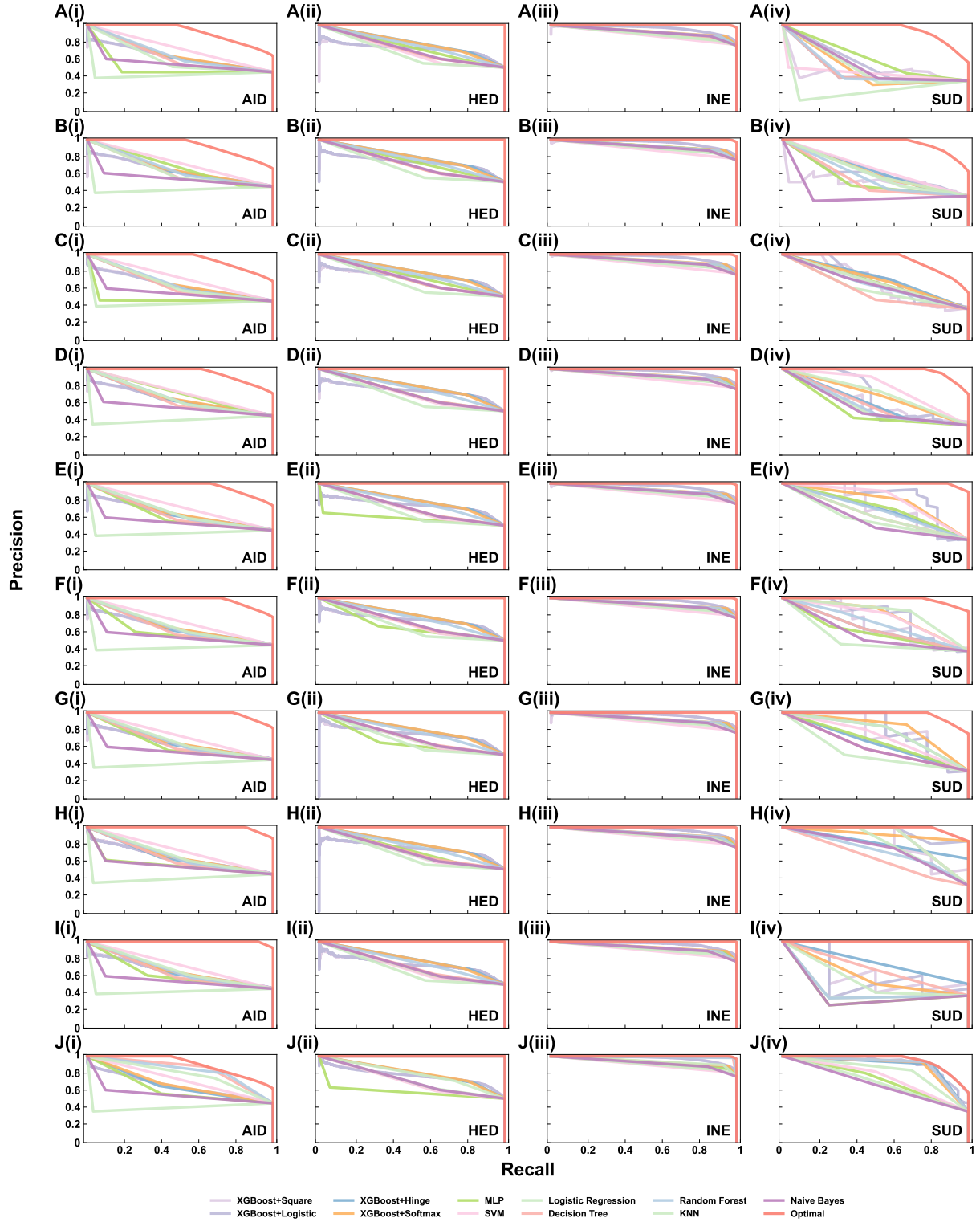


Figure S2. Exact upper bound of AP and corresponding optimal PR curves for four real-world datasets when $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.1$ (A), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.2$ (B), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.3$ (C), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.4$ (D), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.5$ (E), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.6$ (F), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.7$ (G), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.8$ (H), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.9$ (I), $|\mathcal{S}_{train}|/|\mathcal{S}| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal PR curves.

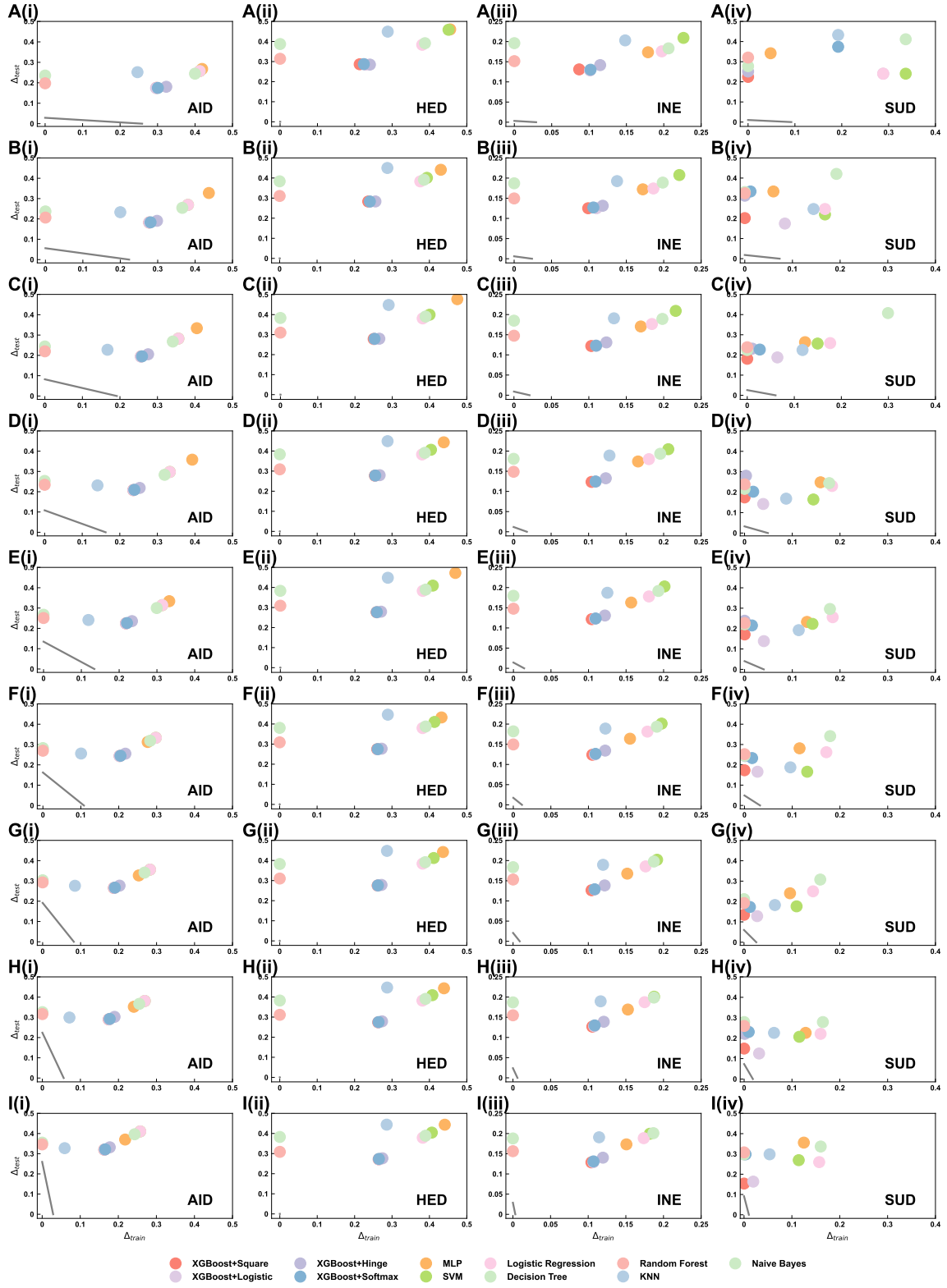


Figure S4. The loss errors of four datasets in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.1$ (A), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.2$ (B), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.3$ (C), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.4$ (D), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.5$ (E), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.6$ (F), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.7$ (G), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.8$ (H), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

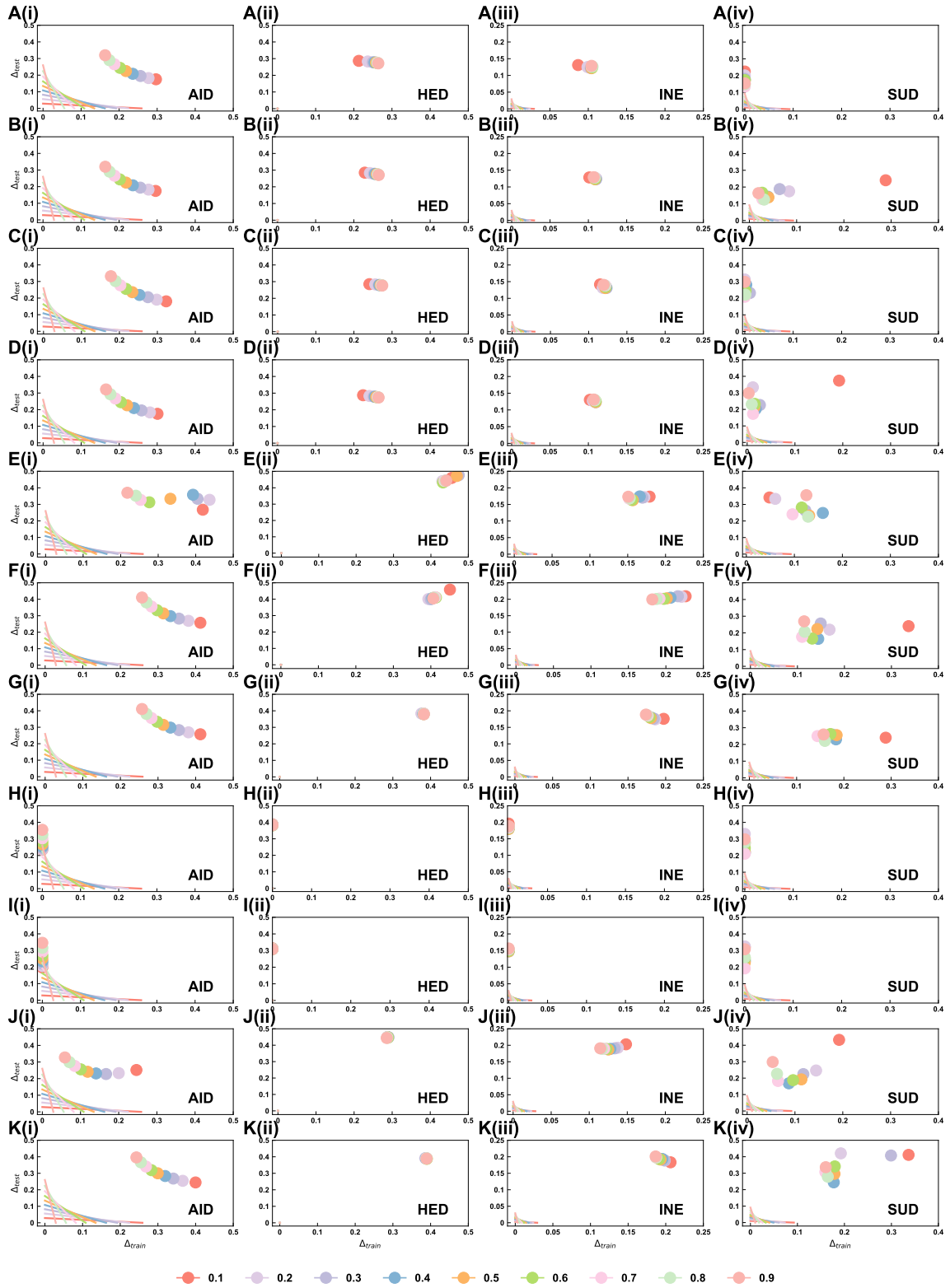


Figure S5. The loss errors of four datasets (AID, HED, INE and SUD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

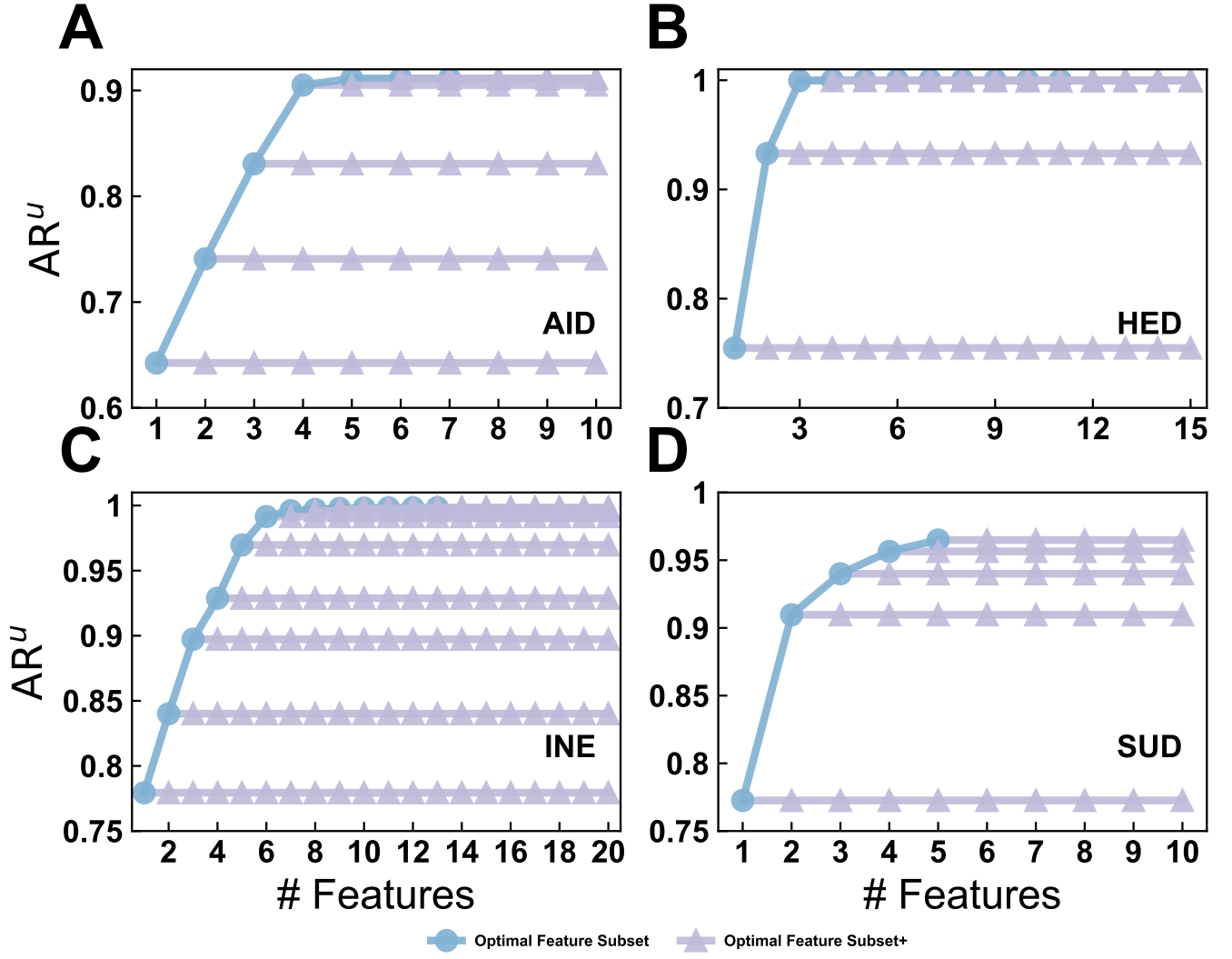


Figure S6. The $AR^u_{k_0}$ versus the optimal k_0 feature subset in feature selection (blue lines and dots). After we selected the optimal k_0 feature subset, we would use the feature extraction skill (LDA) to create new extracted features and add them into the original k_0 feature one by one (see red lines and dots). The datasets we used in this experiment includes AID (A), HED (B), INE (C) and SUD (D).

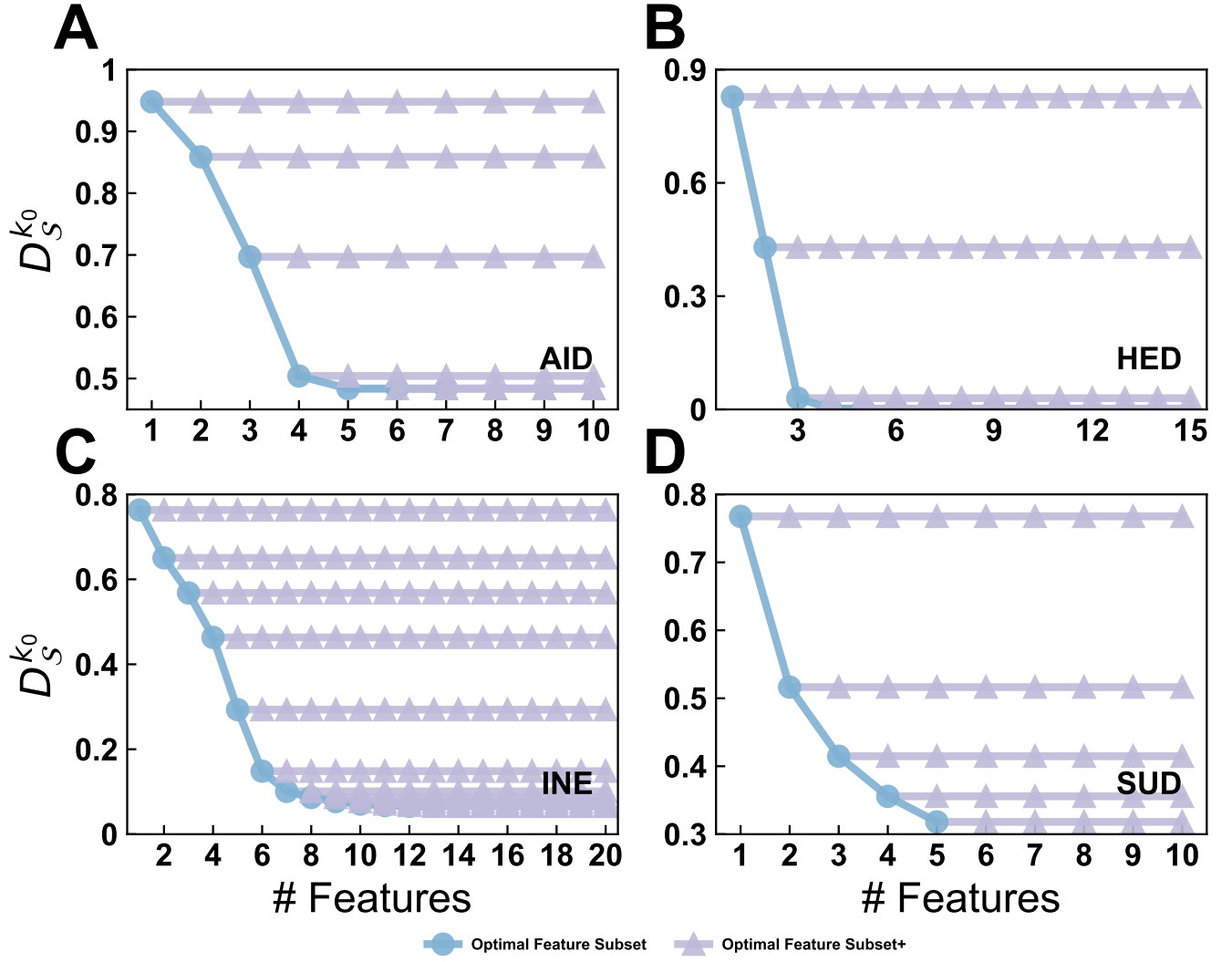


Figure S7. The $D_S^{k_0}$ versus the optimal k_0 feature subset in feature selection (blue lines and dots). After we selected the optimal k_0 feature subset, we would use the feature extraction skill (LDA) to create new extracted features and add them into the original k_0 feature one by one (see red lines and dots). The datasets we used in this experiment includes AID (A), HED (B), INE (C) and SUD (D).

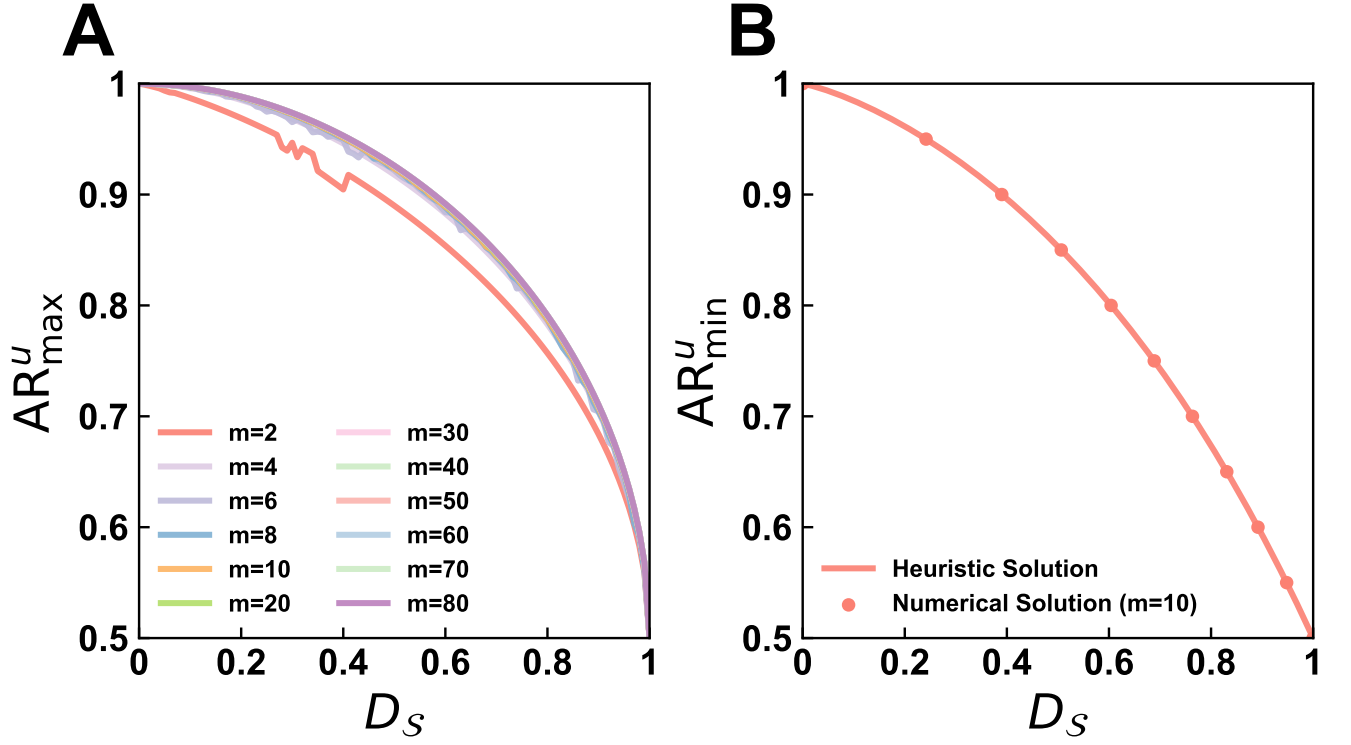


Figure S8. $AR^u_{\max}(D_S)$ curve (A) and $AR^u_{\min}(D_S)$ curve (B). (A) $AR^u_{\max}(D_S)$ curves is numerically solved by the SLSQP solver when $m = 2, 4, 6, 8, 10, 20, 30, 40, 50, 60, 70, 80$. We find that these curves are quickly converged as m increases. Therefore the final $AR^u_{\max}(D_S)$ curve can be approximated to the numerically solved curve when $m = 10$. (B) The heuristic curve for $AR^u_{\min}(D_S)$ is calculated by (107). And the numerically approximated $AR^u_{\min}(D_S)$ curve is derived by the SLSQP solver.

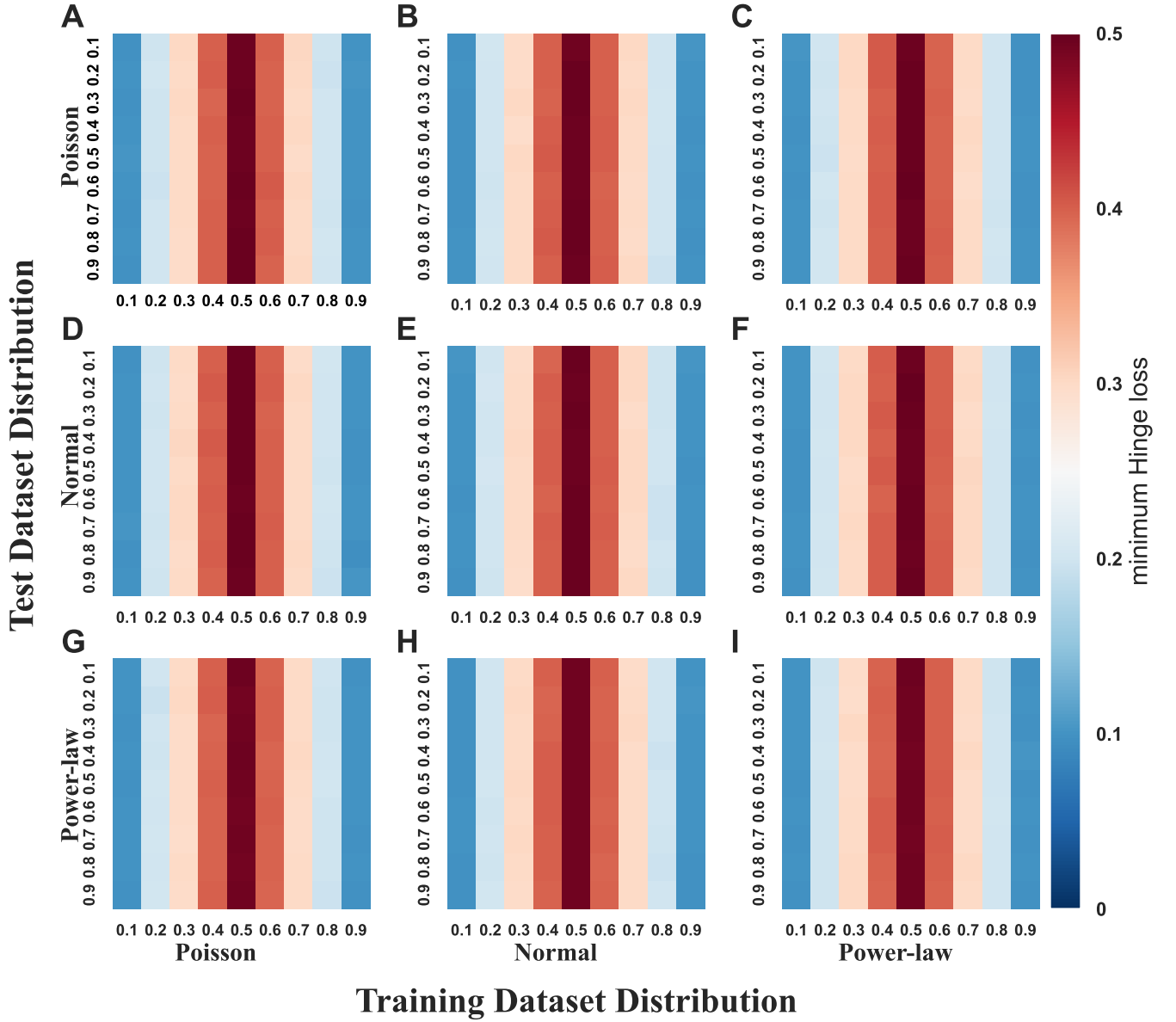


Figure S9. Minimum Hinge loss across synthesized datasets with varied label distributions. Each dataset is bifurcated into distinct training and testing subsets, synthesized using a trio of statistical distributions: Poisson, Normal (Gaussian), and Power-law. Feature vectors for training and testing are generated accordingly. The binary labels are assigned with a probability p for class 1 and $1 - p$ for class 0, where p spans the set $\{0.1, 0.2, \dots, 0.9\}$. This process yields nine unique dataset configurations, denoted as: Poisson & Poisson (A), Normal & Poisson (B), Power-law & Poisson (C), Poisson & Normal (D), Normal & Normal (E), Power-law & Normal (F), Poisson & Power-law (G), Normal & Power-law (H) and Power-law & Power-law (I). Each subset of this matrix is visualized as a heat map, charting the minimal Hinge loss achieved across varying label probabilities within the respective training and testing subsets, under specific distribution pairings. Notably, the minimal Hinge loss for the training subset is computed independently of the testing subset, resulting in identical columns for each heat map.

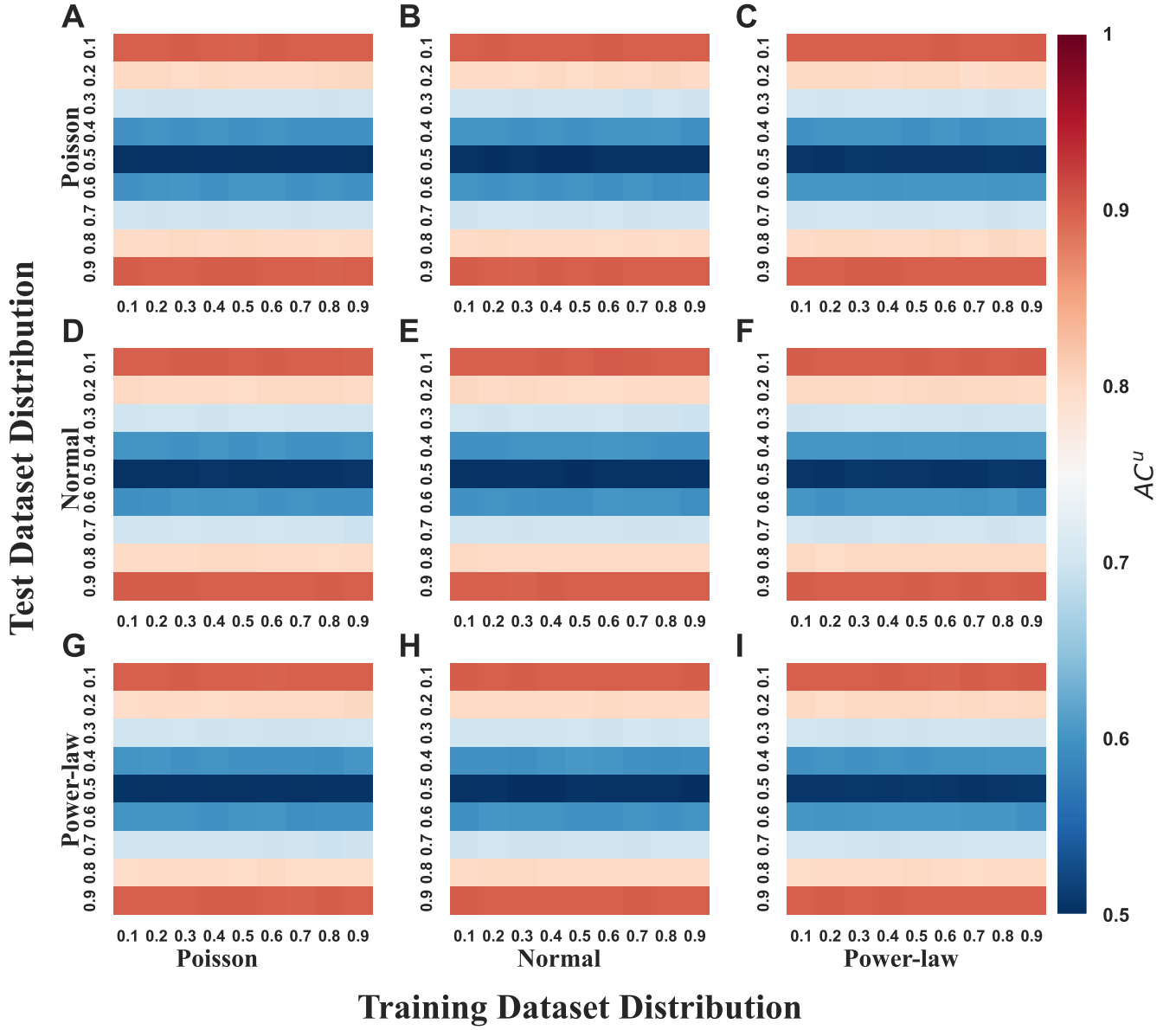


Figure S10. Maximum accuracy (AC^u) across synthesized datasets with varied label distributions. Each dataset is bifurcated into distinct training and testing subsets, synthesized using a trio of statistical distributions: Poisson, Normal (Gaussian), and Power-law. Feature vectors for training and testing are generated accordingly. The binary labels are assigned with a probability p for class 1 and $1-p$ for class 0, where p spans the set $\{0.1, 0.2, \dots, 0.9\}$. This process yields nine unique dataset configurations, denoted as: Poisson & Poisson (A), Normal & Poisson (B), Power-law & Poisson (C), Poisson & Normal (D), Normal & Normal (E), Power-law & Normal (F), Poisson & Power-law (G), Normal & Power-law (H) and Power-law & Power-law (I). Each subset of this matrix is visualized as a heat map, charting the maximum accuracy (AC^u) achieved across varying label probabilities within the respective training and testing subsets, under specific distribution pairings. Notably, the maximum accuracy for the testing subset is computed independently of the training subset, resulting in identical rows for each heat map.

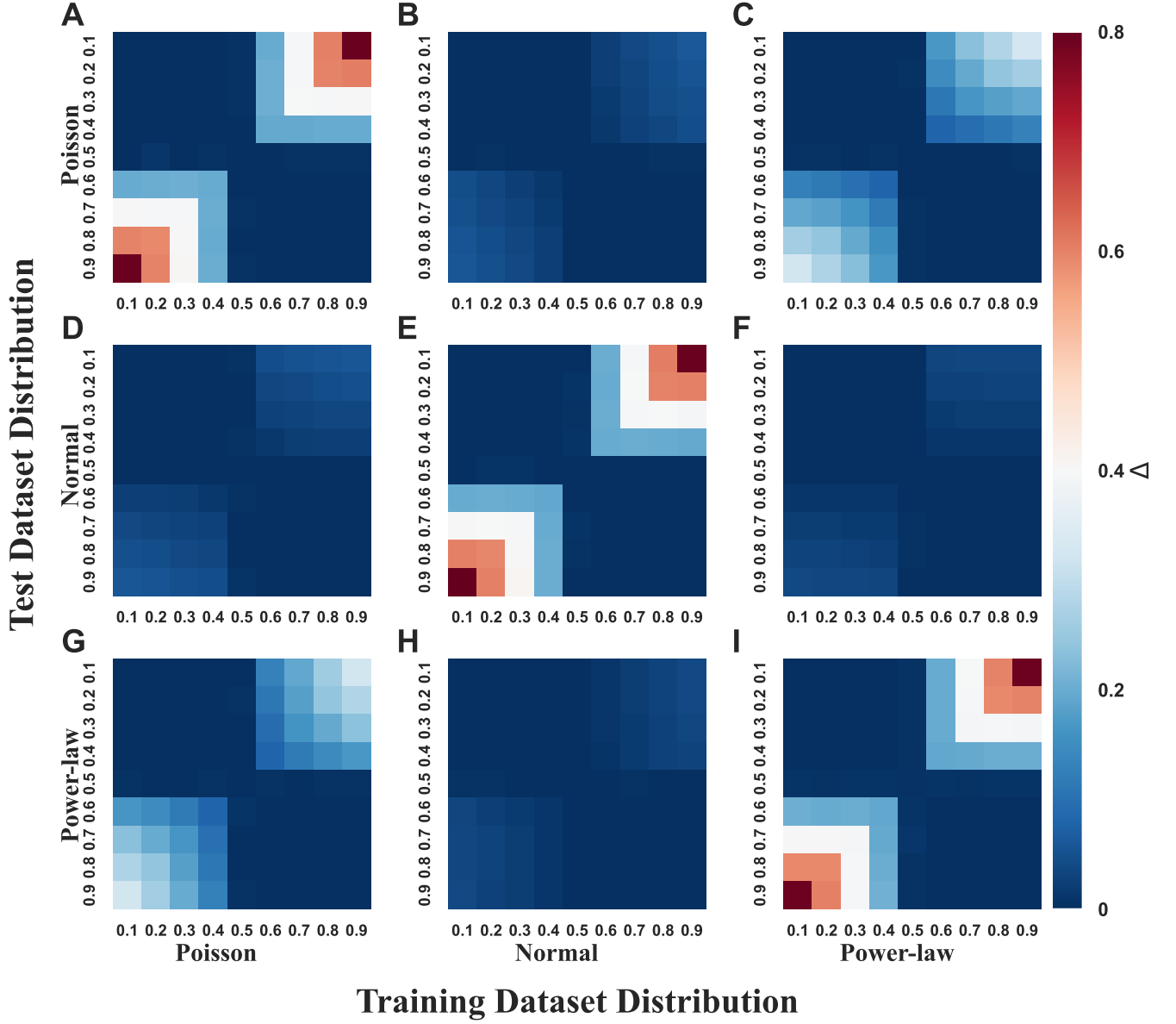


Figure S11. Joint error bound (Δ) across synthesized datasets with varied label distributions. Each dataset is bifurcated into distinct training and testing subsets, synthesized using a trio of statistical distributions: Poisson, Normal (Gaussian), and Power-law. Feature vectors for training and testing are generated accordingly. The binary labels are assigned with a probability p for class 1 and $1-p$ for class 0, where p spans the set $\{0.1, 0.2, \dots, 0.9\}$. This process yields nine unique dataset configurations, denoted as: Poisson & Poisson (A), Normal & Poisson (B), Power-law & Poisson (C), Poisson & Normal (D), Normal & Normal (E), Power-law & Normal (F), Poisson & Power-law (G), Normal & Power-law (H) and Power-law & Power-law (I). Each subset of this matrix is visualized as a heat map, charting the joint error bound (Δ) achieved across varying label probabilities within the respective training and testing subsets, under specific distribution pairings. Notably, Δ for the training and testing subset is computed based on Eq. (54) for each heat map.

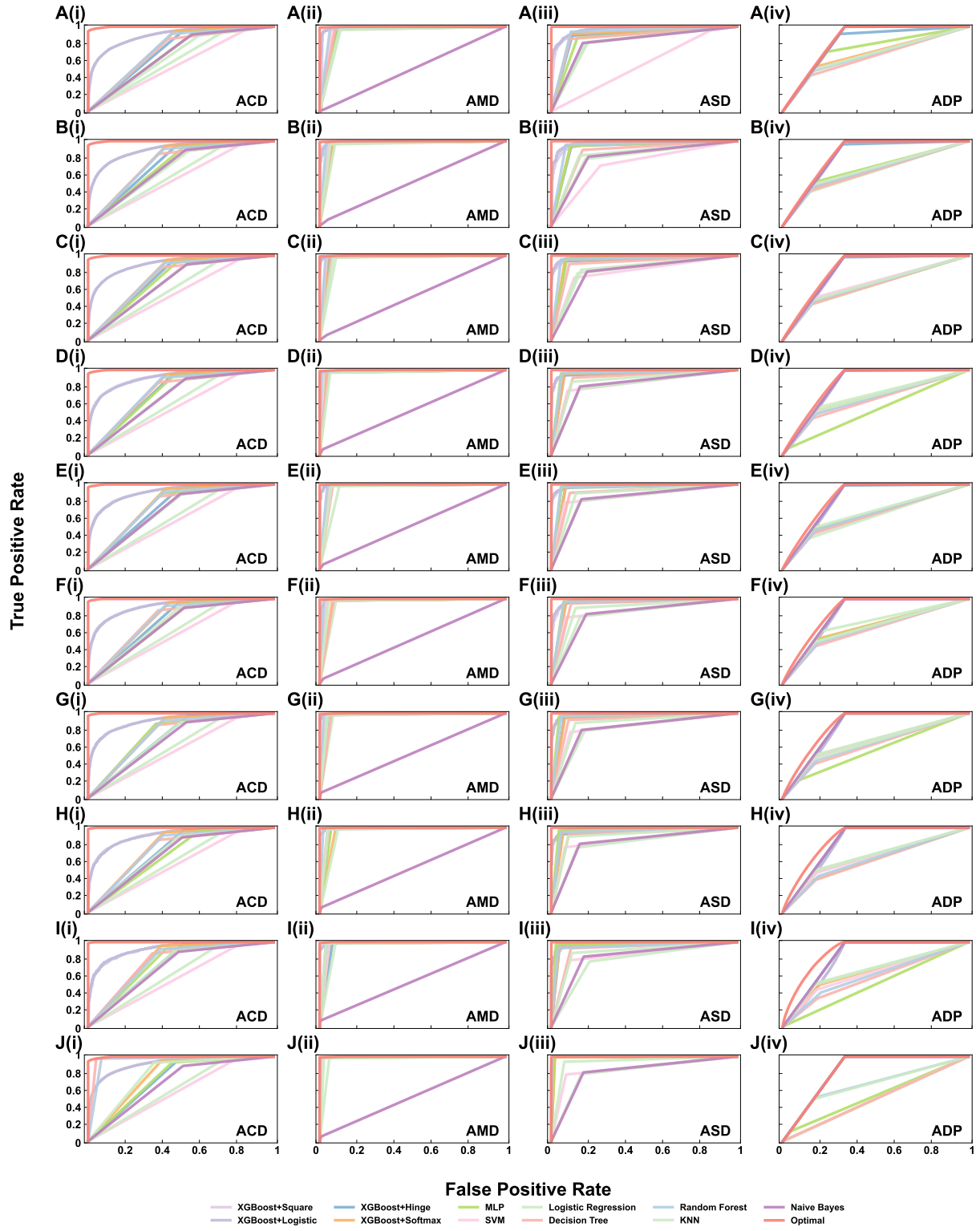


Figure S12. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (ACD, AMD, ASD and ADP) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

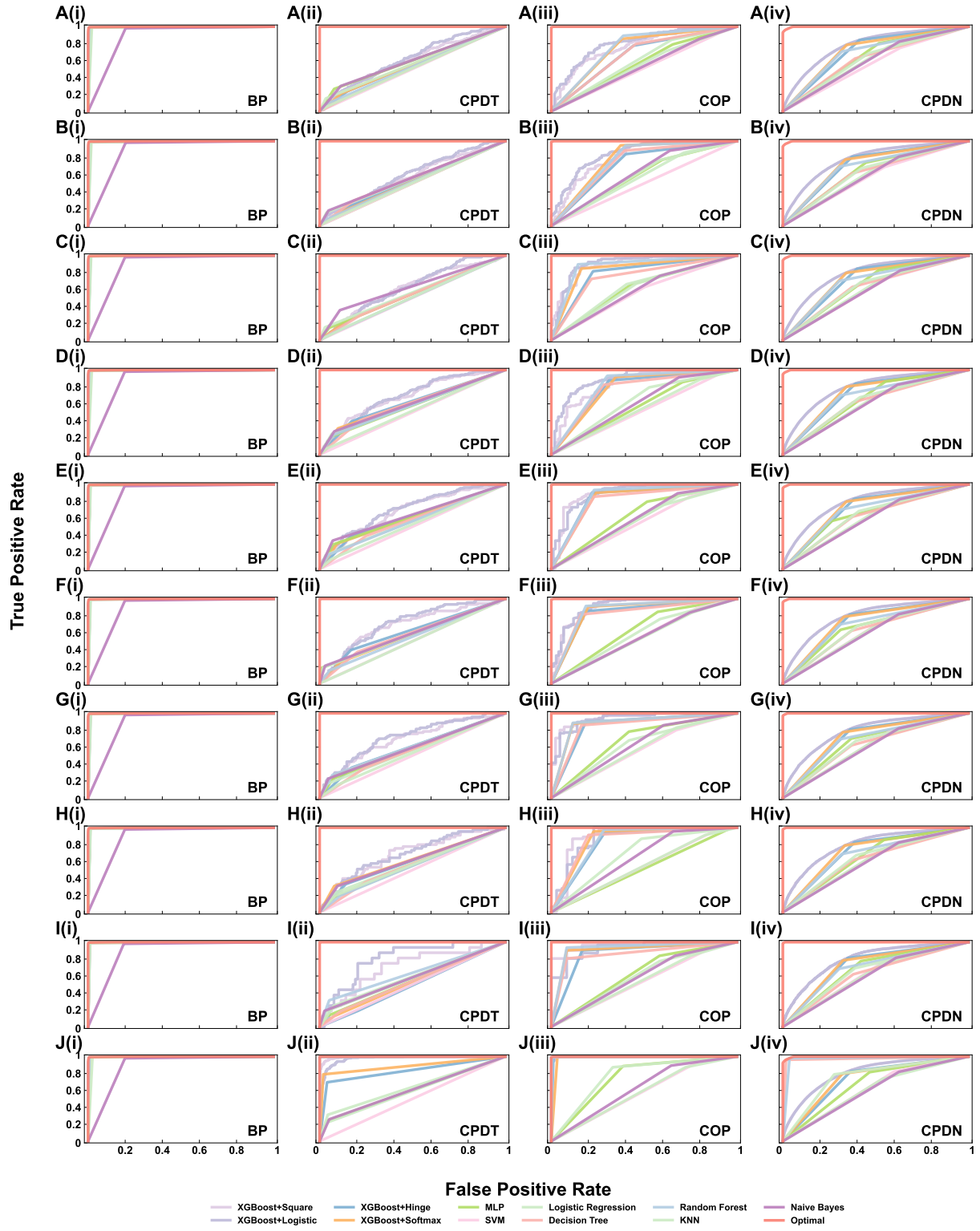


Figure S13. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (BP, CPDT, COP and CPDN) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

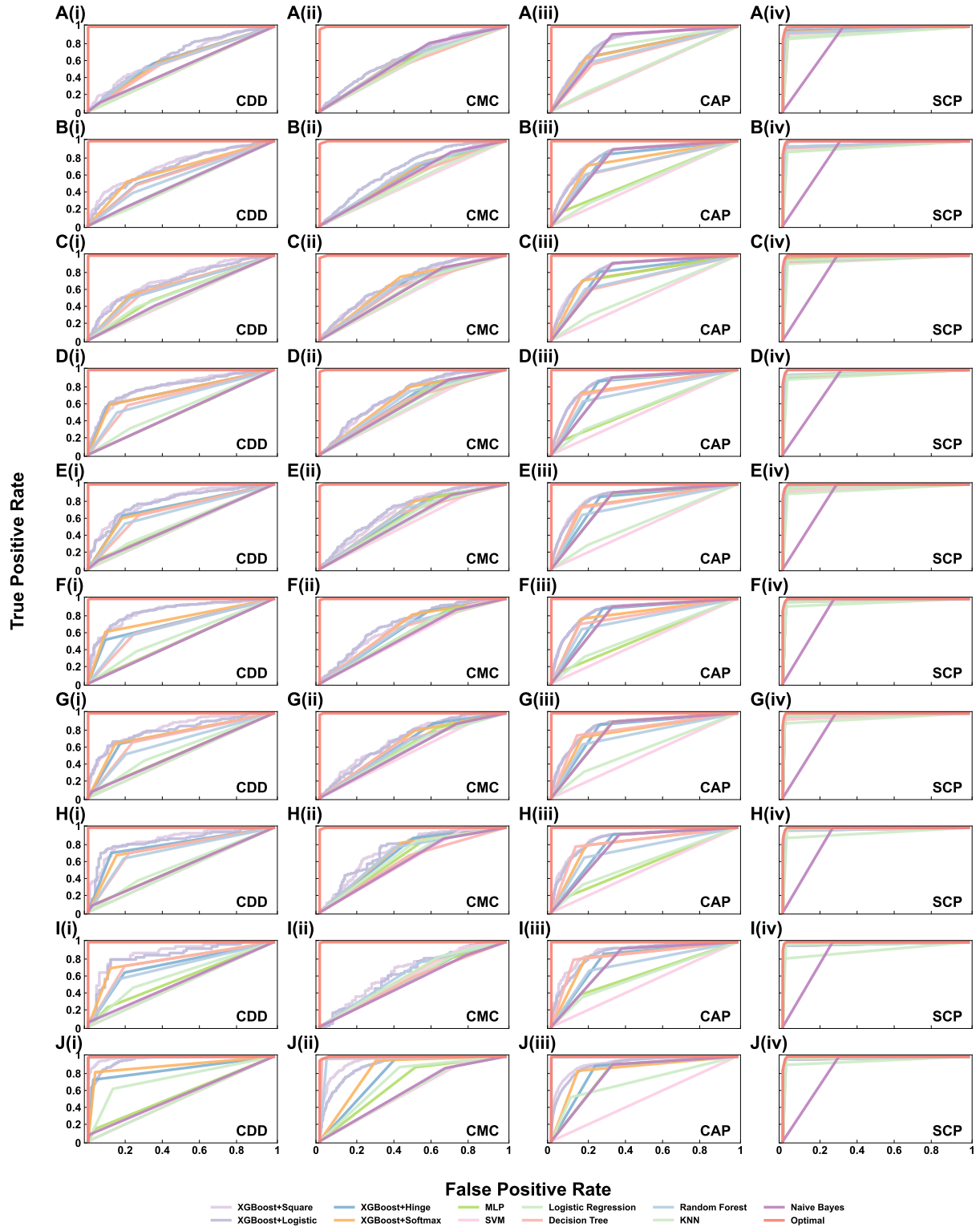


Figure S14. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (CDD, CMC, CAP and SCP) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

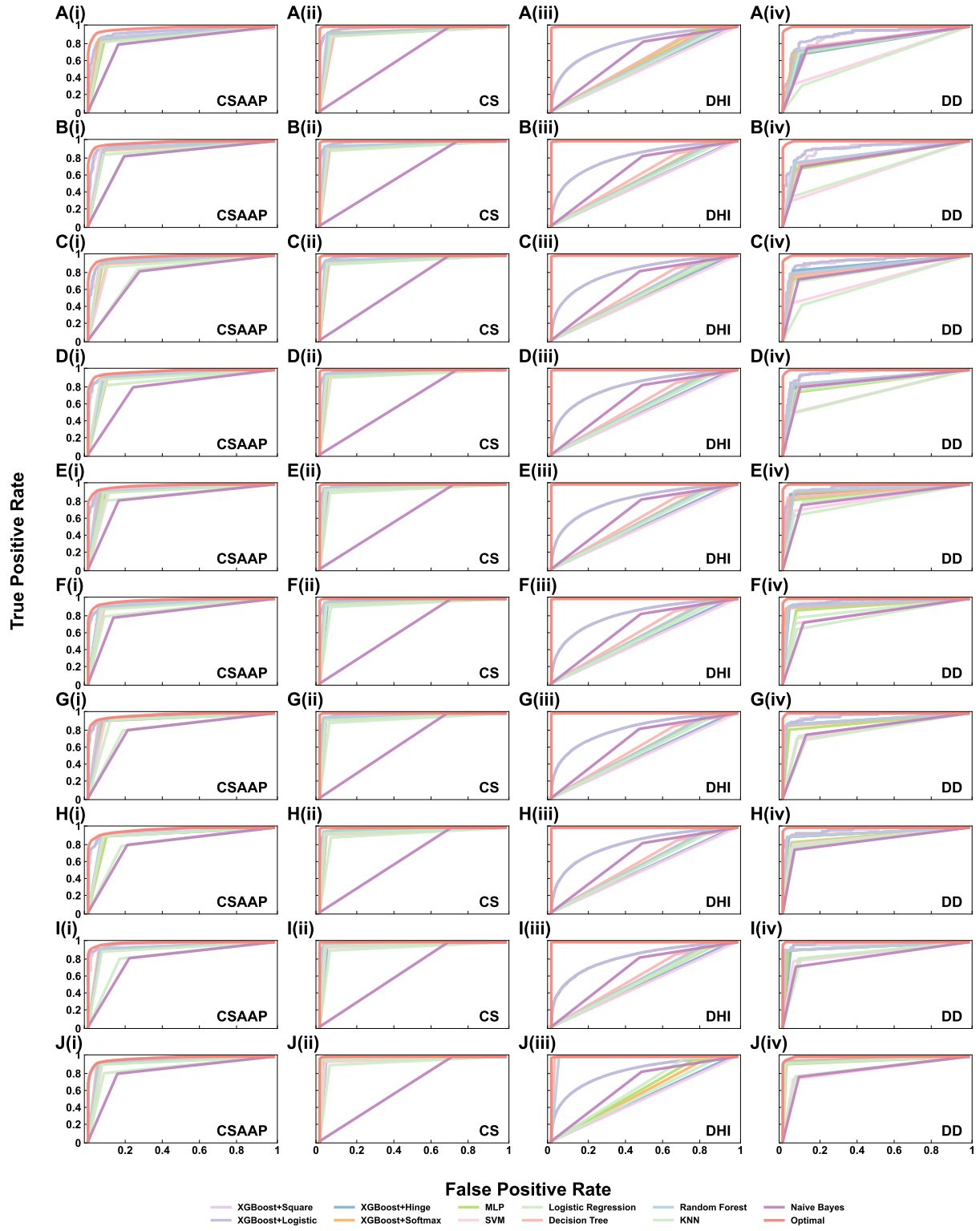


Figure S15. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (CSAAP, CS, DHI and DD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

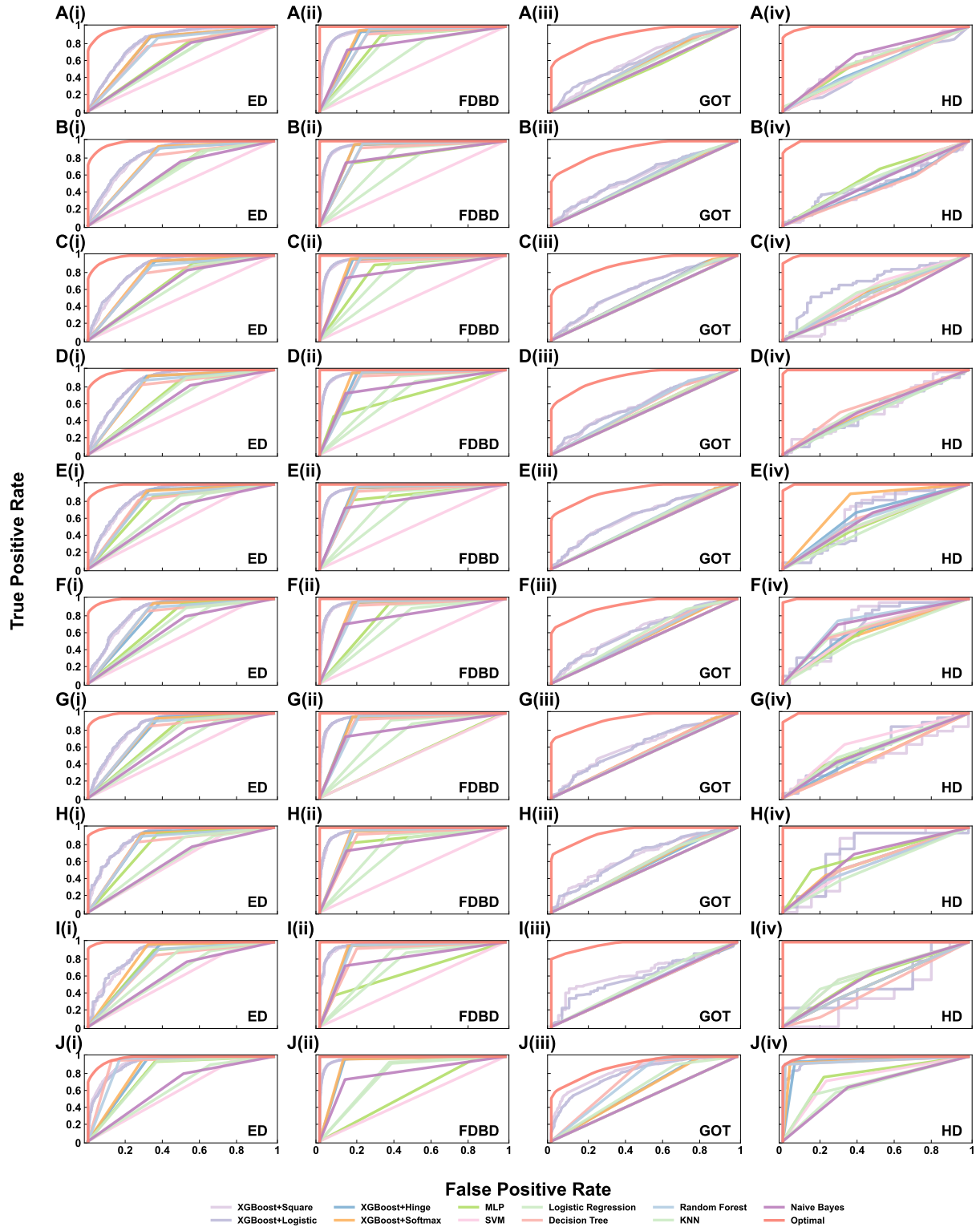


Figure S16. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (ED, FDBD, GOT and HD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

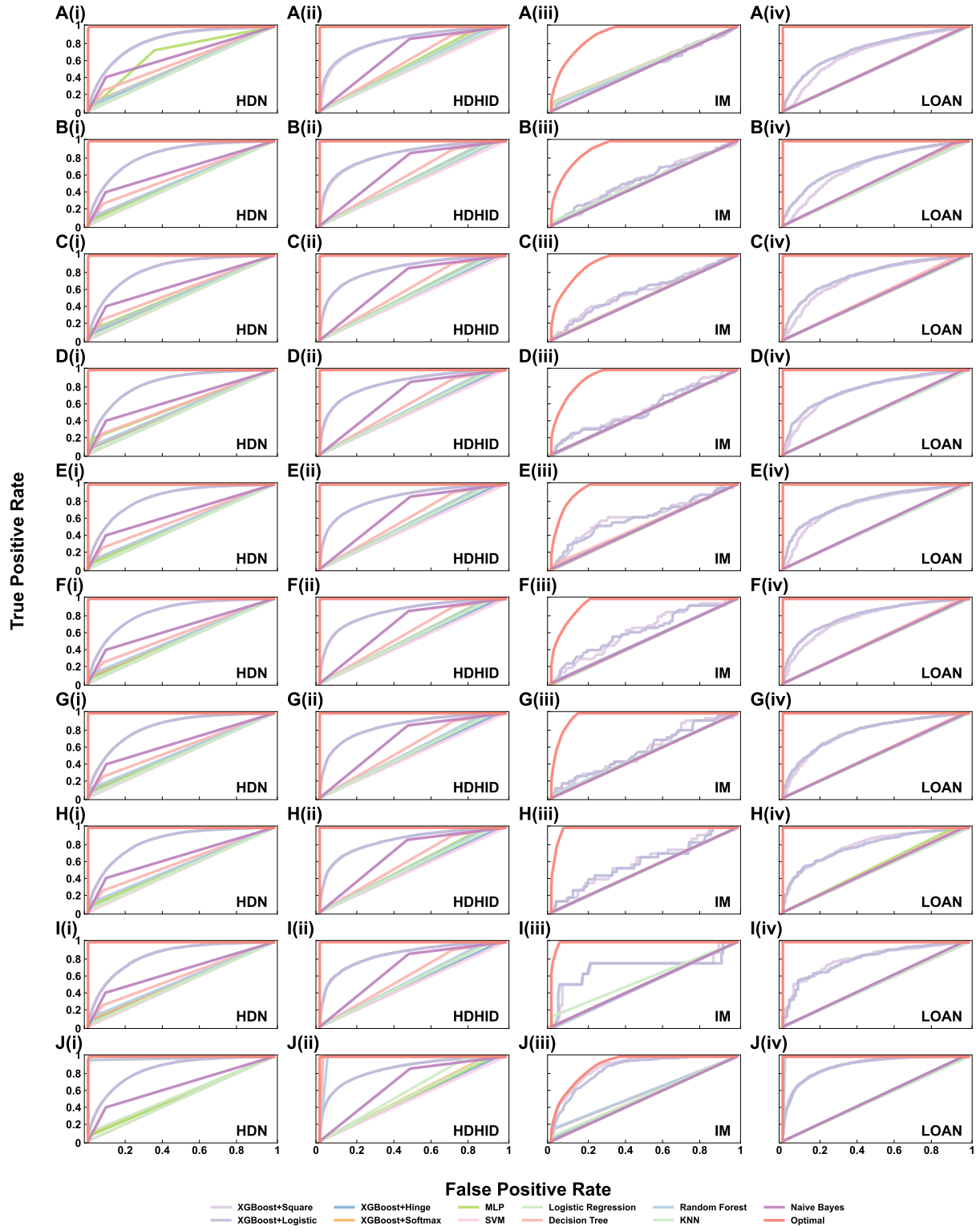


Figure S17. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (HDN, HDHID, IM and LOAN) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

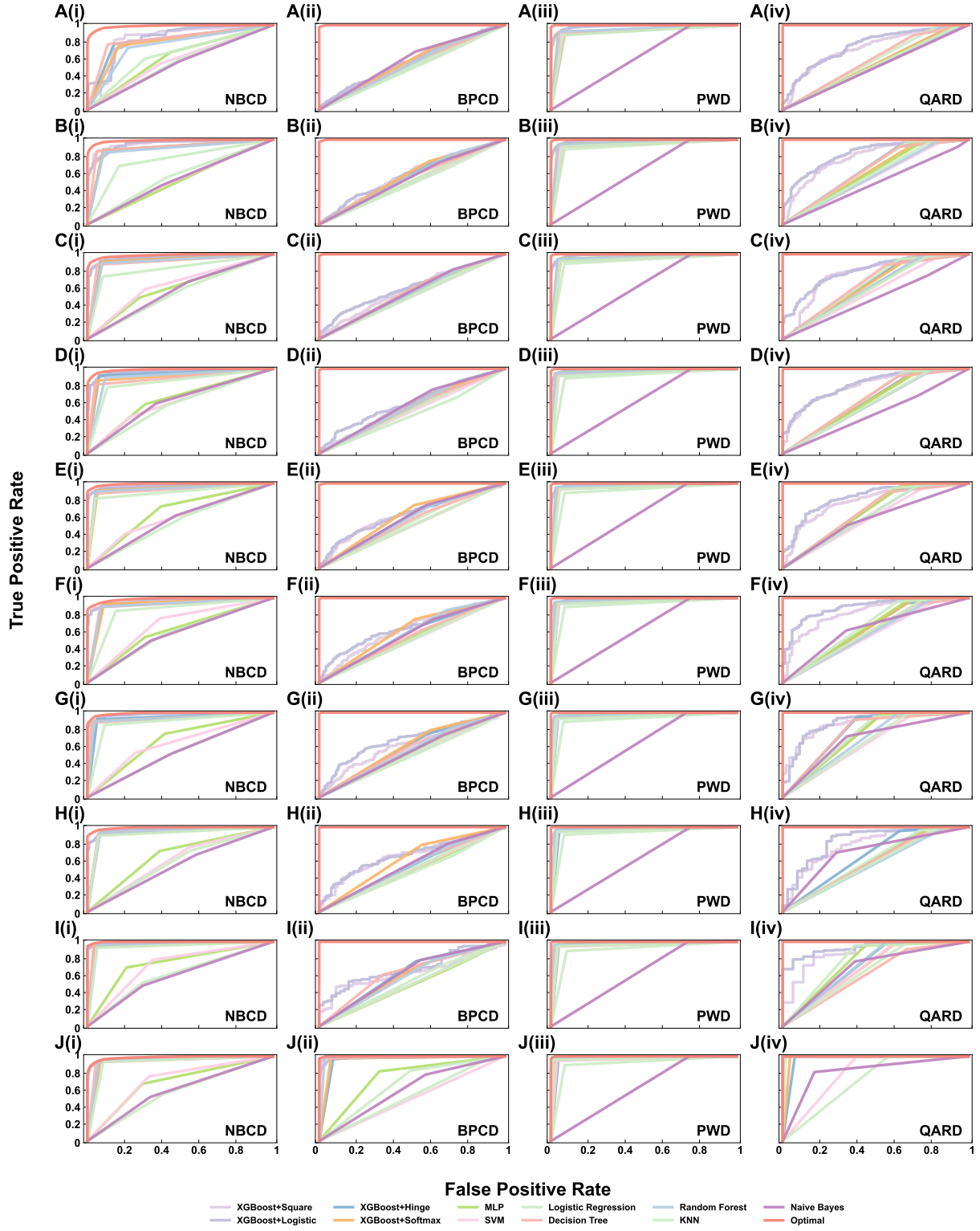


Figure S18. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (NBCD, BPCD, PWD and QARD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

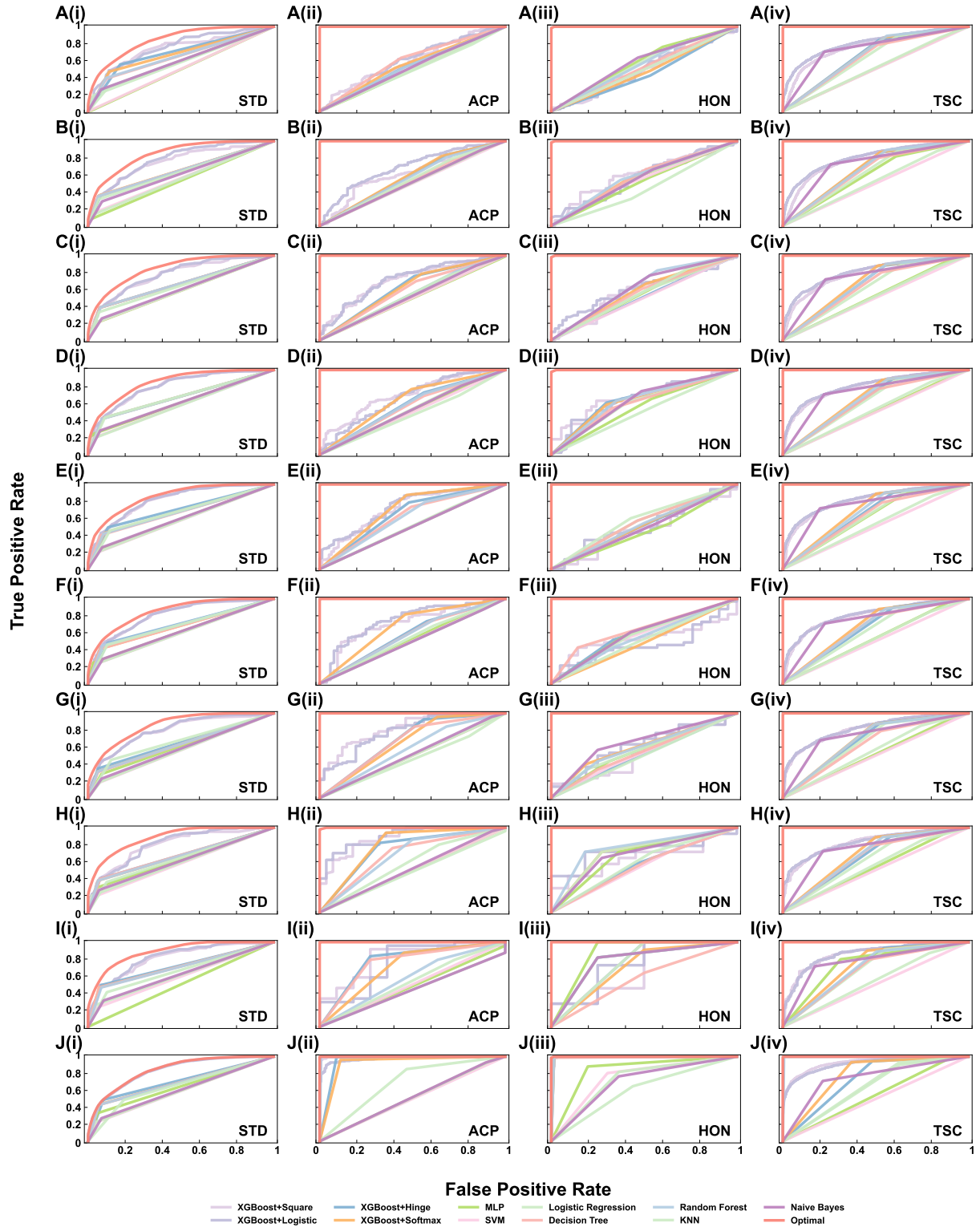


Figure S19. Exact upper bound of AUC and corresponding optimal ROC curves on 4 additional real-world datasets (STD, ACP, HON and TSC) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

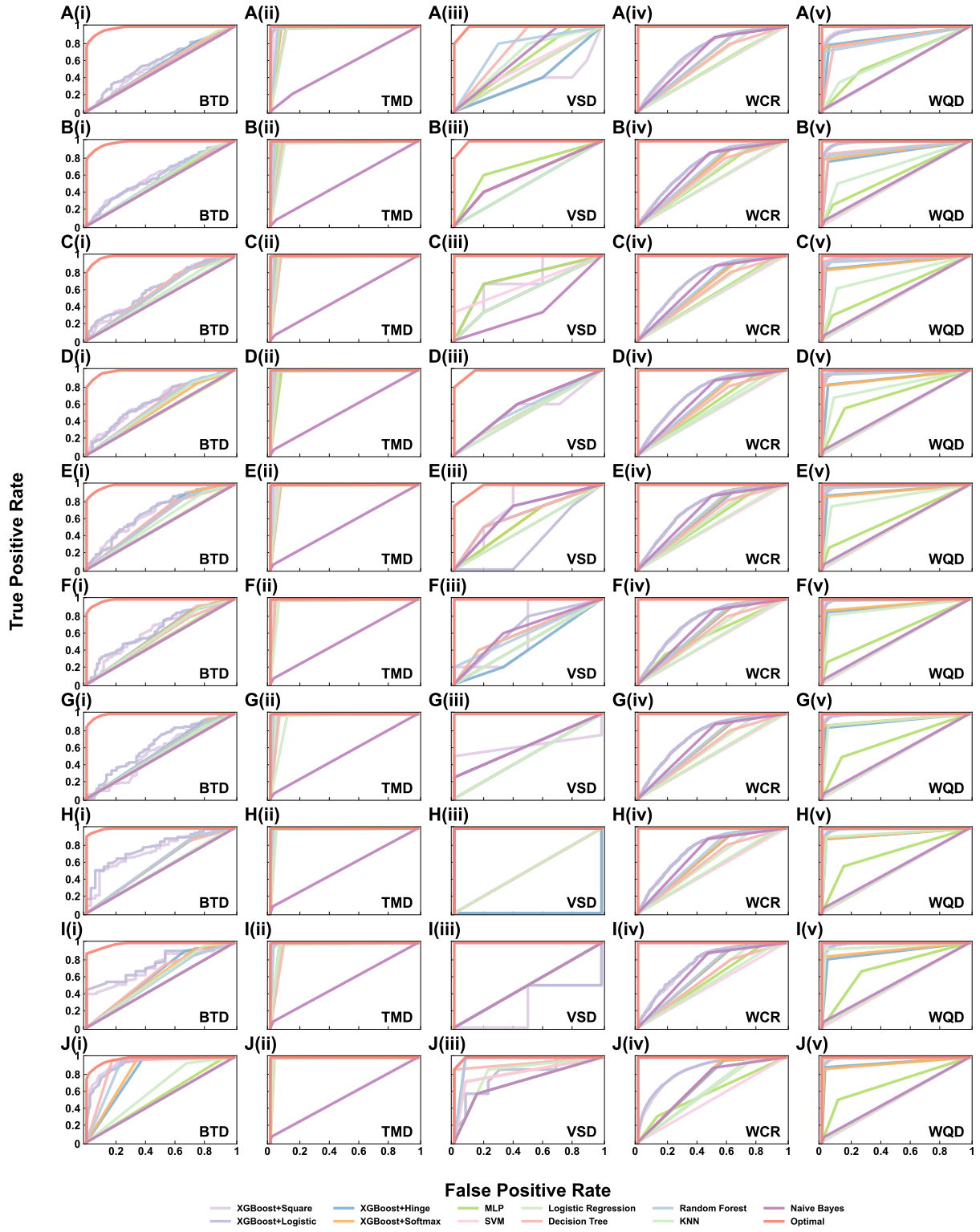


Figure S20. Exact upper bound of AUC and corresponding optimal ROC curves on 5 additional real-world datasets (BT, TMD, VSD, WCR and WQD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

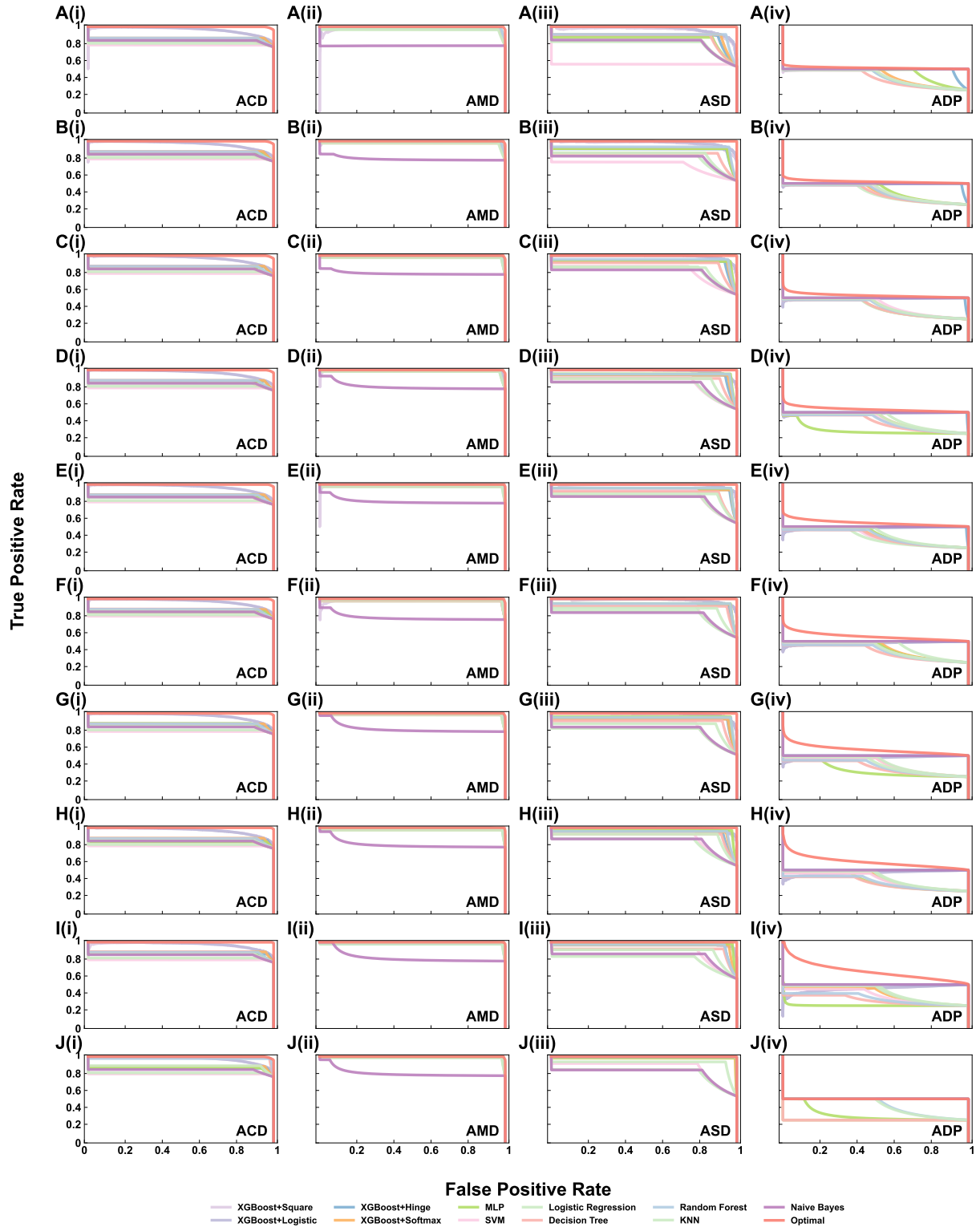


Figure S21. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (ACD, AMD, ASD and ADP) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

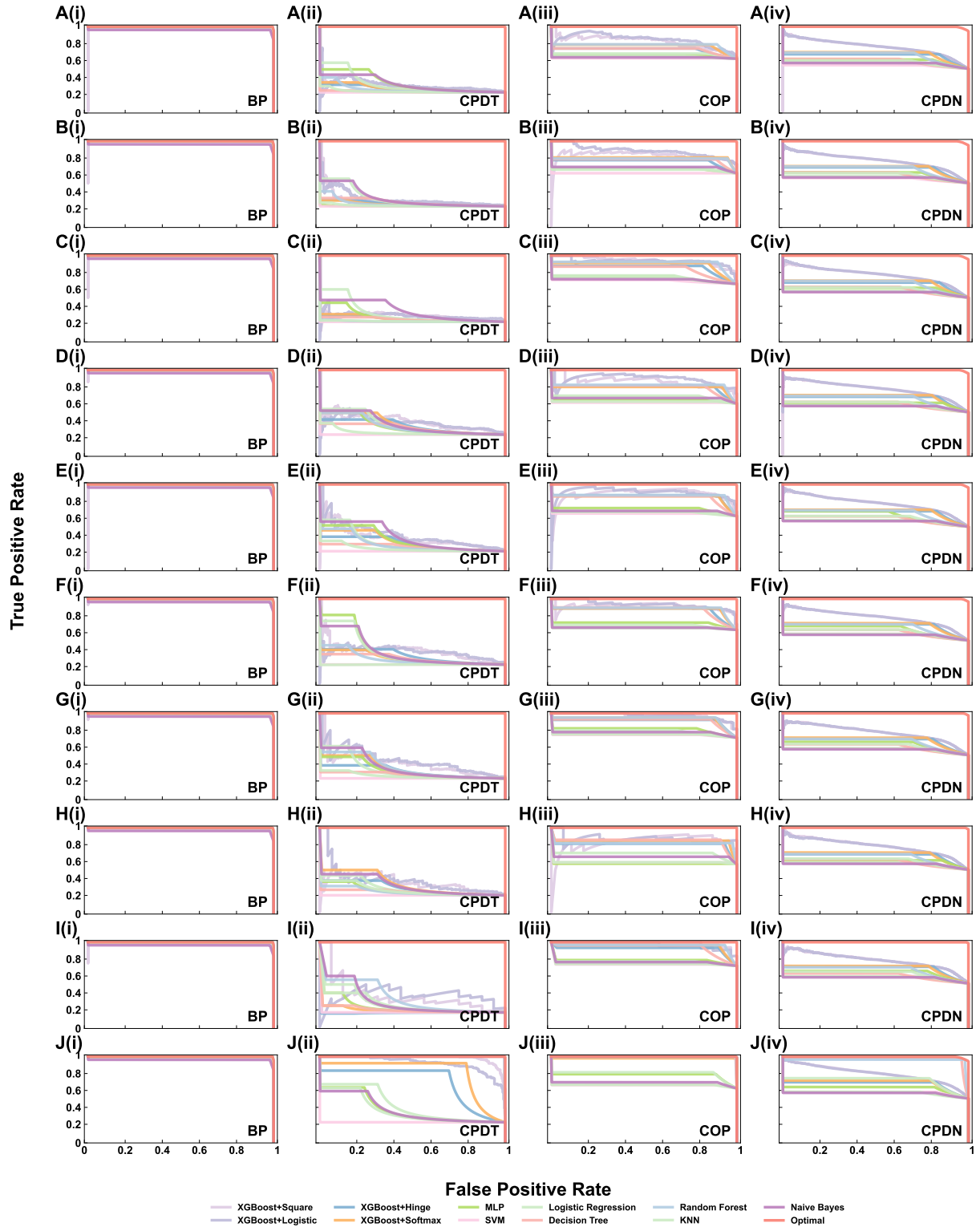


Figure S22. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (BP, CPDT, COP and CPDN) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

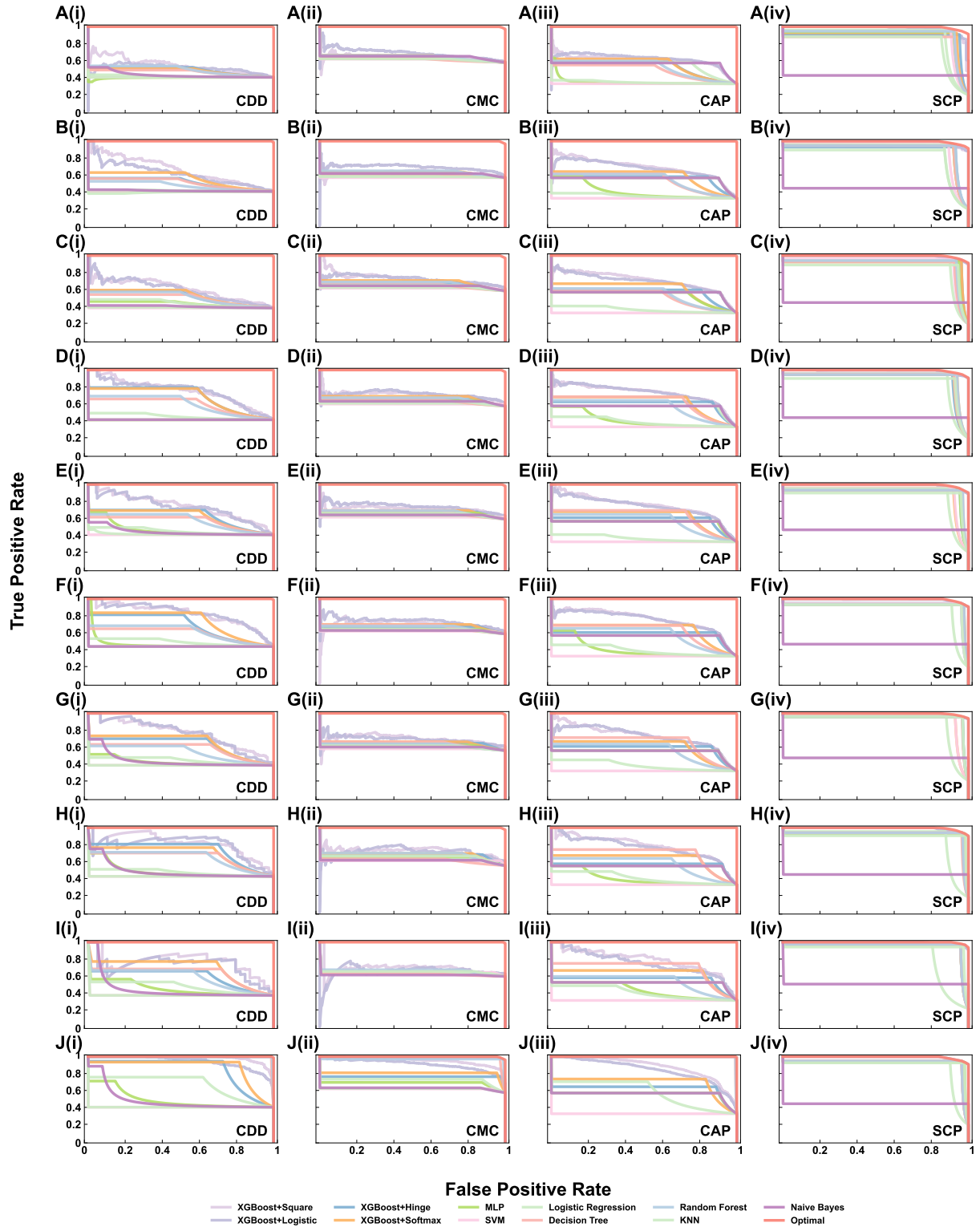


Figure S23. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (CDD, CMC, CAP and SCP) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

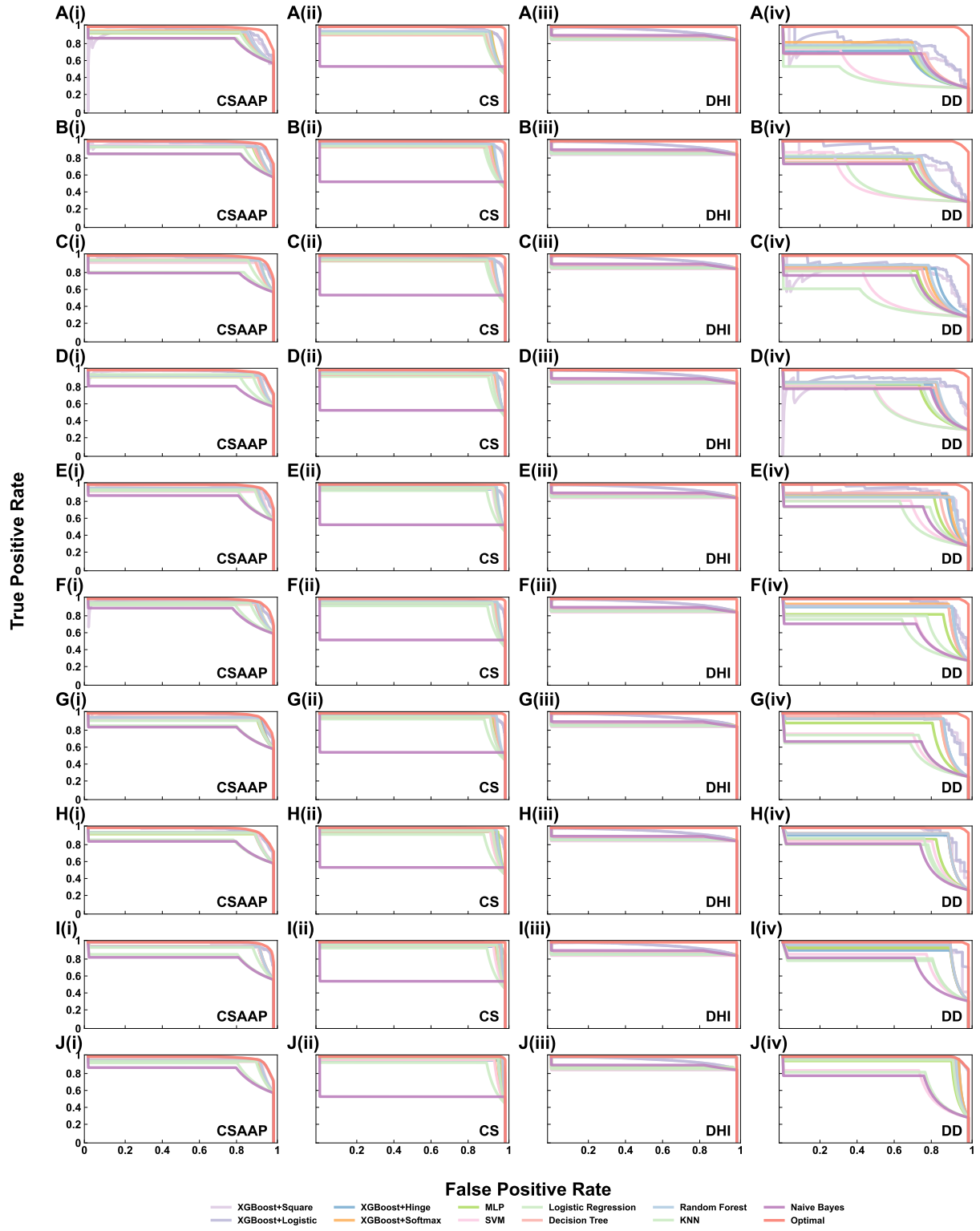


Figure S24. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (CSAAP, CS, DHI and DD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

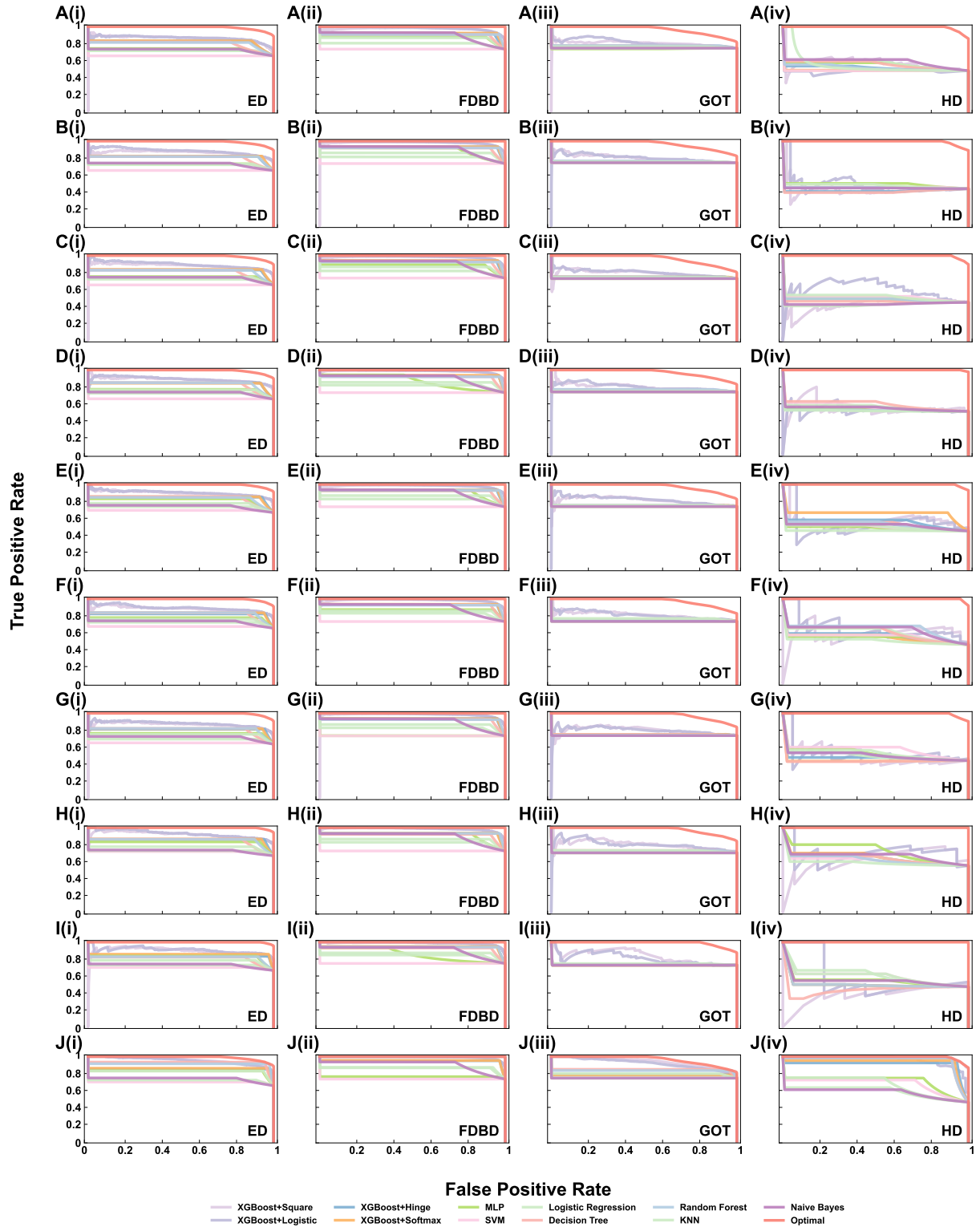


Figure S25. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (ED, FDBD, GOT and HD) when $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.1$ (A), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.2$ (B), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.3$ (C), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.4$ (D), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.5$ (E), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.6$ (F), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.7$ (G), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.8$ (H), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.9$ (I) and $|\mathcal{S}_{train}|/|\mathcal{S}| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

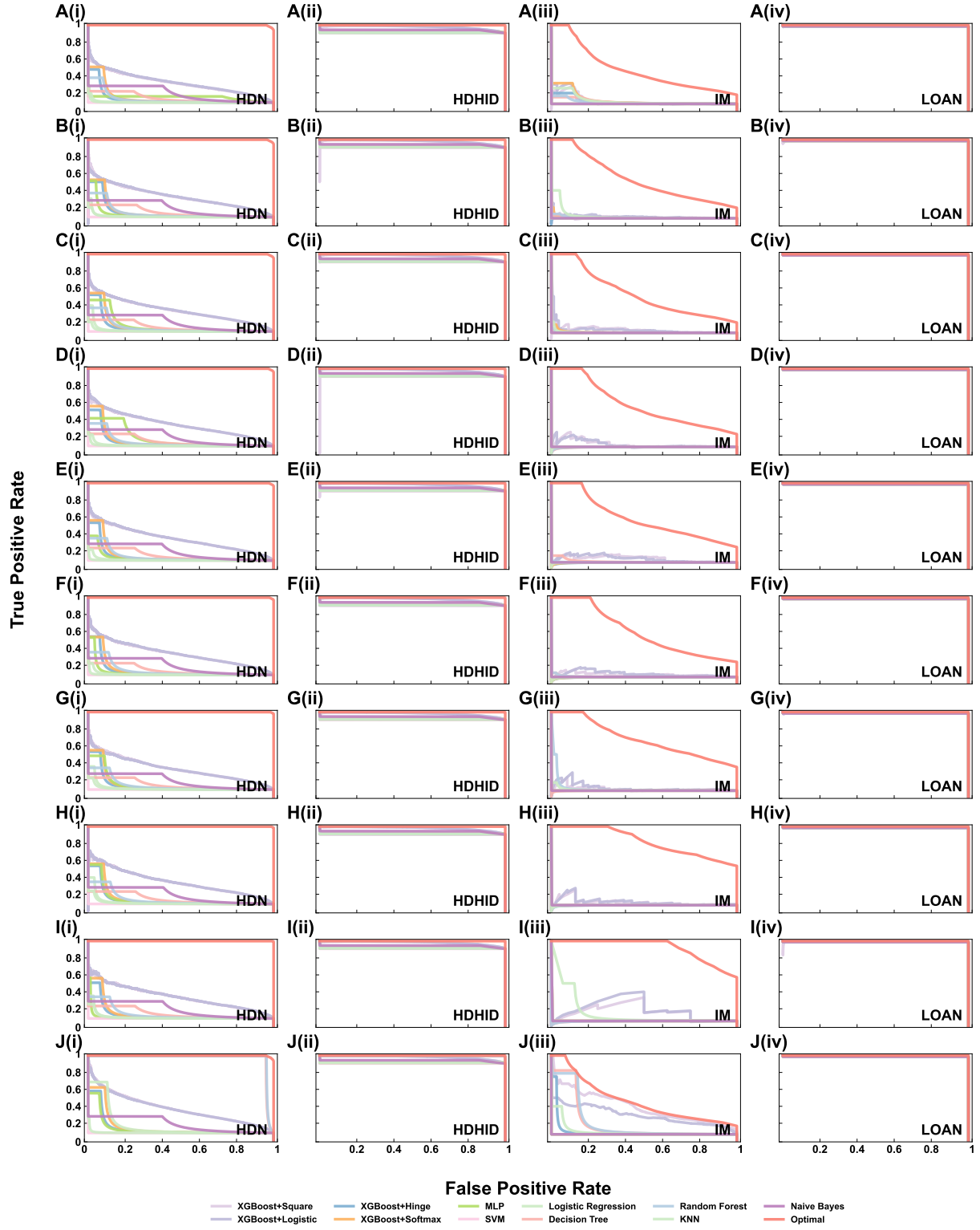


Figure S26. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (HDN, HDHID, IM and LOAN) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

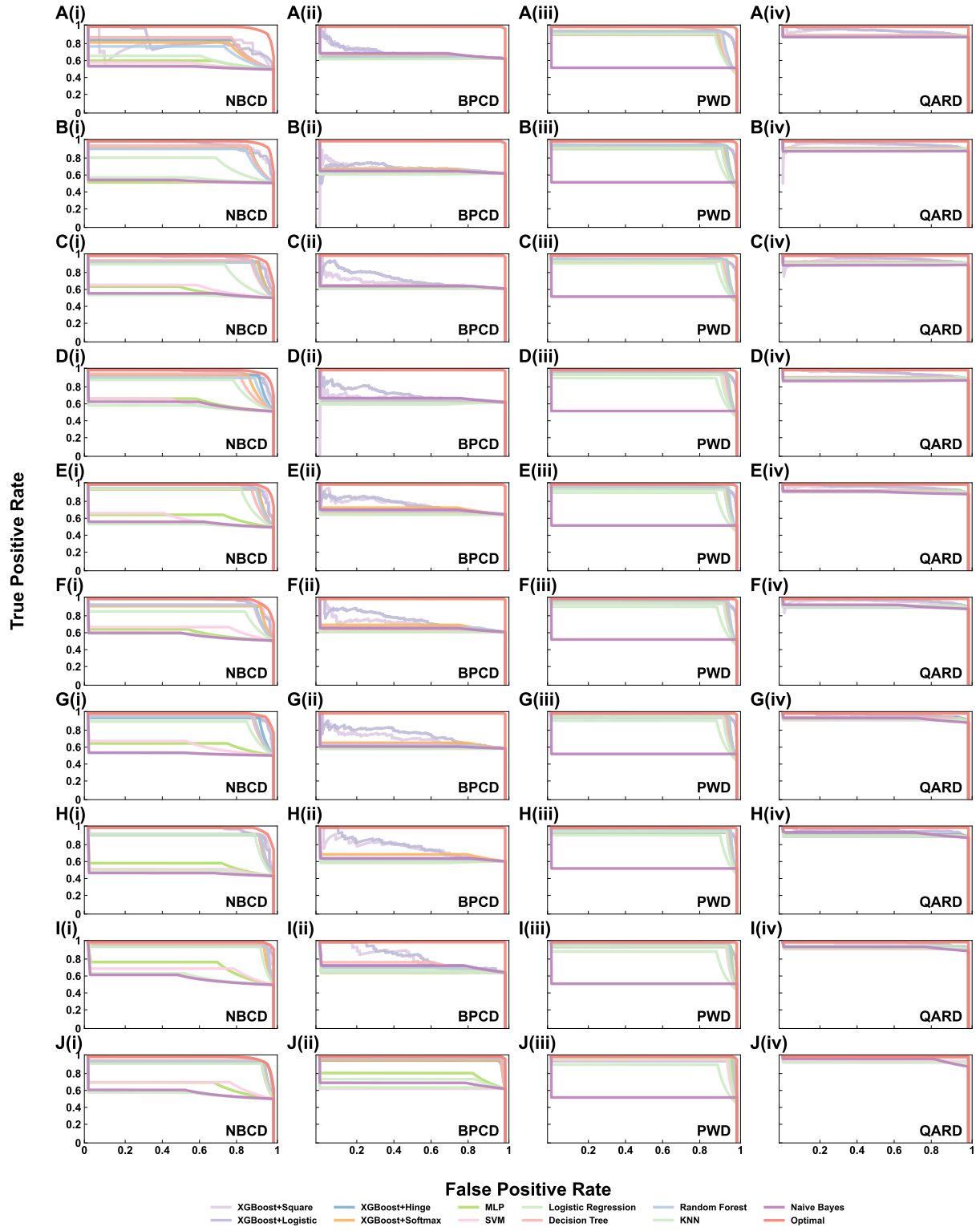


Figure S27. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (NBCD, BPCD, PWD and QARD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

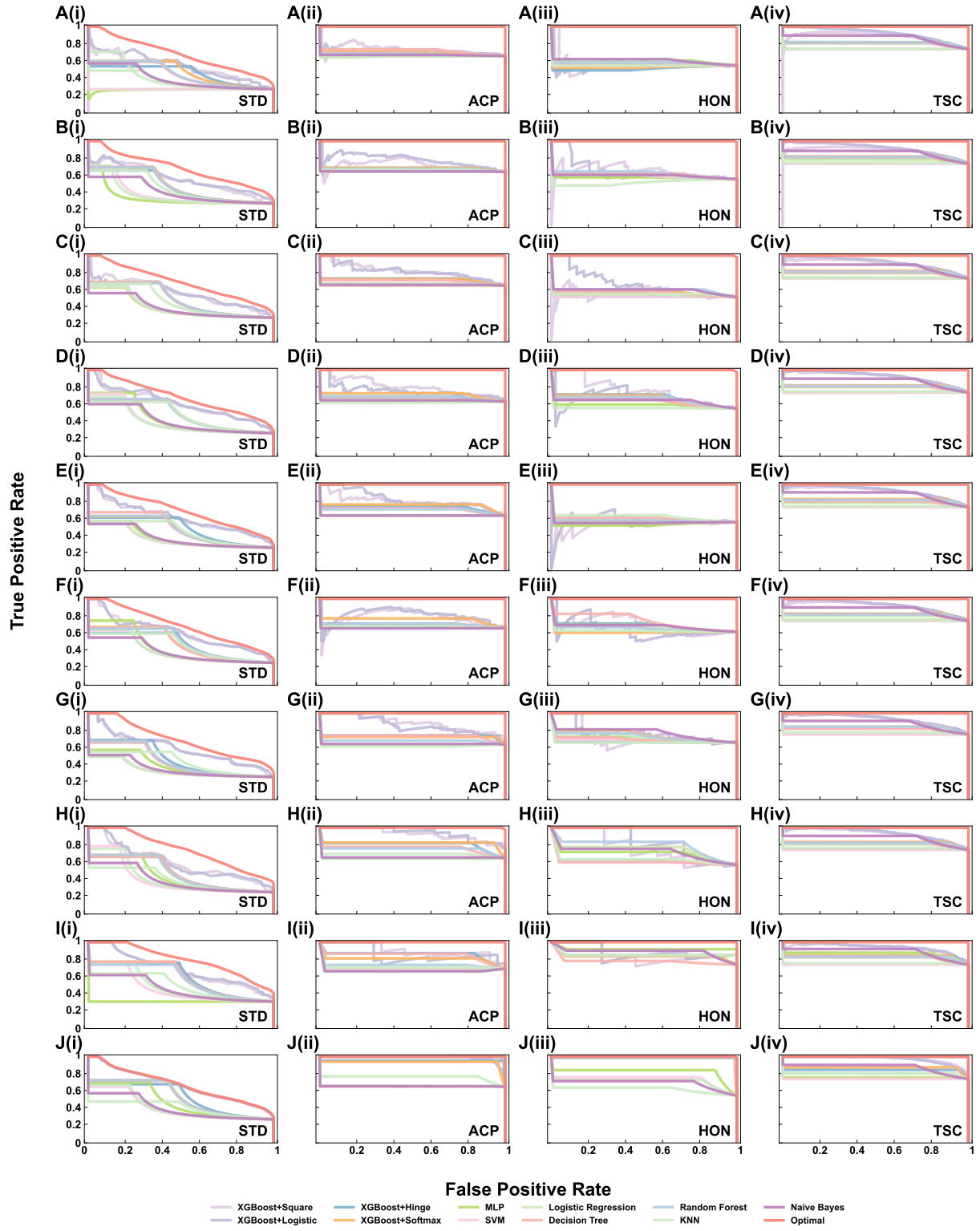


Figure S28. Exact upper bound of AP and corresponding optimal PR curves on 4 additional real-world datasets (STD, ACP, HON and TSC) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

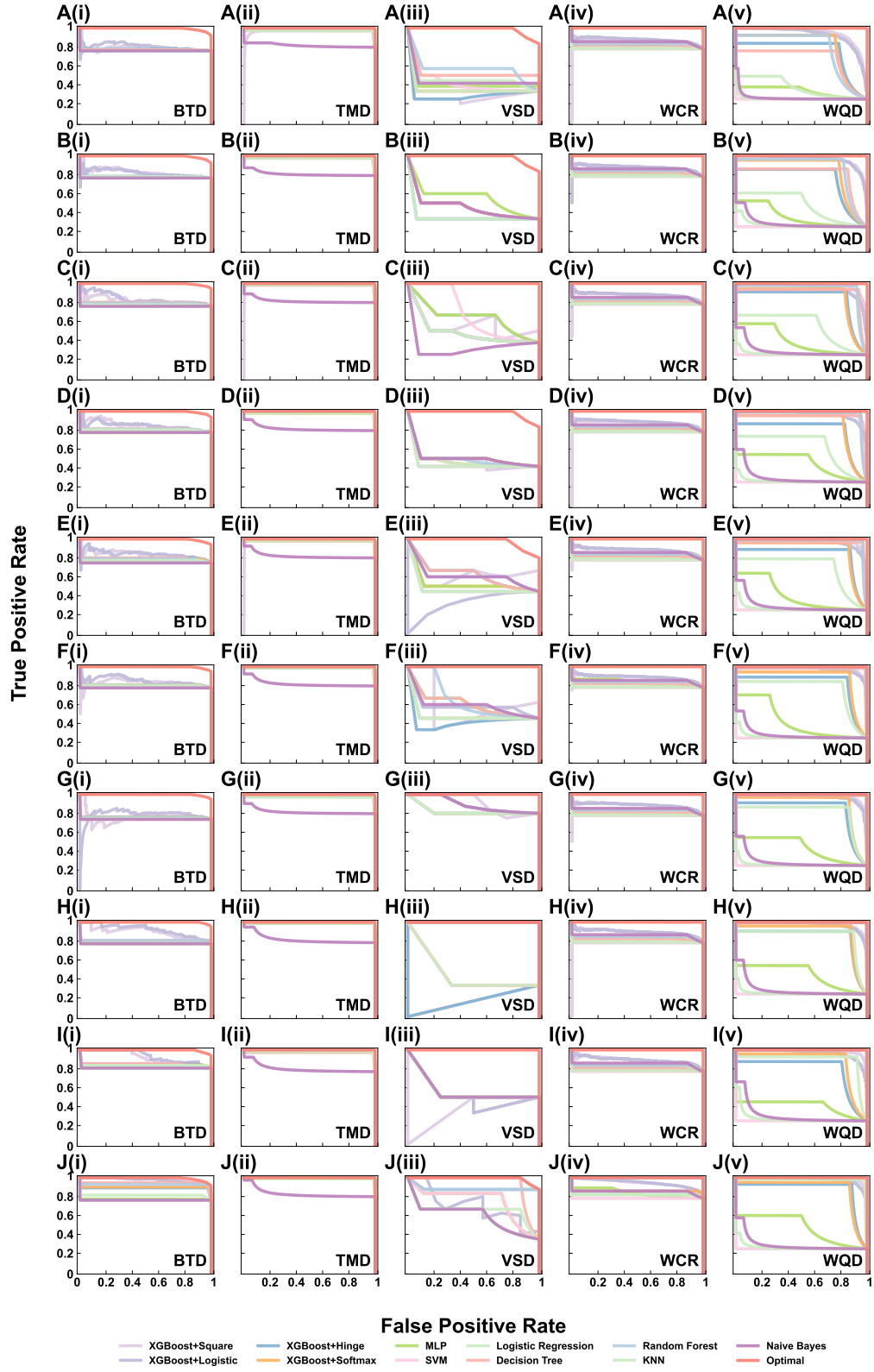


Figure S29. Exact upper bound of AP and corresponding optimal PR curves on 5 additional real-world datasets (BT, TD, VS, WC and WQ) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I) and $|S_{train}|/|S| = 1$ (J). The binary classifiers we used in this experiment include XGBoost, MLP, SVM, Logistic Regression, Decision Tree, Random Forest, KNN and Naive Bayes. Red curves represent the theoretical optimal ROC curves.

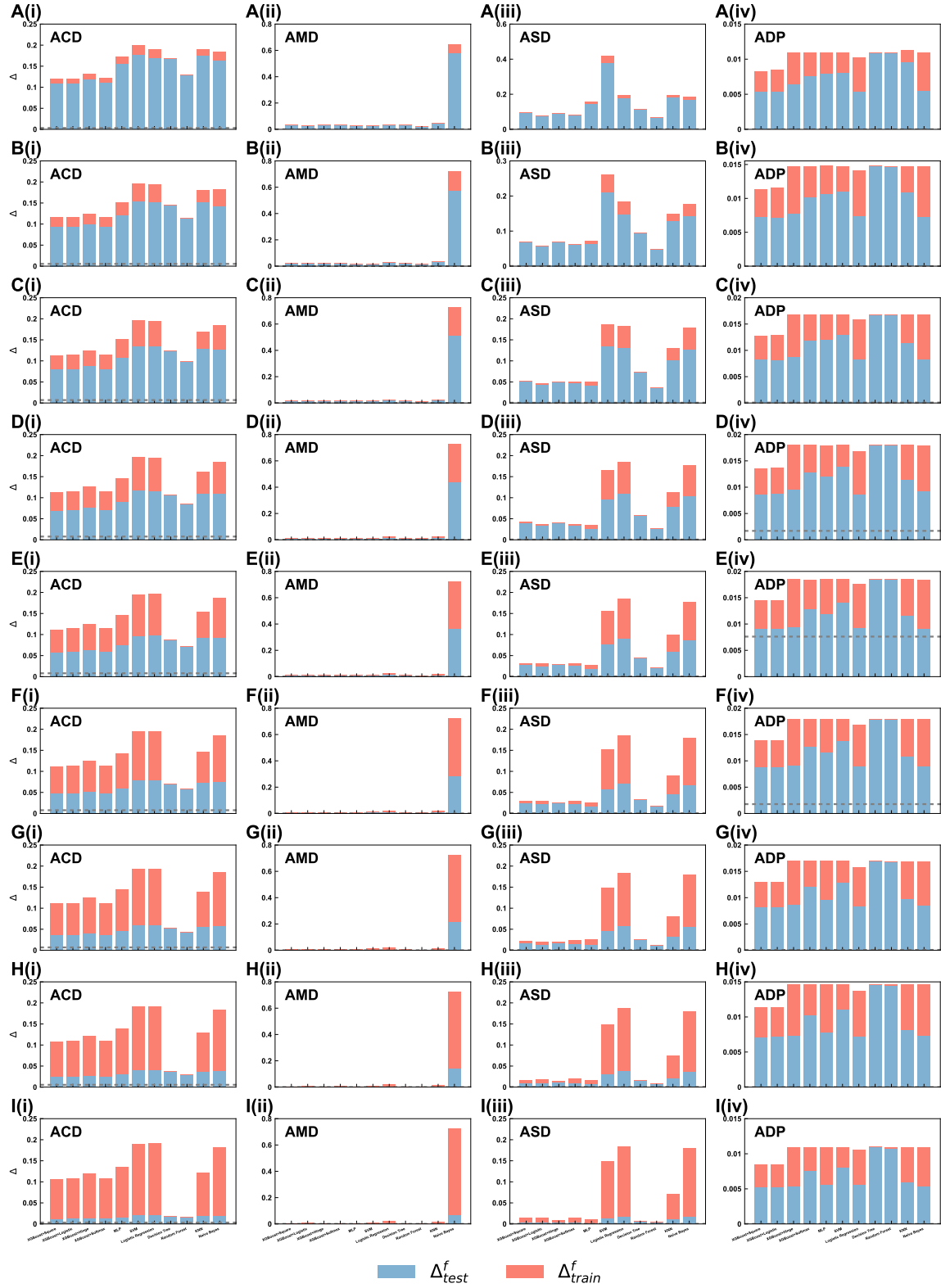


Figure S30. The error dynamics on 4 additional datasets (ACD, AMD, ASD and ADP) in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.



Figure S31. The error dynamics on 4 additional datasets BP, CPDT, COP and CPDN in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.

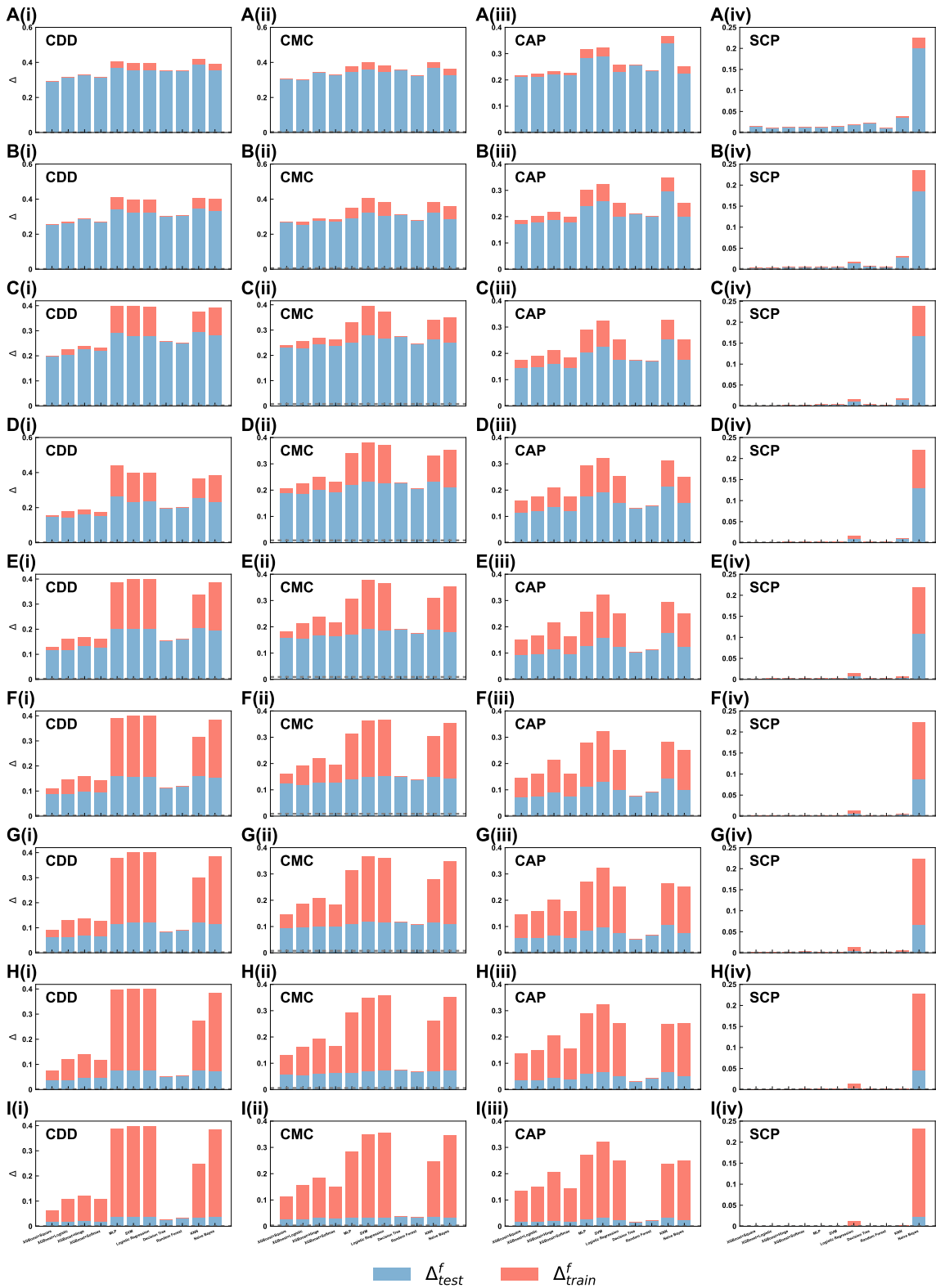


Figure S32. The error dynamics on 4 additional datasets (CDD, CMC, CAP and SCP) in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.

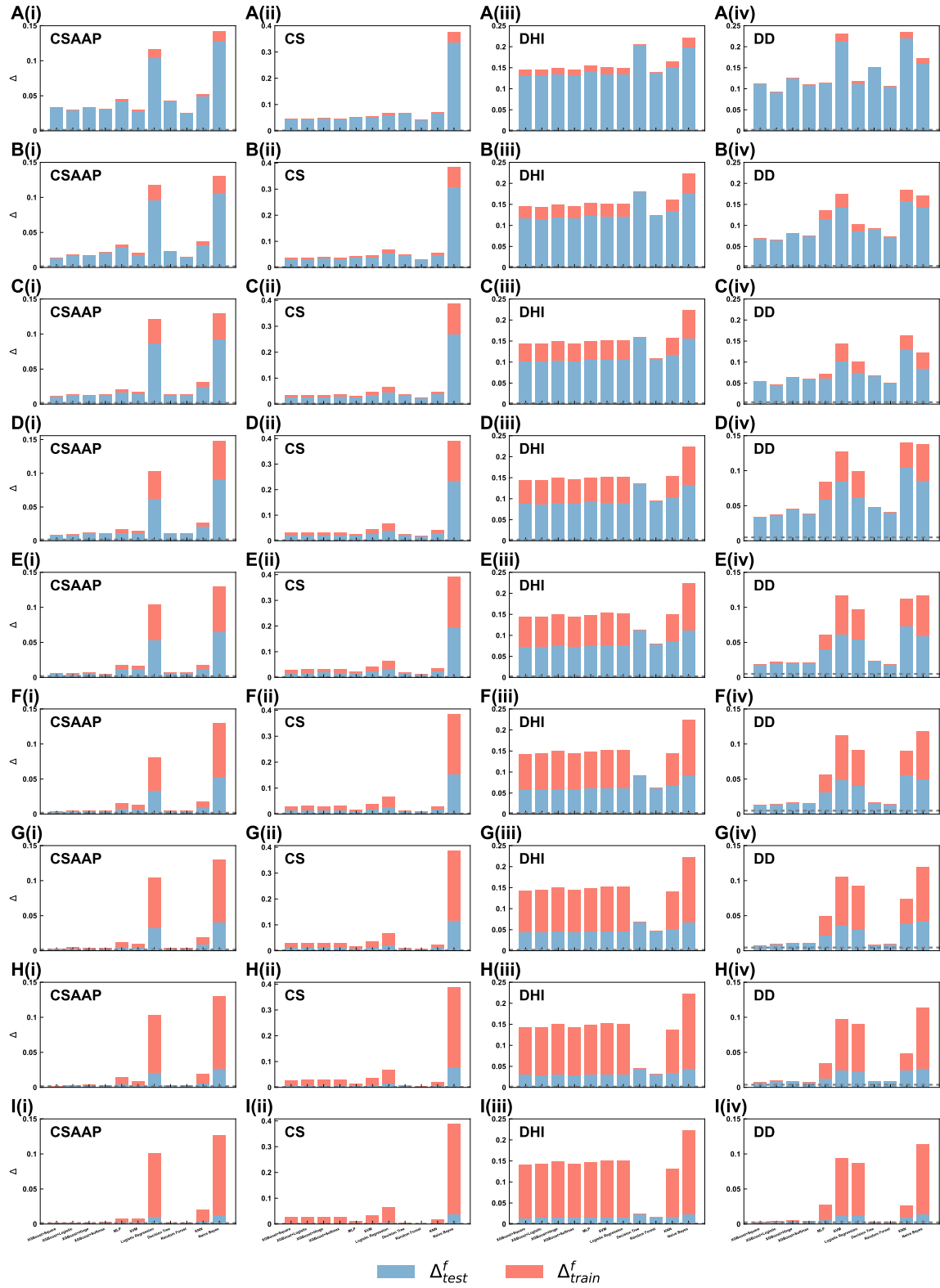


Figure S33. The error dynamics on 4 additional datasets (CSAAP, CS, DHI and DD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.



Figure S34. The error dynamics on 4 additional datasets (ED, FDBD, GOT and HD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.



Figure S35. The error dynamics on 4 additional datasets (HDN, HDHID, IM and LOAN) in training (Δ^f_{train}) and test sets (Δ^f_{test}) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.

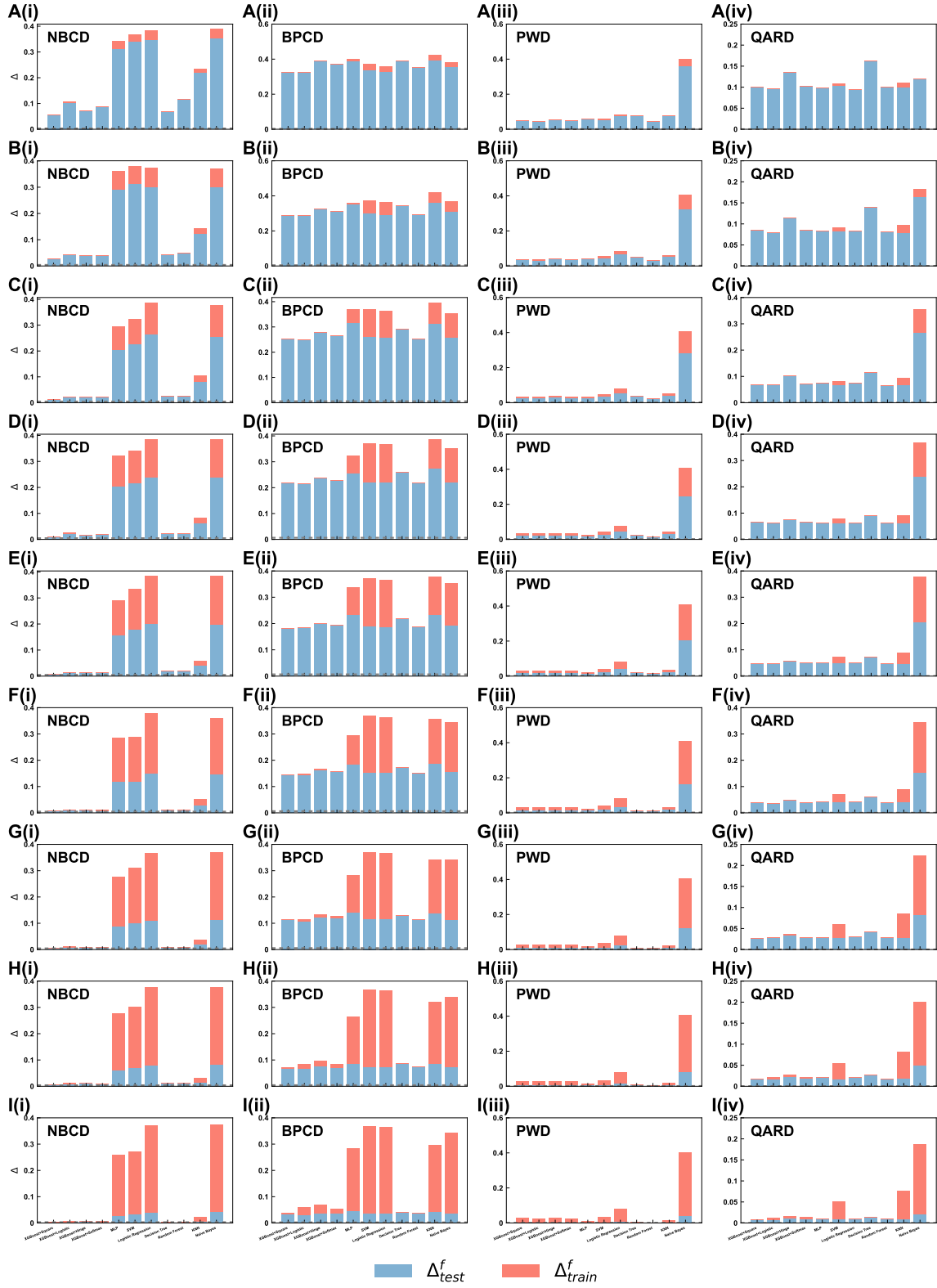


Figure S36. The error dynamics on 4 additional datasets (NBCD, BPCD, PWD and QARD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{\text{train}}|/|S| = 0.1$ (A), $|S_{\text{train}}|/|S| = 0.2$ (B), $|S_{\text{train}}|/|S| = 0.3$ (C), $|S_{\text{train}}|/|S| = 0.4$ (D), $|S_{\text{train}}|/|S| = 0.5$ (E), $|S_{\text{train}}|/|S| = 0.6$ (F), $|S_{\text{train}}|/|S| = 0.7$ (G), $|S_{\text{train}}|/|S| = 0.8$ (H), $|S_{\text{train}}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.



Figure S37. The error dynamics on 4 additional datasets (STD, ACP, HON and TSC) in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.

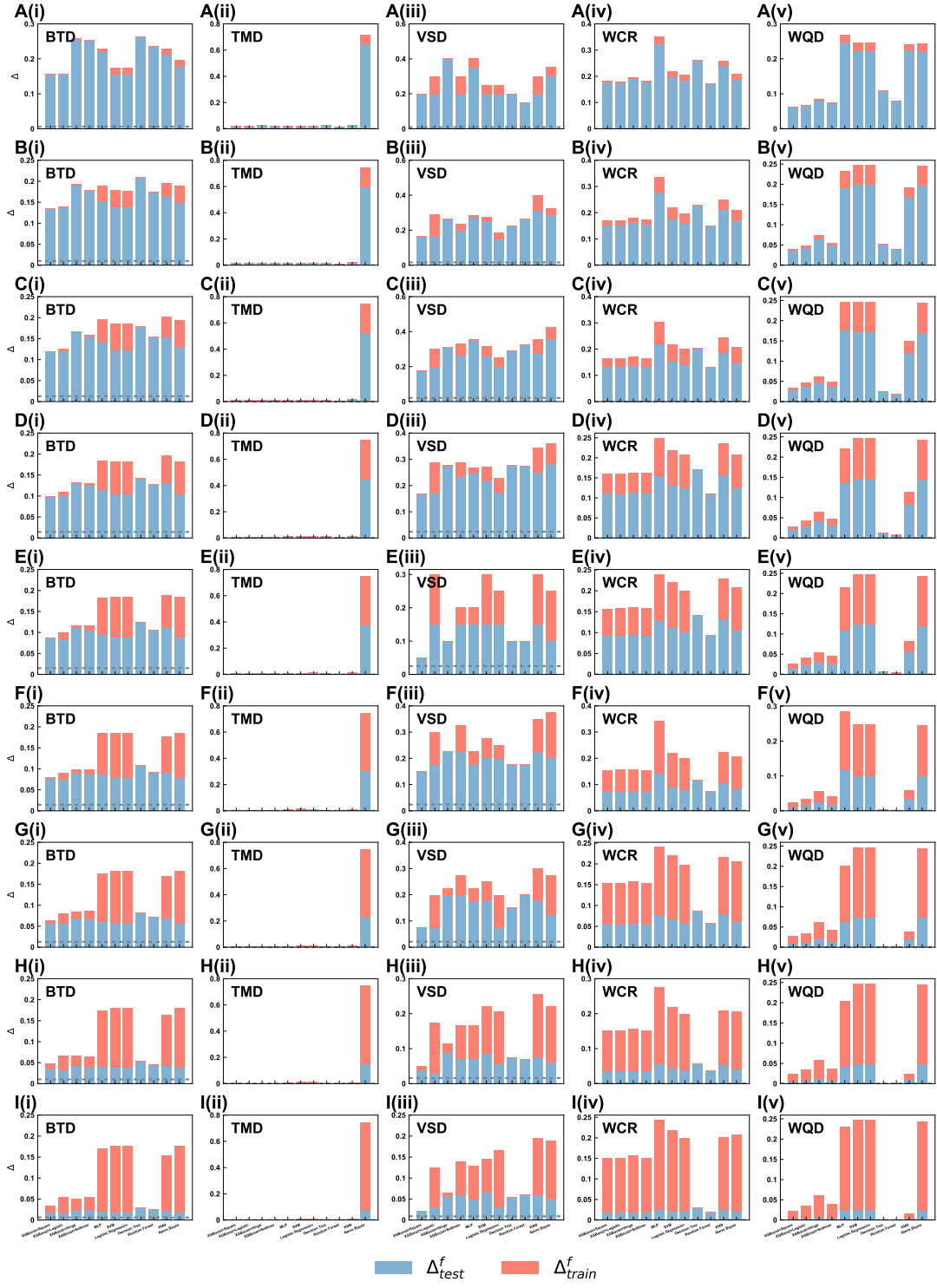


Figure S38. The error dynamics on 5 additional datasets (BT, TMD, VSD, WCR and WQD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Dash line represents the expected error of optimal classifier based on Eq. 94.

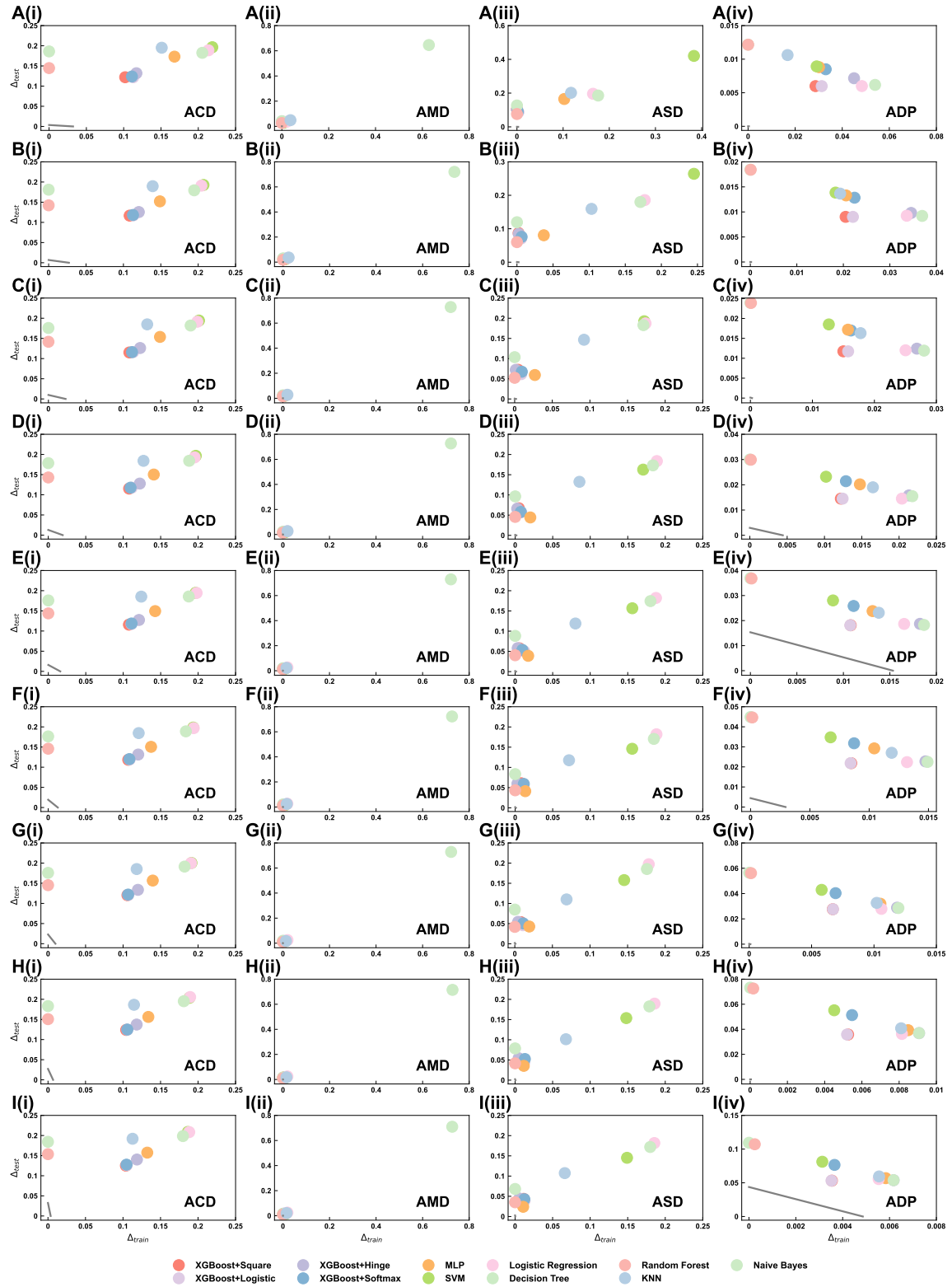


Figure S39. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (ACD, AMD, ASD and ADP) when $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.1$ (A), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.2$ (B), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.3$ (C), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.4$ (D), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.5$ (E), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.6$ (F), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.7$ (G), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.8$ (H), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

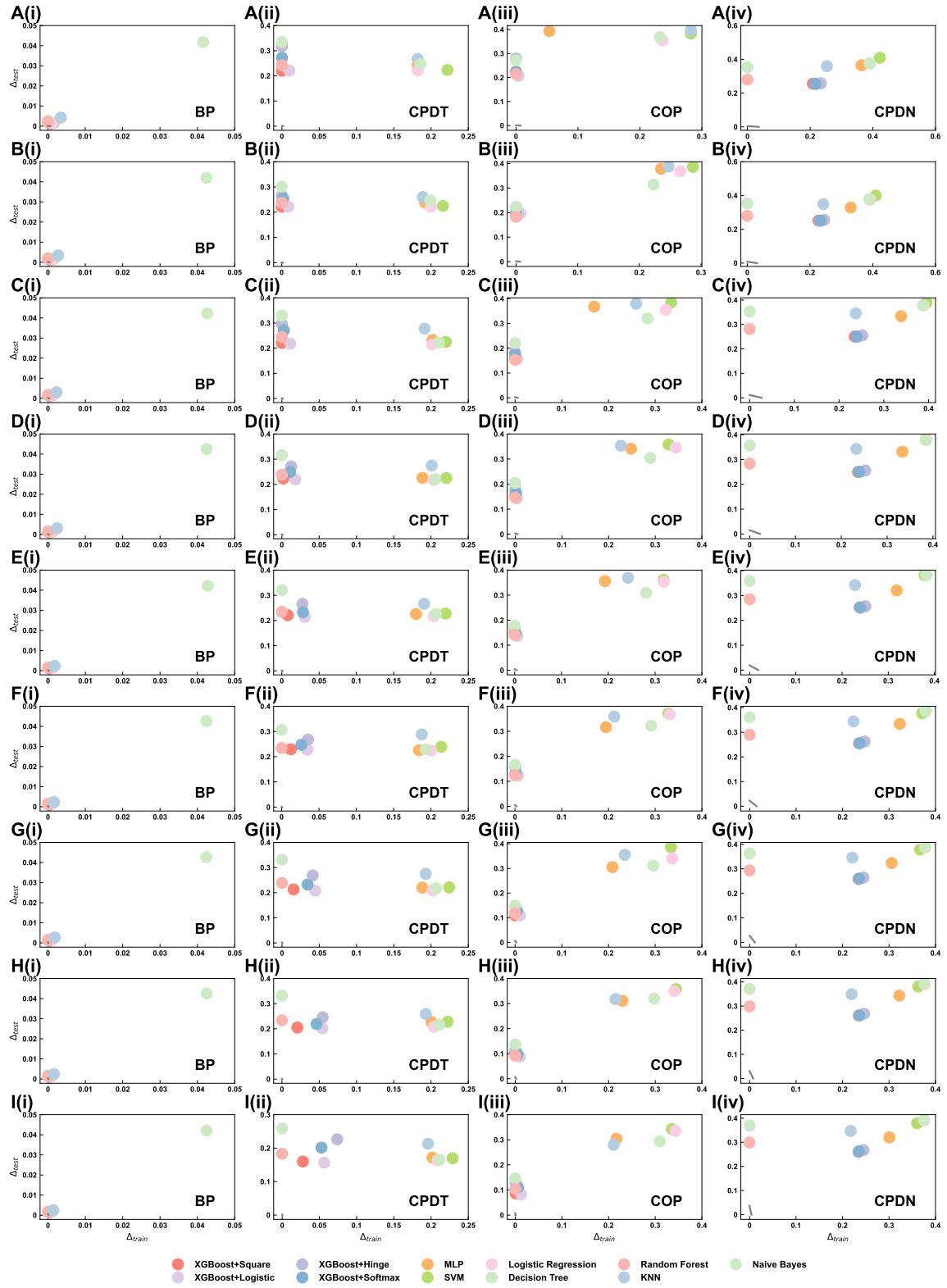


Figure S40. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (BP, CPDT, COP and CPDN) when $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.1$ (A), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.2$ (B), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.3$ (C), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.4$ (D), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.5$ (E), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.6$ (F), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.7$ (G), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.8$ (H), $|\mathcal{S}_{train}|/|\mathcal{S}| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

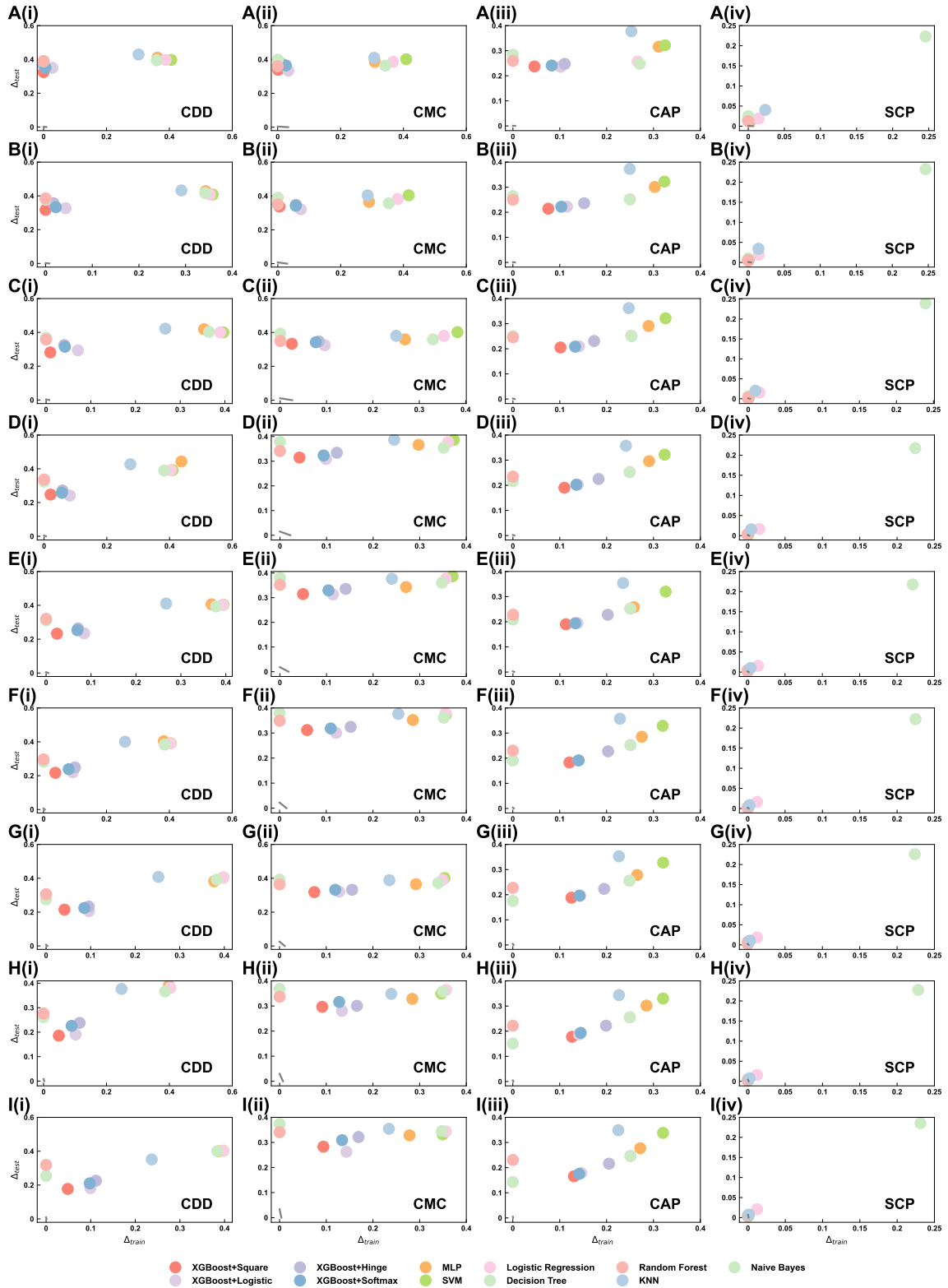


Figure S41. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (CDD, CMC, CAP and SCP) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

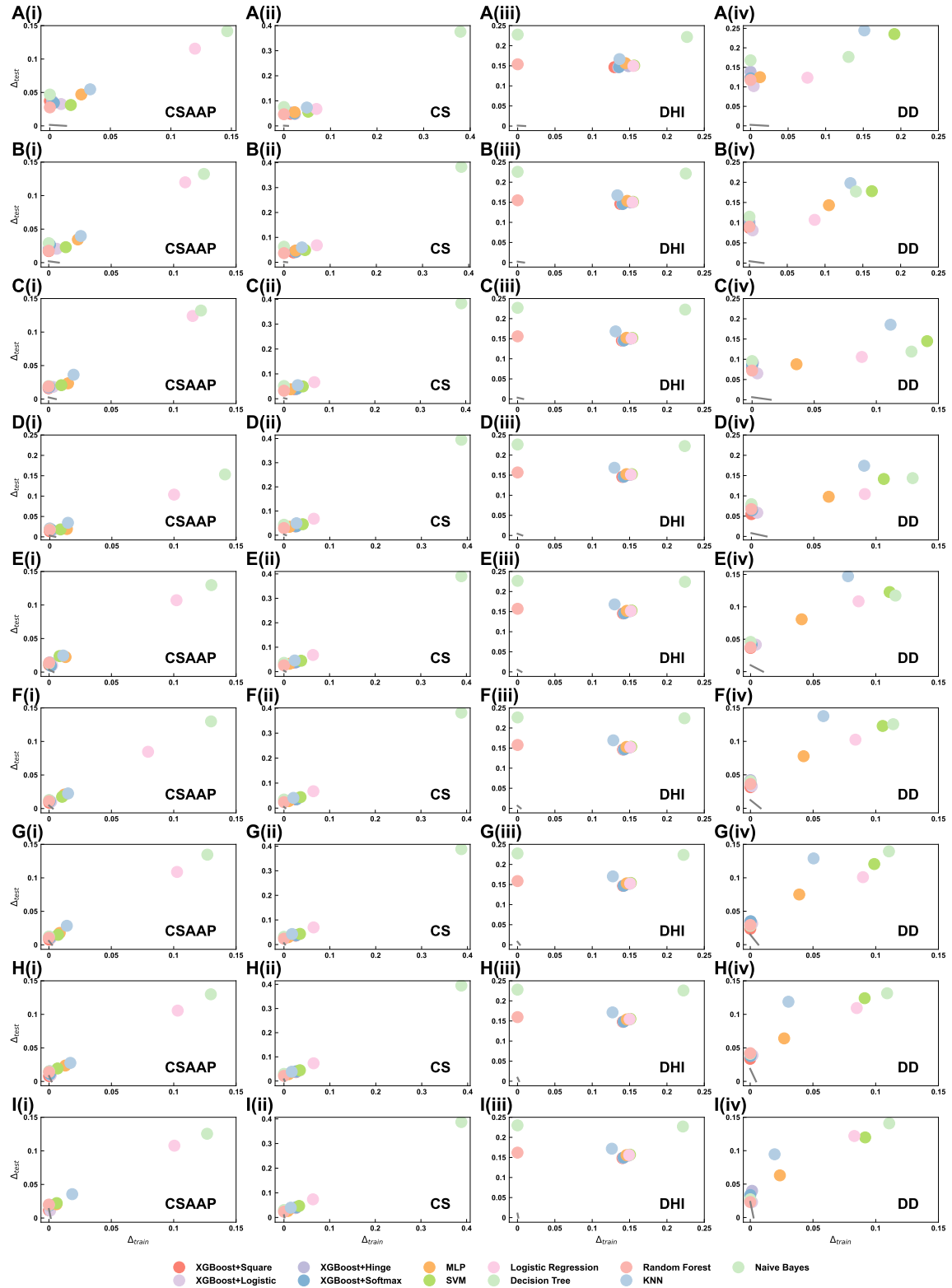


Figure S42. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (CSAAP, CS, DHI and DD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.



Figure S43. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (ED, FDBD, GOT and HD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

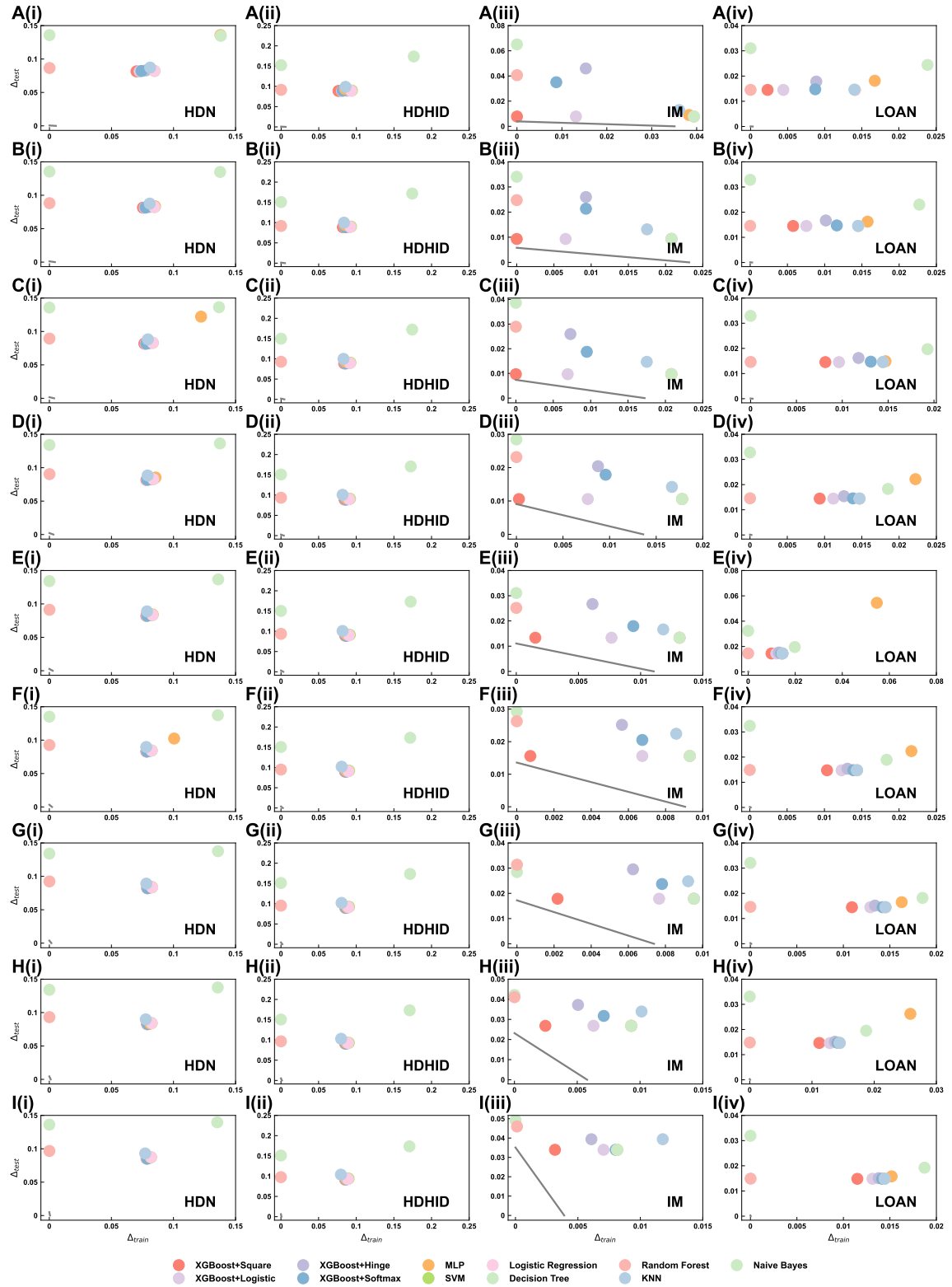


Figure S44. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (HDN, HDHID, IM and LOAN) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

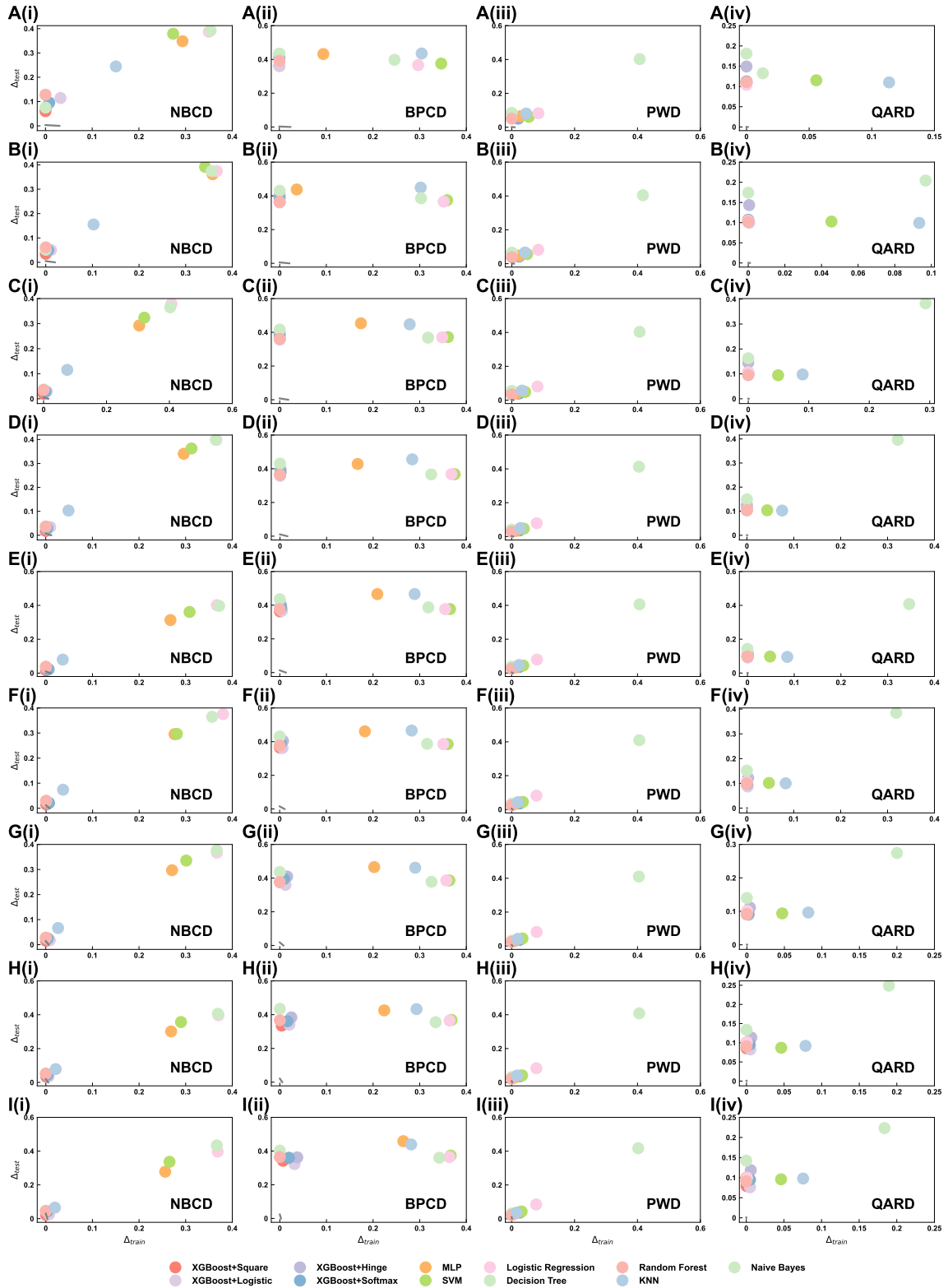


Figure S45. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (NBCD, BPCD, PWD and QARD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

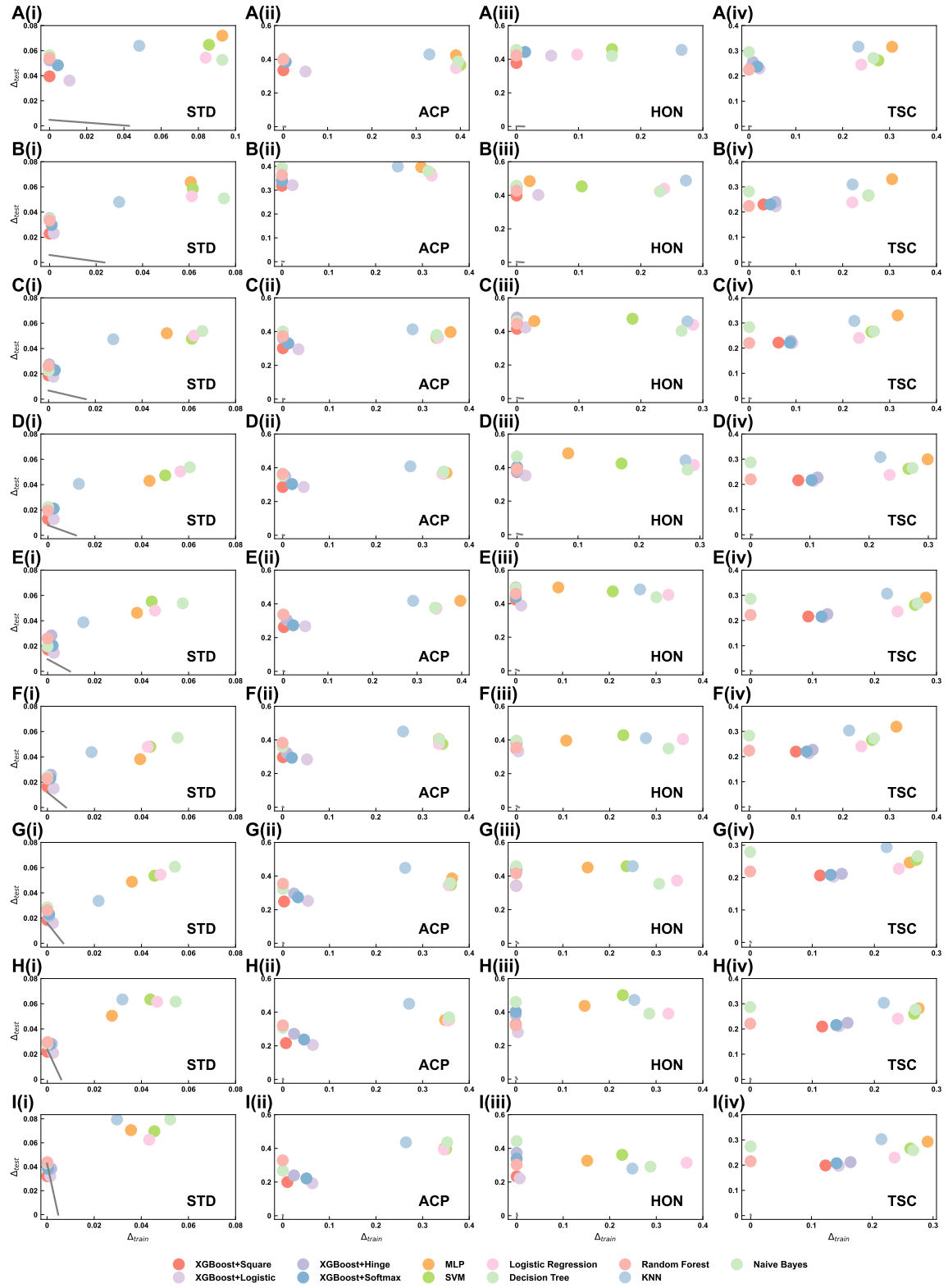


Figure S46. The correlation between Δ_{train}^f and Δ_{test}^f on 4 additional datasets (STD, ACP, HON and TSC) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

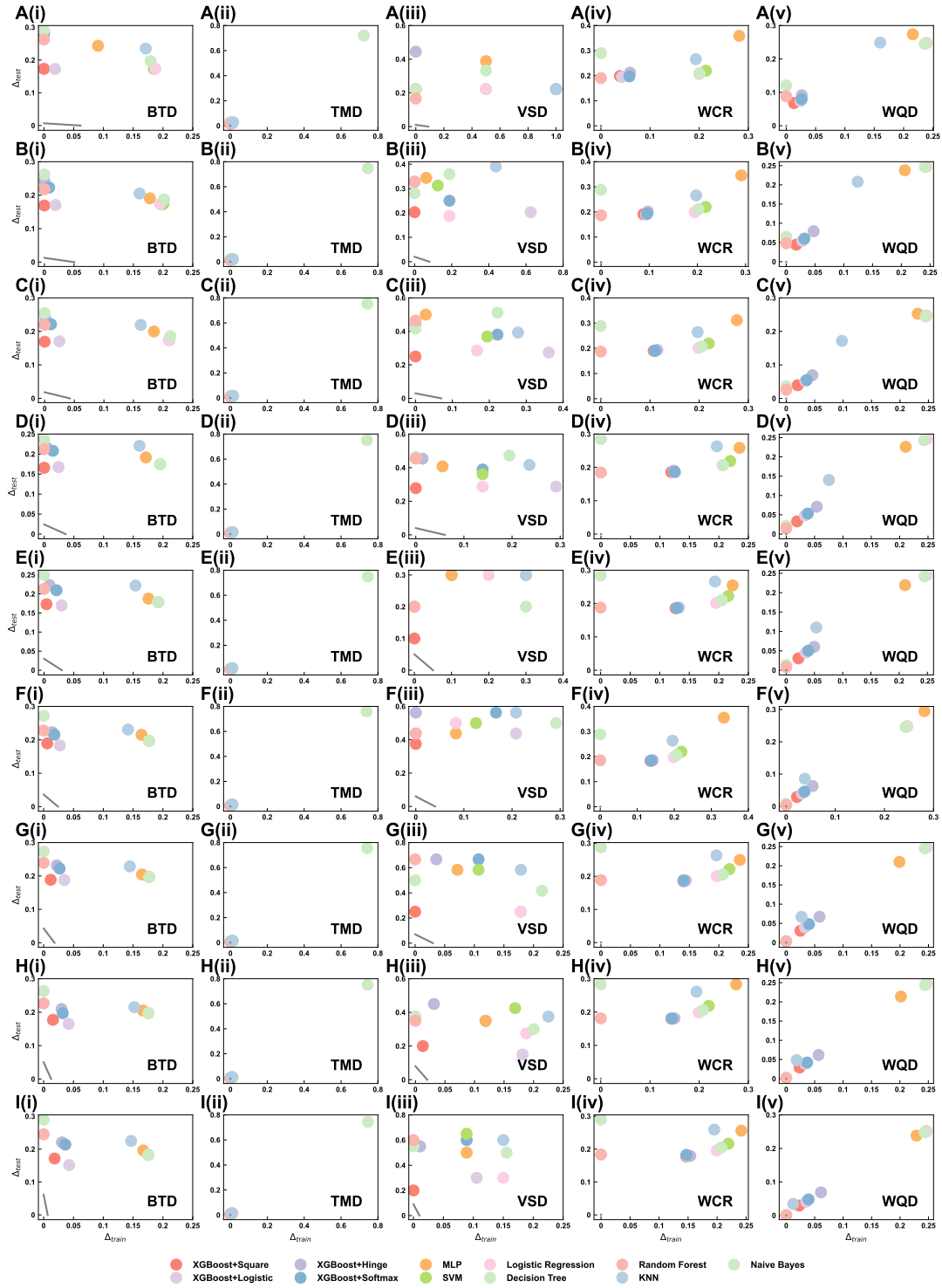


Figure S47. The correlation between Δ_{train}^f and Δ_{test}^f on 5 additional datasets (BTD, TMD, VSD, WCR and WQD) when $|S_{train}|/|S| = 0.1$ (A), $|S_{train}|/|S| = 0.2$ (B), $|S_{train}|/|S| = 0.3$ (C), $|S_{train}|/|S| = 0.4$ (D), $|S_{train}|/|S| = 0.5$ (E), $|S_{train}|/|S| = 0.6$ (F), $|S_{train}|/|S| = 0.7$ (G), $|S_{train}|/|S| = 0.8$ (H), $|S_{train}|/|S| = 0.9$ (I). Gray line represents the expected error of optimal classifier based on Eq. 94.

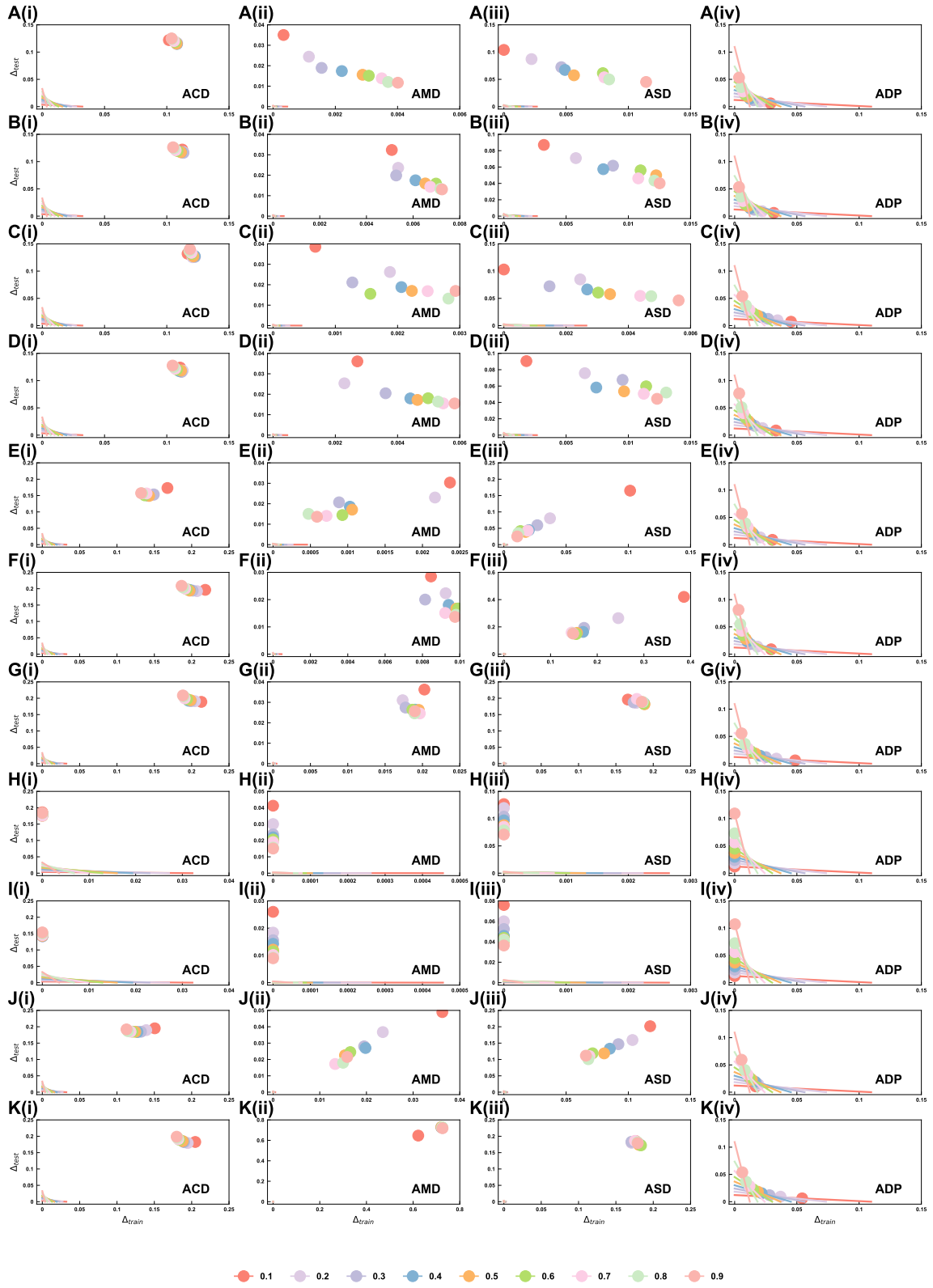


Figure S48. The loss errors on four additional datasets (ACD, AMD, ASD and ADP) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

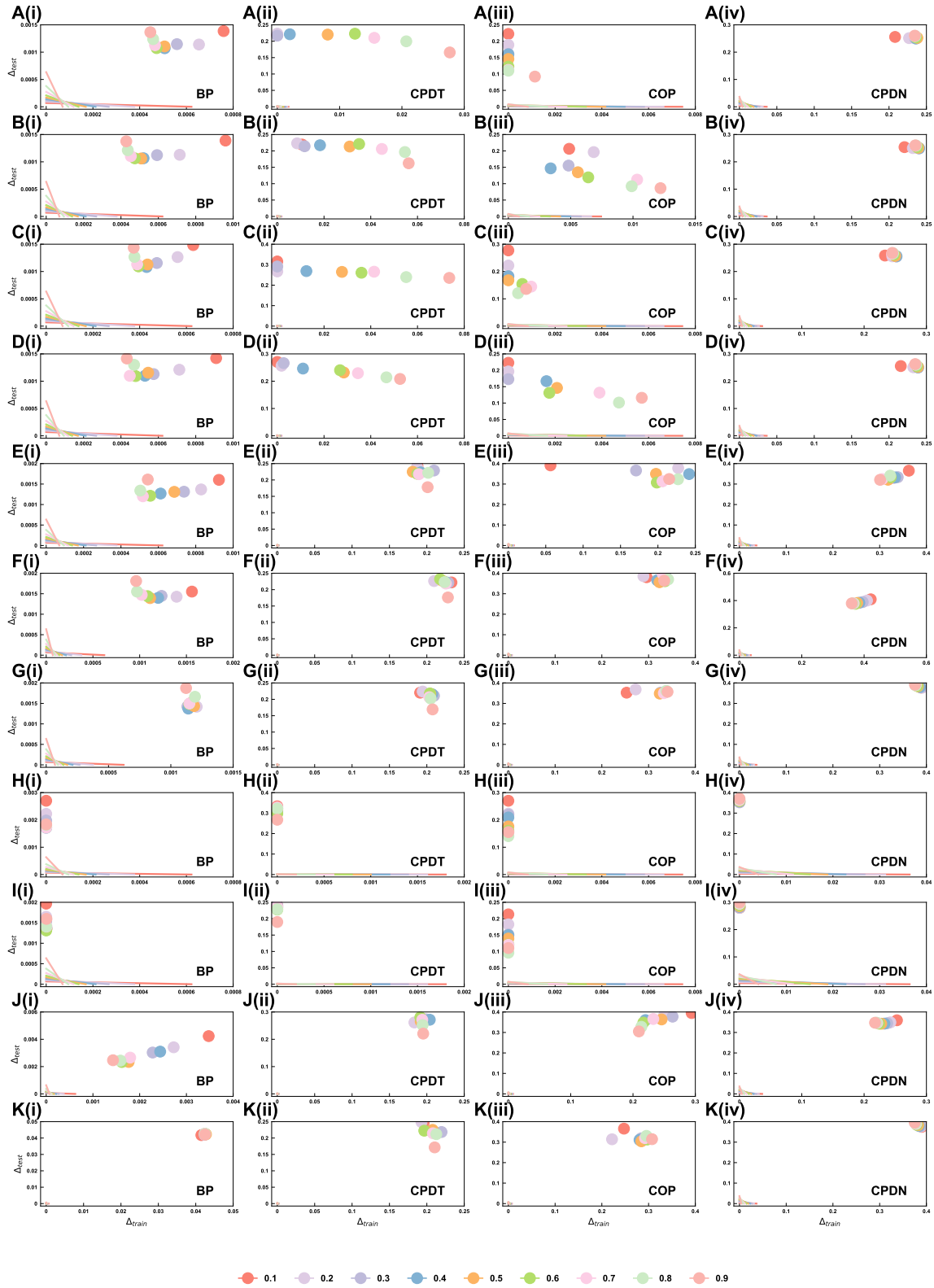


Figure S49. The loss errors on four additional datasets (BP, CPDT, COP and CPDN) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

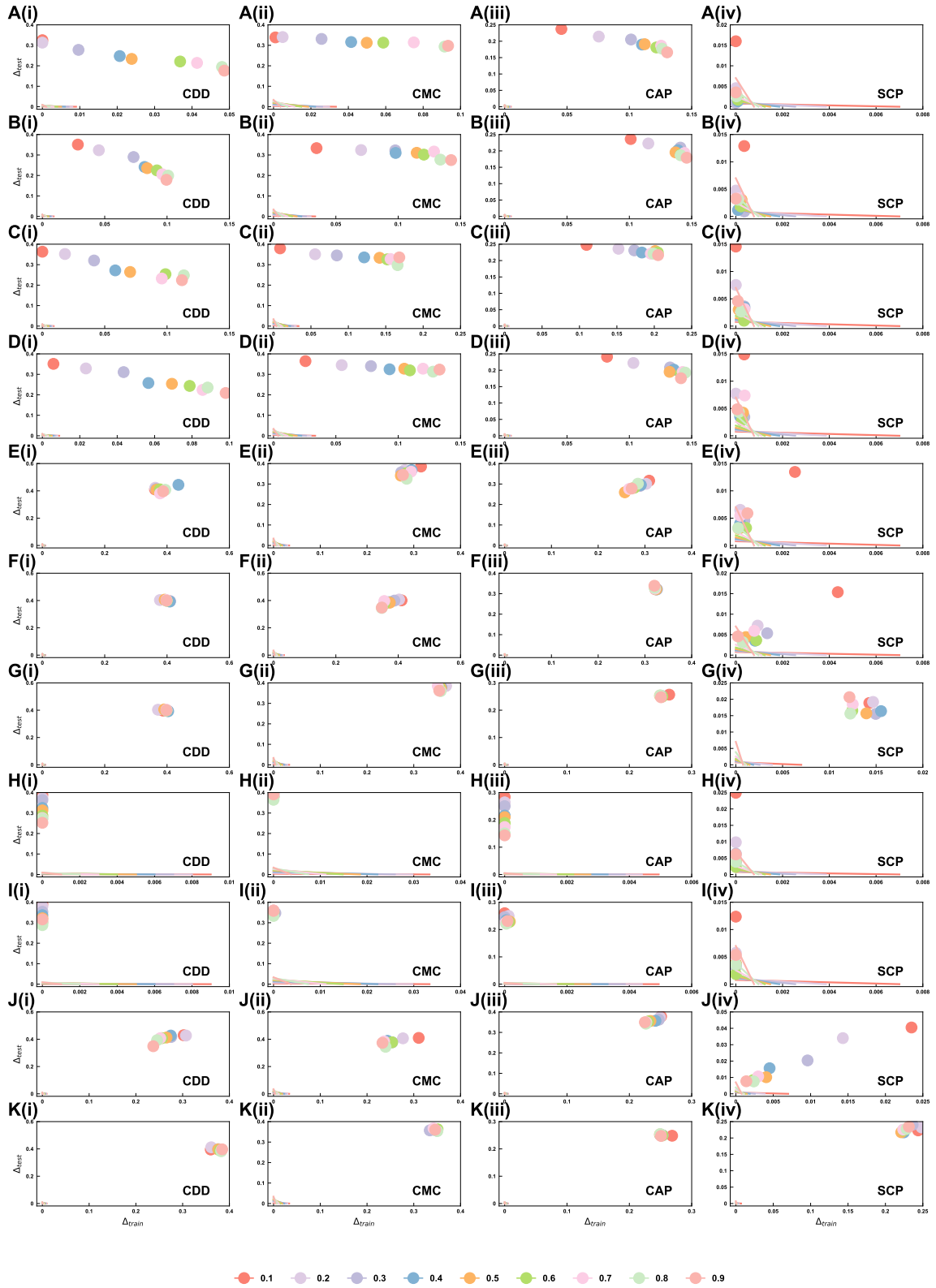


Figure S50. The loss errors on four additional datasets (CDD, CMC, CAP and SCP) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

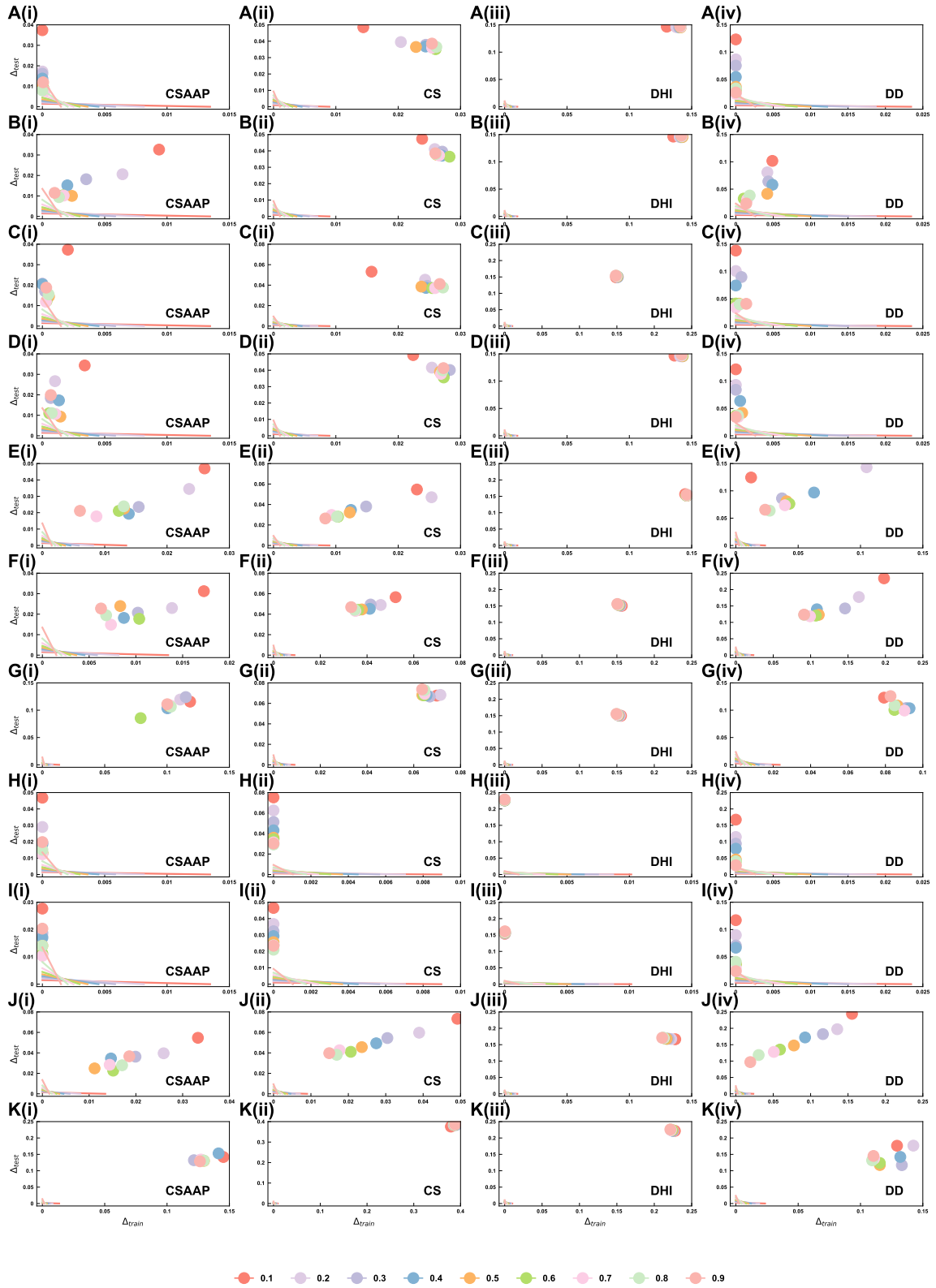


Figure S51. The loss errors on four additional datasets (CSAAP, CS, DHI and DD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

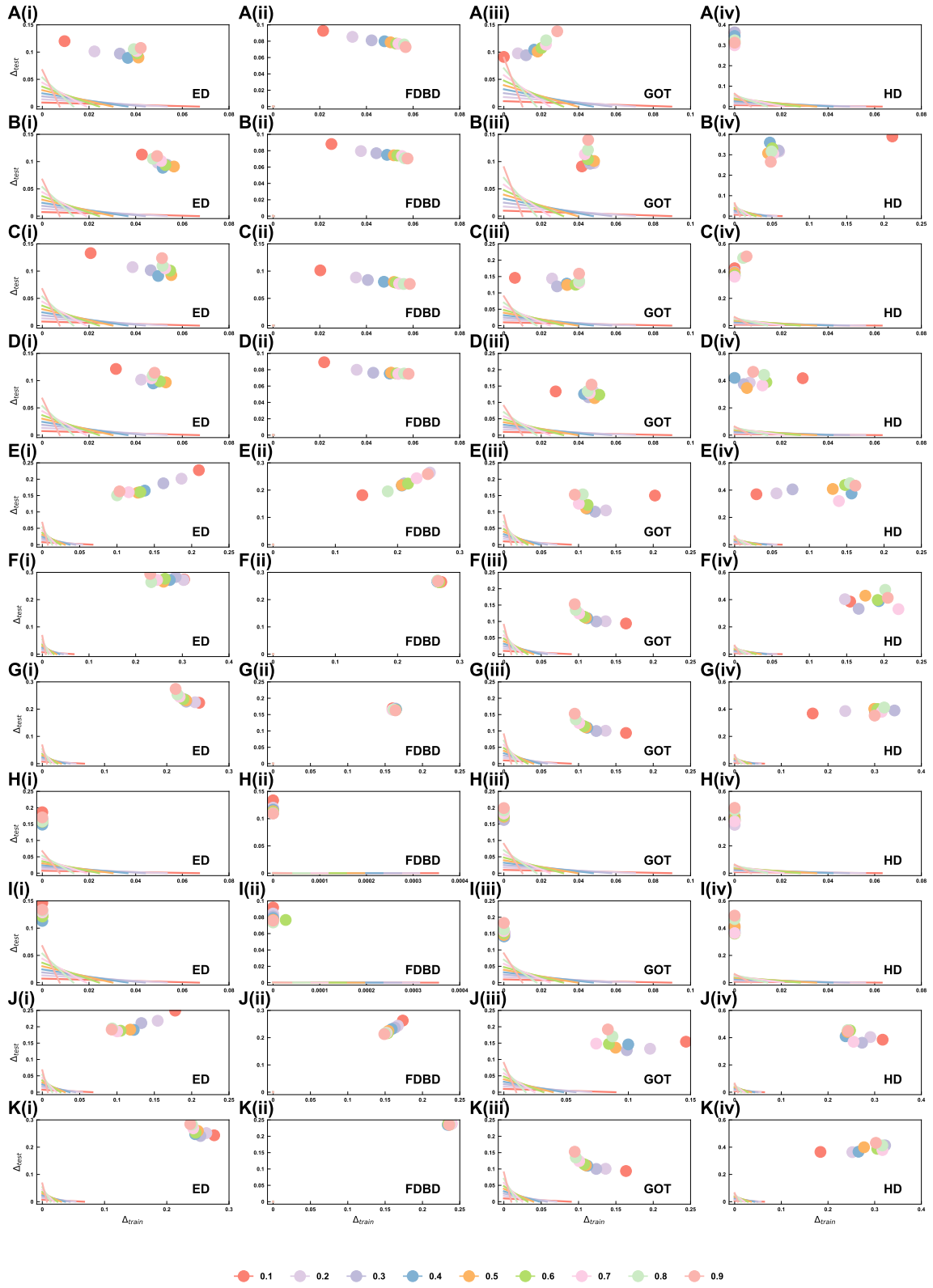


Figure S52. The loss errors on four additional datasets (ED, FDBD, GOT and HD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

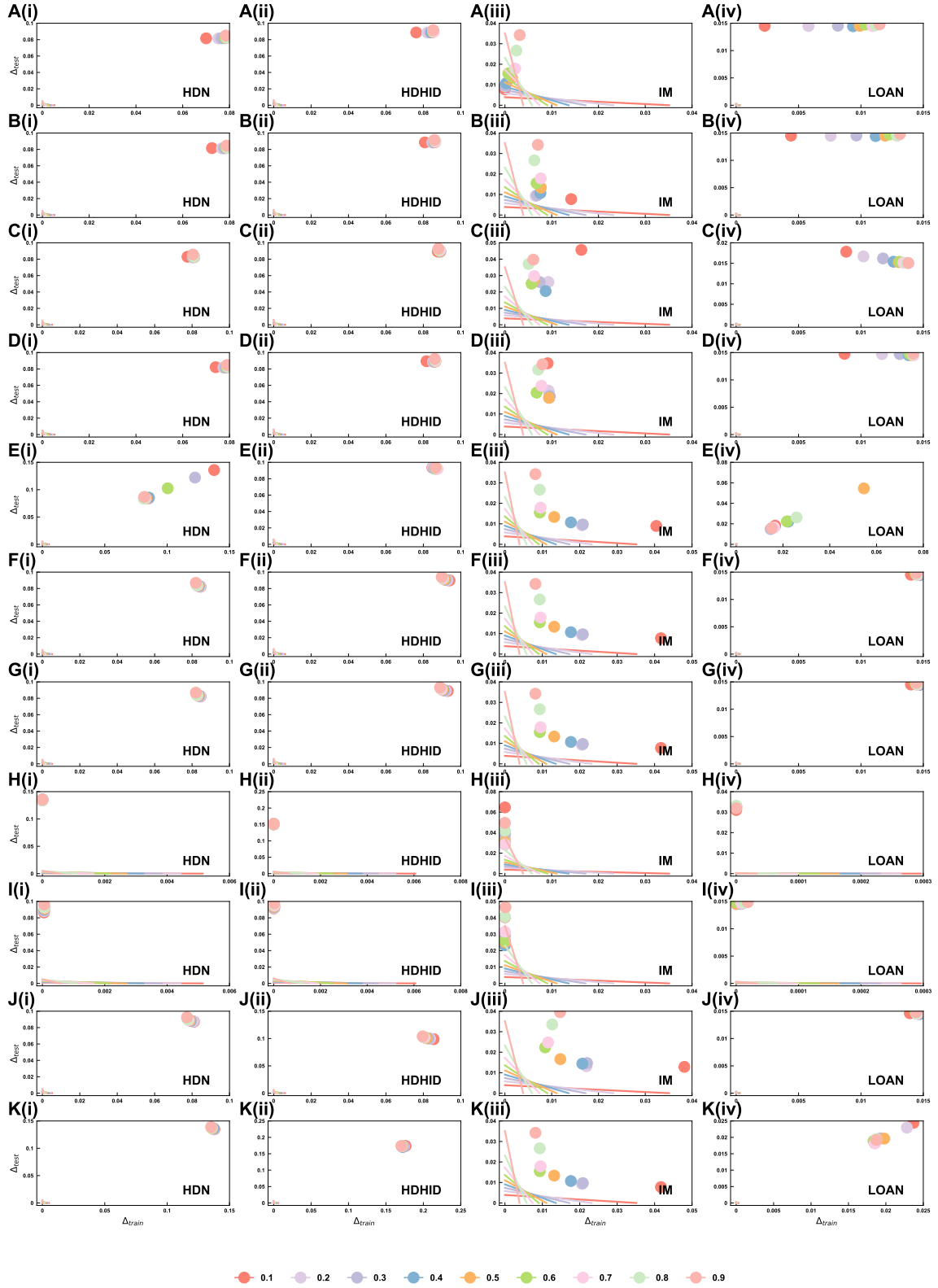


Figure S53. The loss errors on four additional datasets (HDN, HDHID, IM and LOAN) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

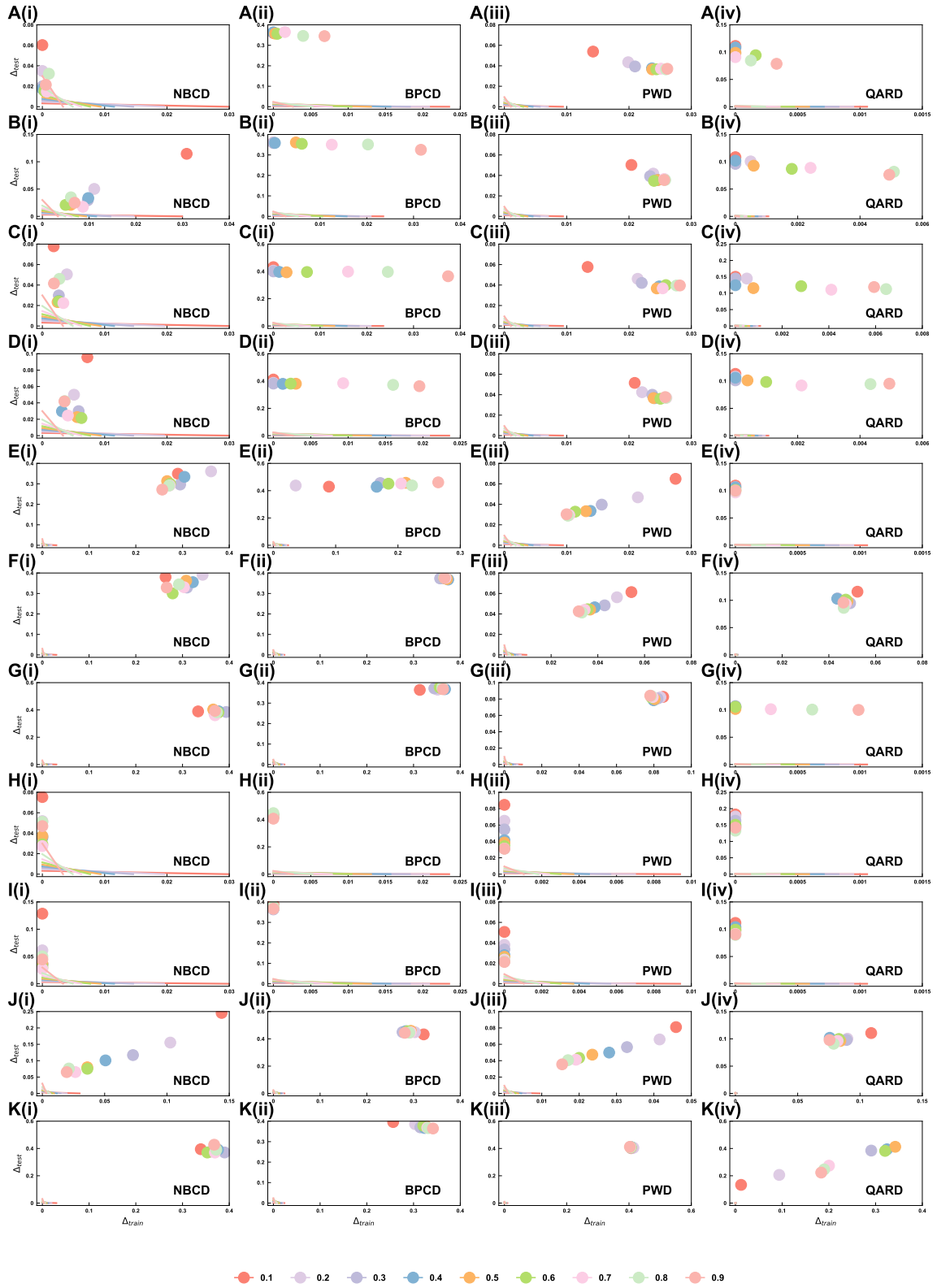


Figure S54. The loss errors on four additional datasets (NBCD, BPCD, PWD and QARD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

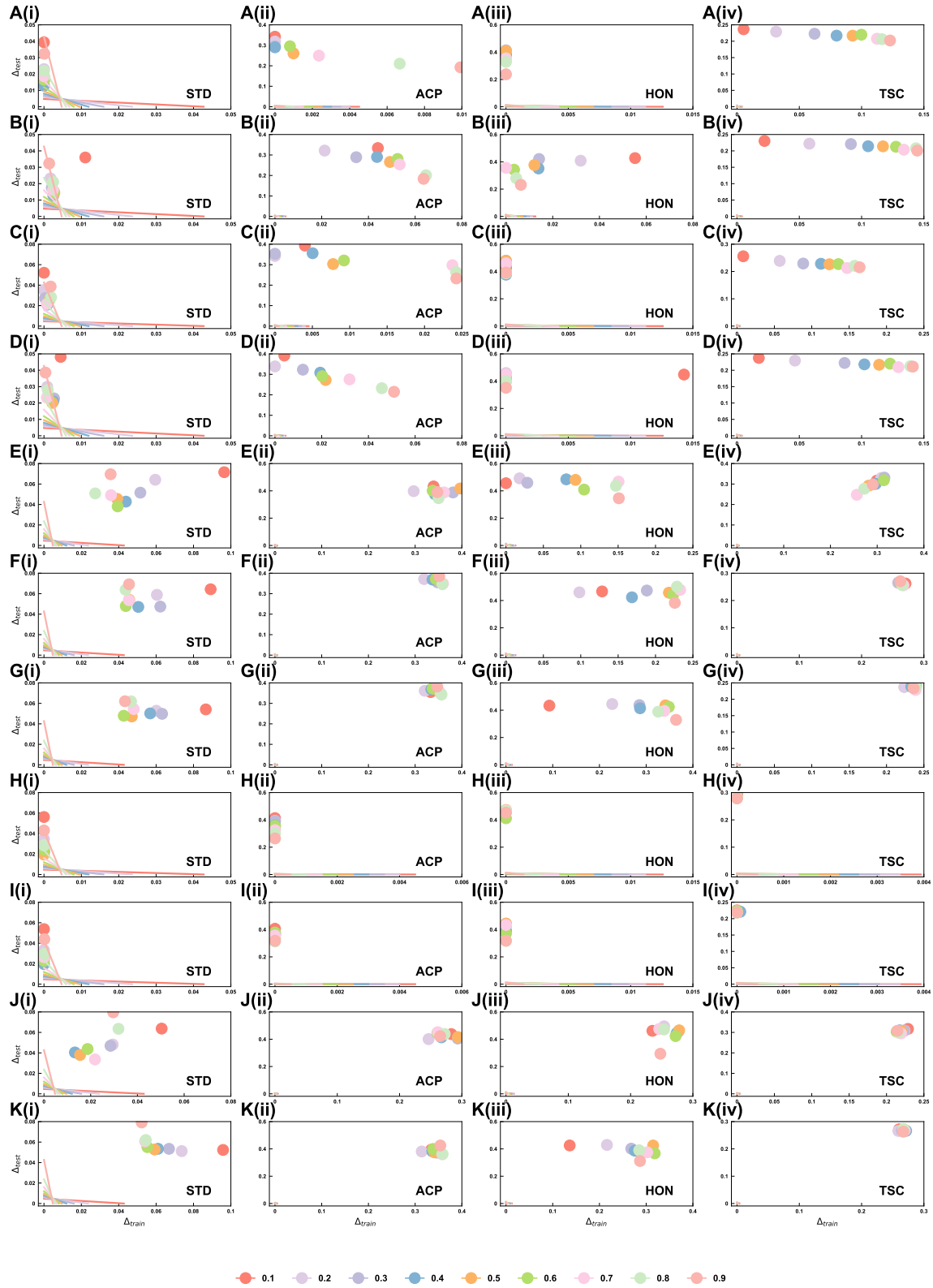


Figure S55. The loss errors on four additional datasets (STD, ACP, HON and TSC) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

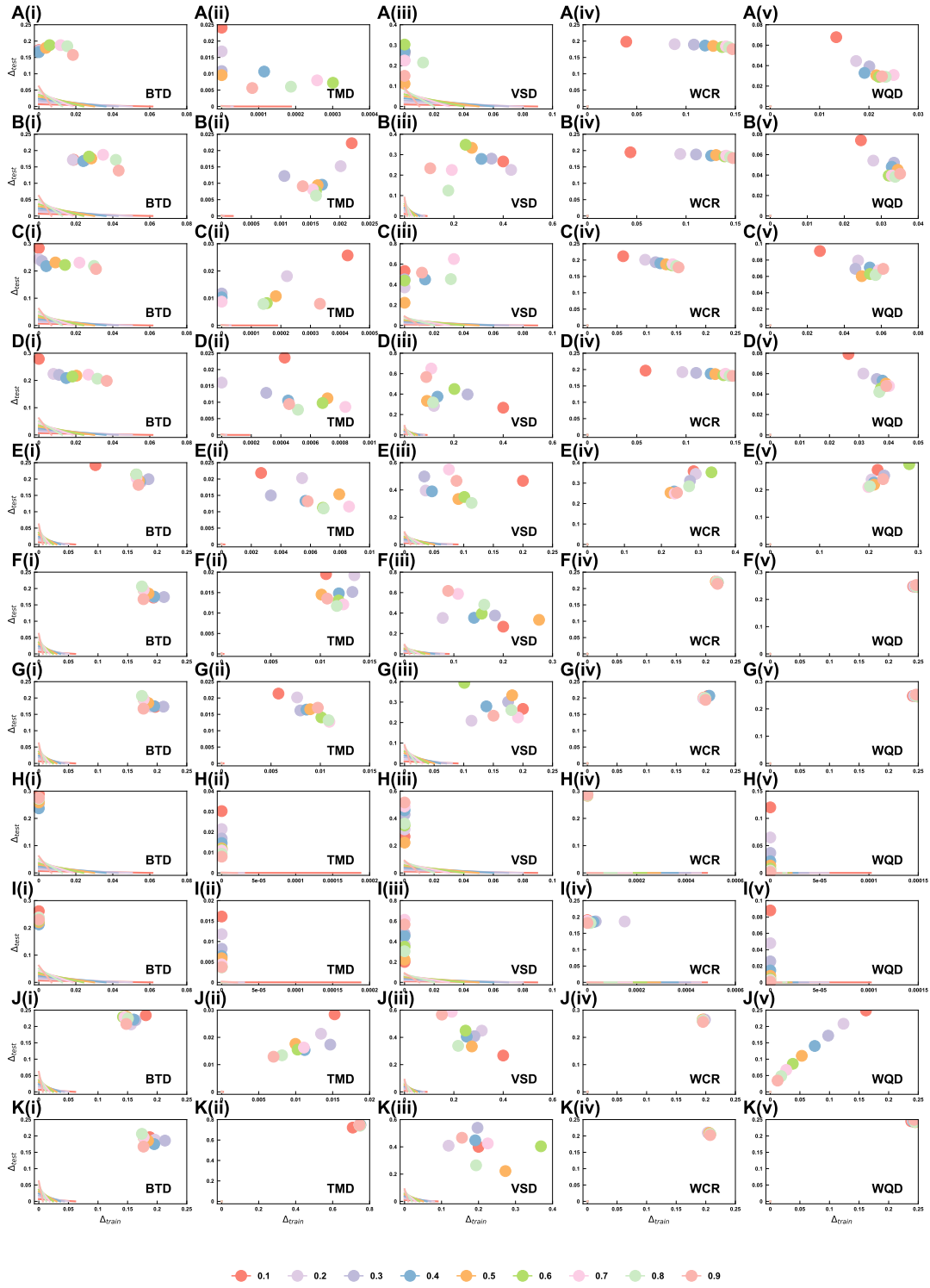


Figure S56. The loss errors on five additional datasets (BT, TMD, VSD, WCR and WQD) in training (Δ_{train}^f) and test sets (Δ_{test}^f) of different binary classifiers, including XGBoost with four classical objectives (A-D), MLP (E), SVM (F), Logistic Regression (G), Decision Tree (H), Random Forest (I), KNN (J). Colorful dots and lines represent different $|S_{train}|/|S|$ ranging from 0.1 to 0.9.

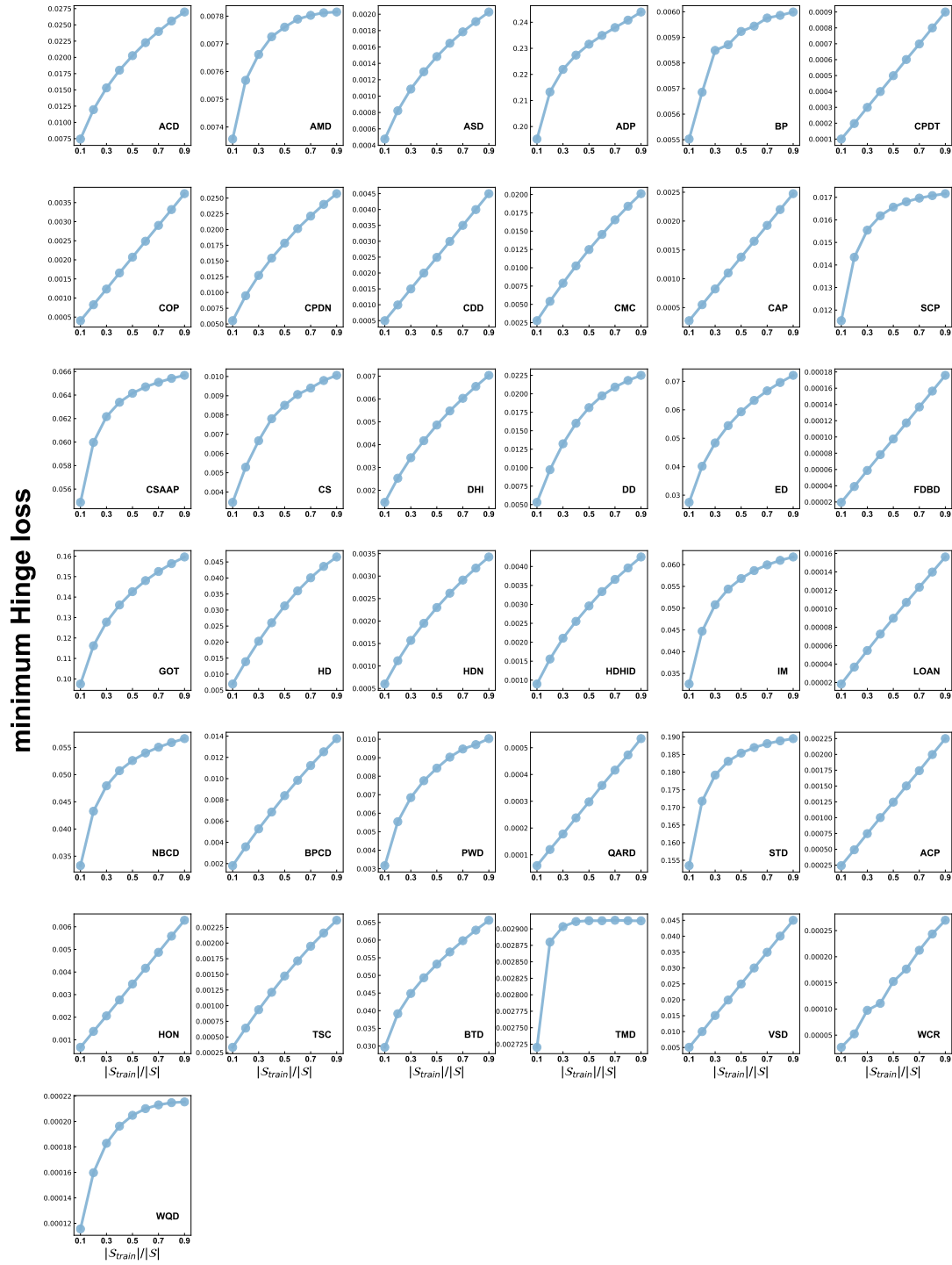


Figure S57. Minimum Hinge loss for 37 additional datasets under different random divisions In these panels, dots correspond to numerical results derived from the data divisions, while lines represent the theoretical predictions adjusted for the respective division ratios (see Eqs. 54 and 56).

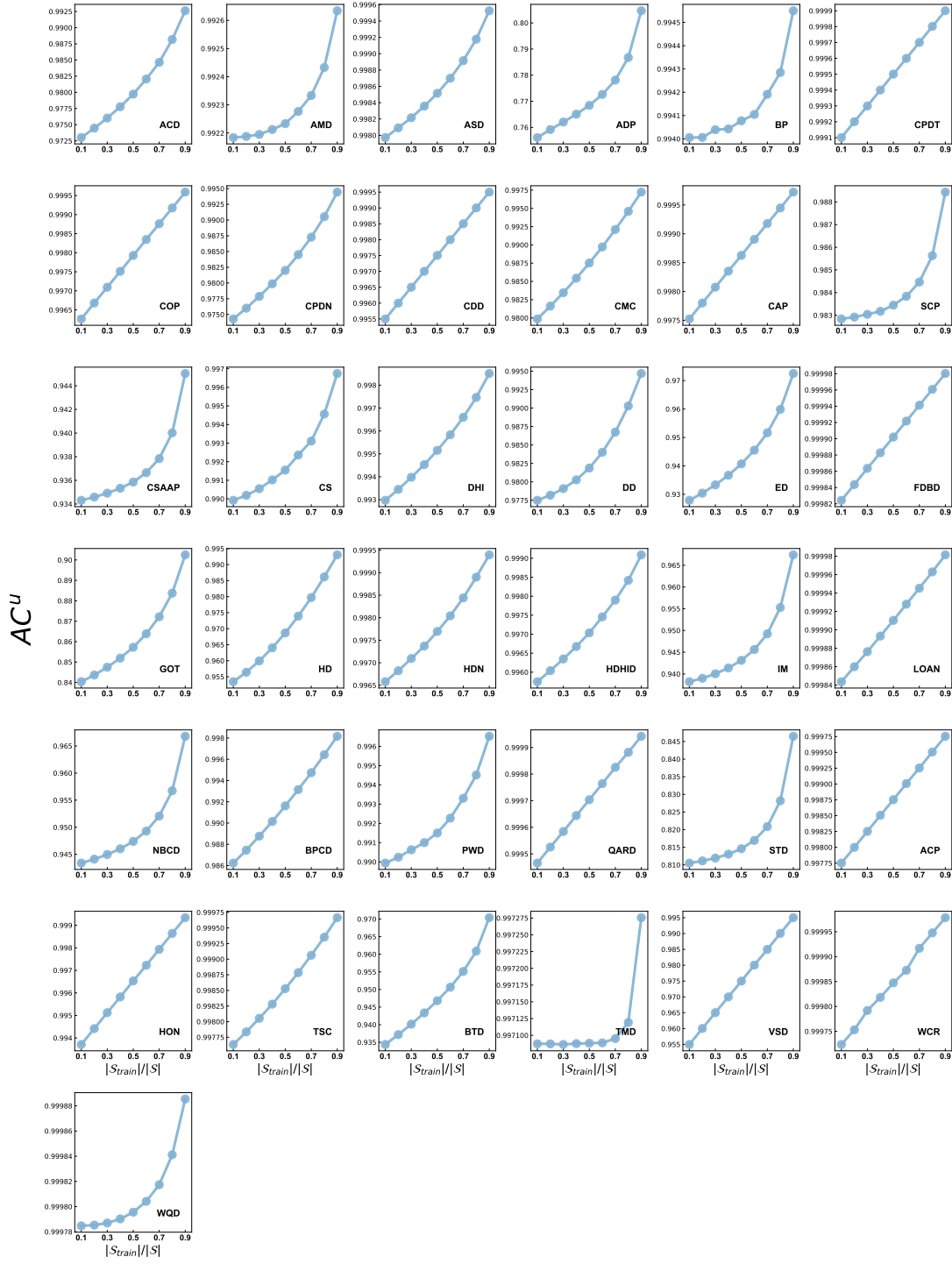


Figure S58. Upper bound of accuracy (AC^u) for 37 additional datasets under different random divisions. In these panels, dots correspond to numerical results derived from the data divisions, while lines represent the theoretical predictions adjusted for the respective division ratios (see Eqs. 54 and 56).

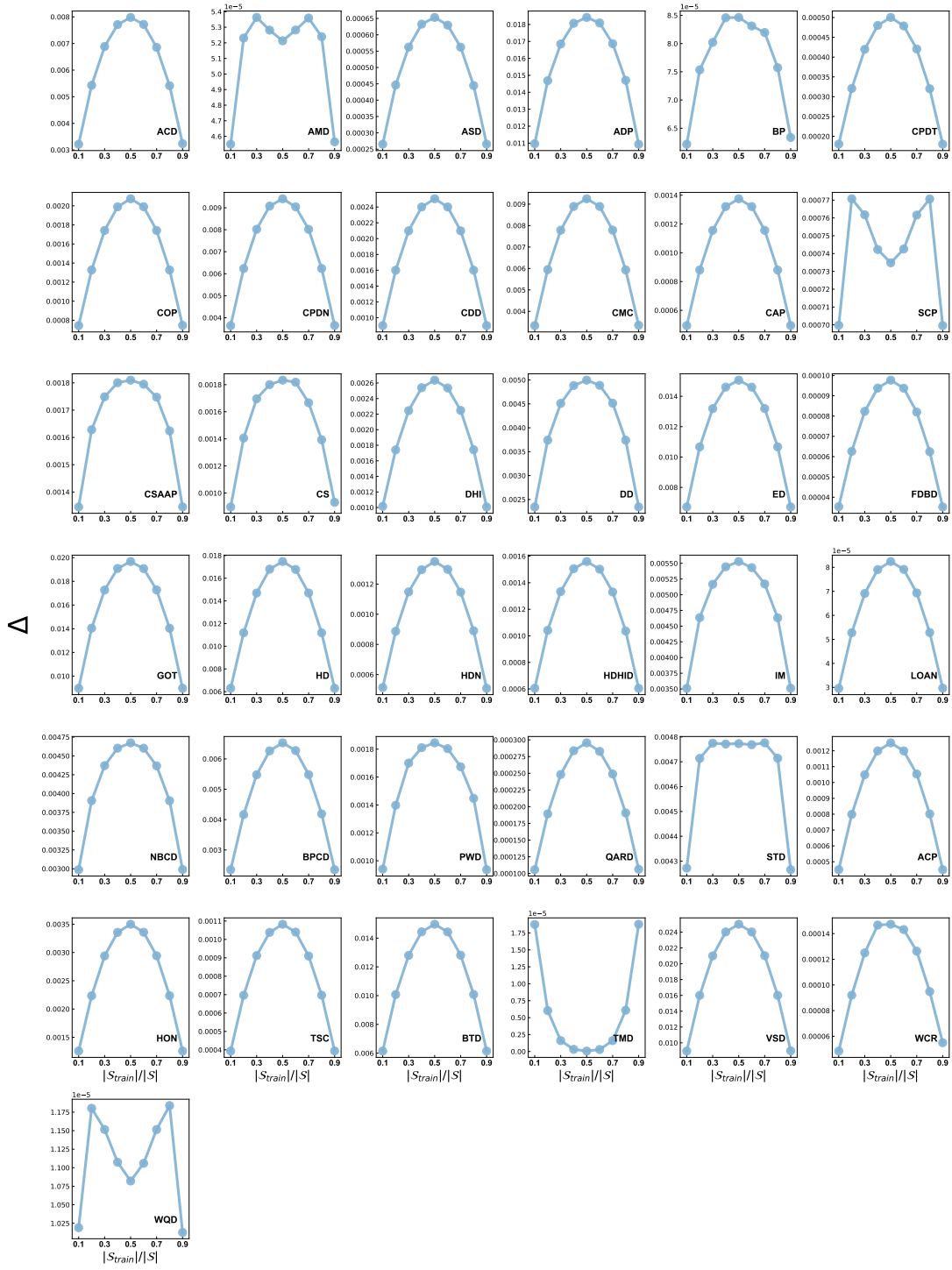


Figure S59. Anticipated optimal errors (Δ) for 37 additional datasets under different random divisions. In these panels, dots correspond to numerical results derived from the data divisions, while lines represent the theoretical predictions adjusted for the respective division ratios (see Eqs. 54 and 56).



Figure S60. The $AR^u_{k_0}$ versus the optimal k_0 feature subset in feature selection (blue lines and dots) for 37 additional datasets. After we selected the optimal k_0 feature subset, we would use the feature extraction skill (LDA) to create new extracted features and add them into the original k_0 feature one by one (see red lines and dots).

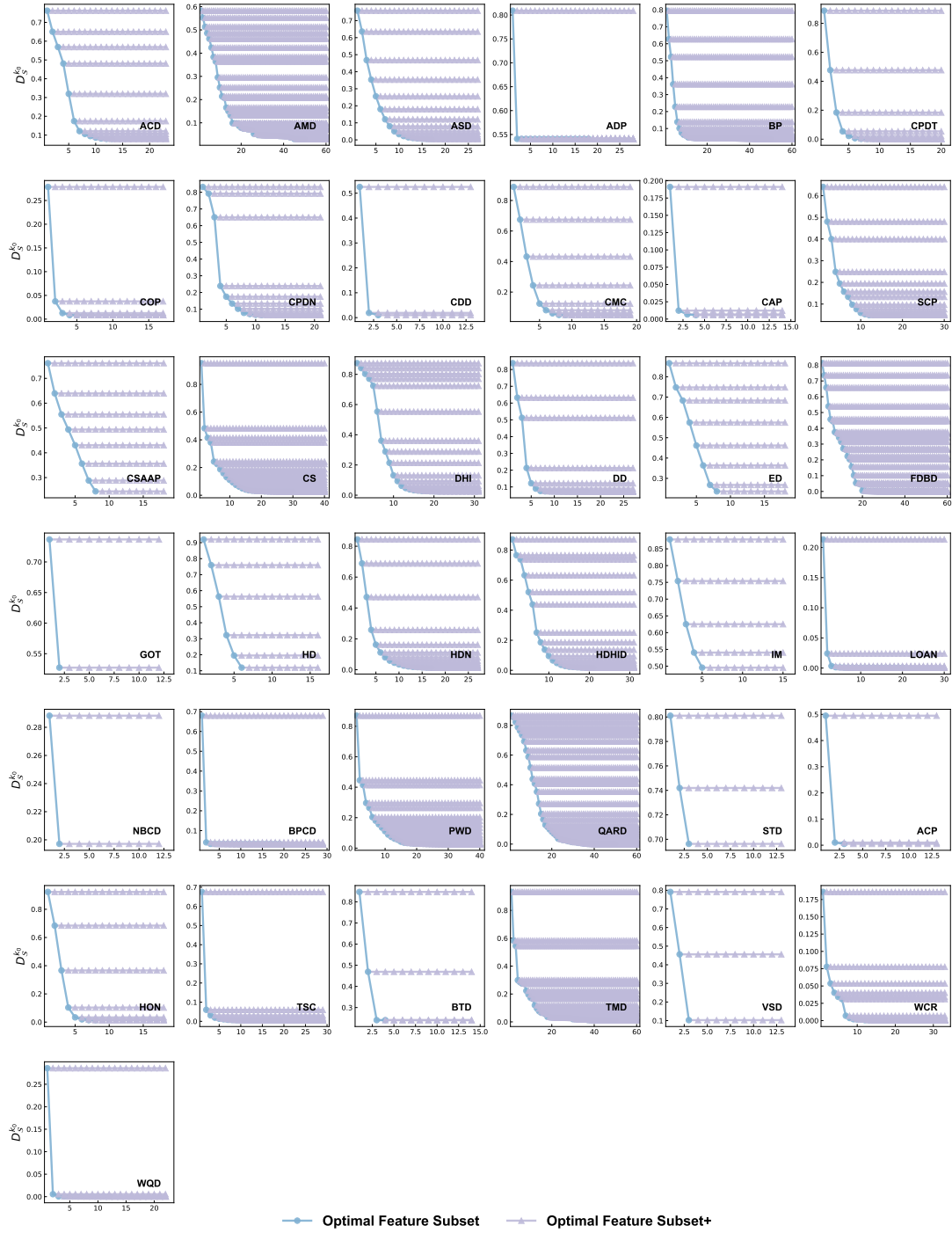


Figure S61. The $D_S^{k_0}$ versus the optimal k_0 feature subset in feature selection (blue lines and dots) for 37 additional datasets. After we selected the optimal k_0 feature subset, we would use the feature extraction skill (LDA) to create new extracted features and add them into the original k_0 feature one by one (see red lines and dots)